

First Edition

Hands-On Python Mastery: From Basics to Advanced

Exploring Core Concepts and Tools





This element signifies a general note.



This element signifies a tip or suggestion.



This element indicates a warning or caution.

Preface.....

Table of Content

Topic 1: Python Basics and Setup	5
1.1) Introduction to Python	5
1.2) Installing & Working with Python IDLE	6
1.3) Configuring Environmental Variables - Command Window	6
1.4) Working with libraries	15
Topic 2: Python Programming Fundamentals	15
2.1) Simple Arithmetic	15
2.2) Introduction to Strings	18
2.3) Indexing and Slicing with string	21
Fig 2.3.6 Negative Indexing Code Output	22
2.4)String Properties and Method	23
2.5)Print Formatting in Strings	27
Topic 3: Python Data Structures and Control Flow	30
3.1) Lists in Python	30
3.2) Dictionaries in python	32
3.3) Tuples in python	34
3.4) Sets in python:	36
3.5) Booleans in python	38
Fig 3.5.2 Booleans In Python Output	41
3.6) I/O with basic file in python	41
Fig 3.6.3 I/O With Basic File In Python Output	42
3.7) Python Object and Data Structure	43
3.8) Comparison Operator in Python	45
3.10) If Elif and Else Statement in Python	48
3.11) For Loop in Python	50
3.12) While loop in python	52

Fig 3.12.4 While loop in python Output	53
3.13) Useful Operator in Python	54
3.14) List Comprehensions in Python	56
Topic 4: Python Functions and Classes	57
4.1) Methods and Python Documentation	57
4.2) Introduction to Function	58
4.3) Basics of Python Function	59
4.4) Logic with Python Functions	61
4.5) Tuple Unpacking with Python Function	62
4.6) *args and **Kwags in Python	64
4.7) Lambda Expression,Map and Filter Functions	66
4.8) Attributes and Class Keywords	69
4.9) Class Object Attributes and Methods	71
4.10) Inheritance and Polymorphism	73
4.11) Special (Magic/Dunder) Methods	77
Topic 5: Advanced Python Concepts	80
5.1) Modules and packages	80
5.2) name and “main”	82
5.3) Error and exception handling	84
5.4) Pylint Overview	86
5.5) Decorators with Python Overview	88
5.6) Generators with Python	90
5.7) Python Collection Modules	93
5.8) Opening and Reading Files and Folders	96
5.9) Python Math and Random Module	100
5.10) Python Debugger	103
5.11) Python Regular Expressions	106
5.12) Timing Your Python Code	110

5.13) Zipping and Unzipping files using Python	113
5.14) Setting Up Web Scraping Libraries	116
5.15) Grabbing a Title	117
5.16) Grabbing an Image	118
5.17) Book Examples	119
5.18) Introduction to Images with Python	121
5.19) Working with CSV files in python	123
5.20) Working with PDF Files in Python	129
5.21) Sending Emails with Python	133
5.22)Receiving Gmail with Python	141
5.23) Creating a BMI Calculator	144
5.24) Creating a Password generator	146
5.25)Hangman Game in Python	148
5.26) Creating a Calculator using Code	150

Topic 1: Python Basicand Setup

1.1) Introduction to Python

Python is a high-level, versatile, and interpreted programming language known for its simplicity and readability. Created by Guido van Rossum and first released in 1991, Python has since become one of the most popular programming languages in the world. Its design emphasizes code readability and a clear, concise syntax, making it an excellent choice for beginners and experienced developers alike.

1.2) Installing & Working with Python IDLE

To begin your Python journey, you can either search for "Python download" in your web browser or simply follow this link: <https://www.python.org/downloads/>. It is recommended to download Python version 3.10.6 as it is a stable and widely compatible release. While you may be tempted to install the latest version, it's generally advisable to avoid doing so as the newest releases might not be as stable or compatible with certain libraries and packages.. Once you've downloaded the installer, remember to grant the necessary permissions for the installation process to proceed.

1.3) Configuring Environmental Variables - Command Window

Environment variables are predetermined values that are typically used to provide the ability to configure a value in your code from outside of your application. Before creating an environment variable, you need to know where the Python application is located on your system.



Fig 1.3.1 To Select Path Image

This information is crucial because setting the environment variable correctly ensures that the operating system can find and execute Python scripts. On my system, Python is installed in a specific location, as shown in Figure . By knowing this path, I can set the appropriate environment variables to ensure seamless execution and module access for Python programs.

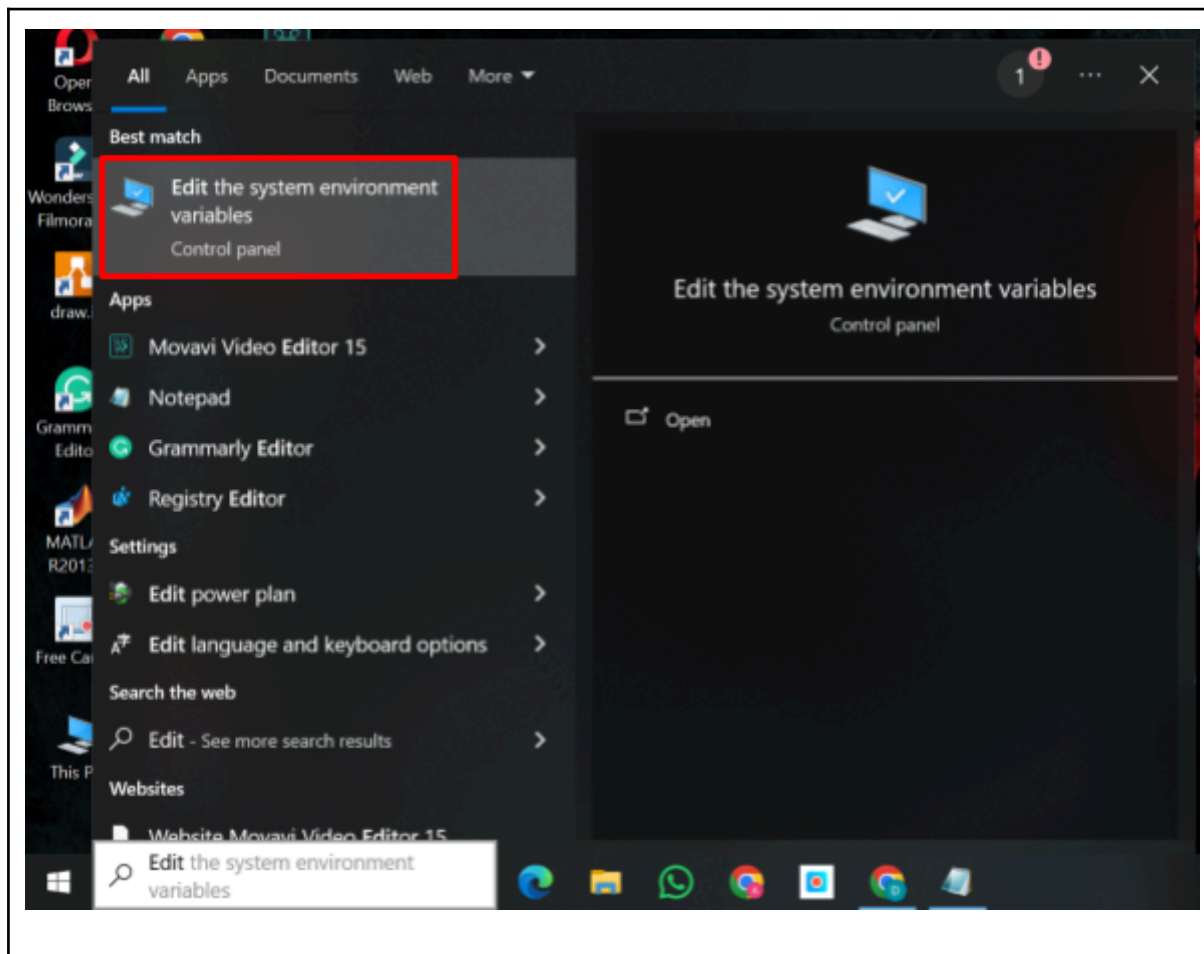


Fig 1.3.2 Environment Variable Set

To proceed, refer to Figure 1.3.2 for the "Edit system environment variables" option. Before setting the environment variable, copy your Python script's location. Access system settings, then "Edit system environment variables" . This step customizes variables for your Python app, ensuring accurate path information. Refer to Figure 1.3.2 for configuring system environment variables. Clicking "Settings" in the "Edit System Environment Variables" section reveals additional options. These settings let you customize variables, paths, and system configurations. Accessing "Settings" ensures your system aligns with your preferences, including adjustments for Python app performance. This simplifies tailoring your environment.

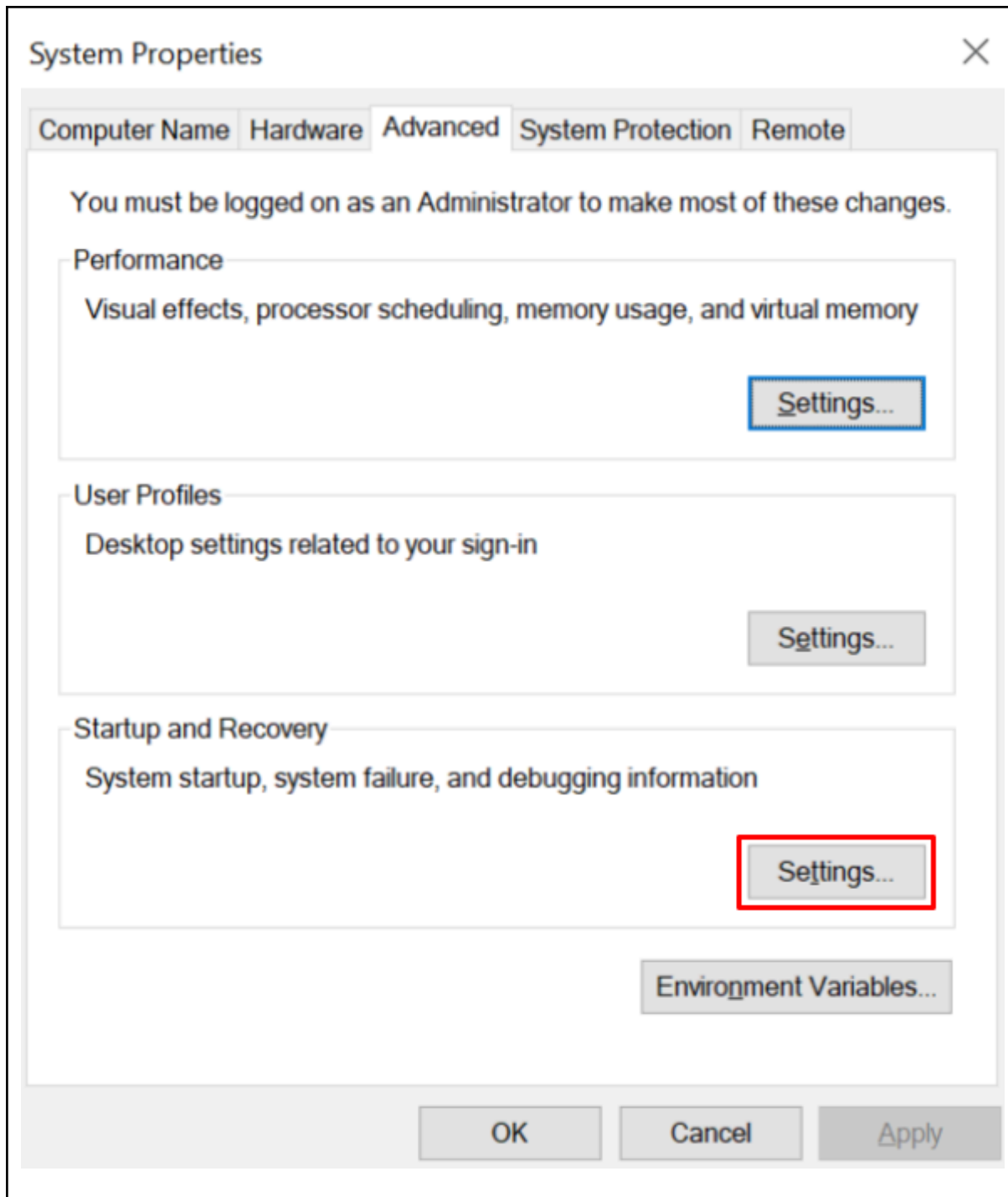


Fig 1.3.3 Environment Variable Set

Once you have accessed the "Edit System Environment Variables" section, a new tab will open, as illustrated in Figure 1.3.3. Within this tab, you will find a list of system variables that can be modified. To proceed, locate the "Path" variable among the system variables. Clicking on the "Path" variable, as depicted in Figure 1.3.3, will allow you to edit the system paths. This step is crucial for configuring the environment specifically for your Python

application. By selecting "Path," you can ensure that your application has access to the necessary directories and files, enabling it to function correctly.

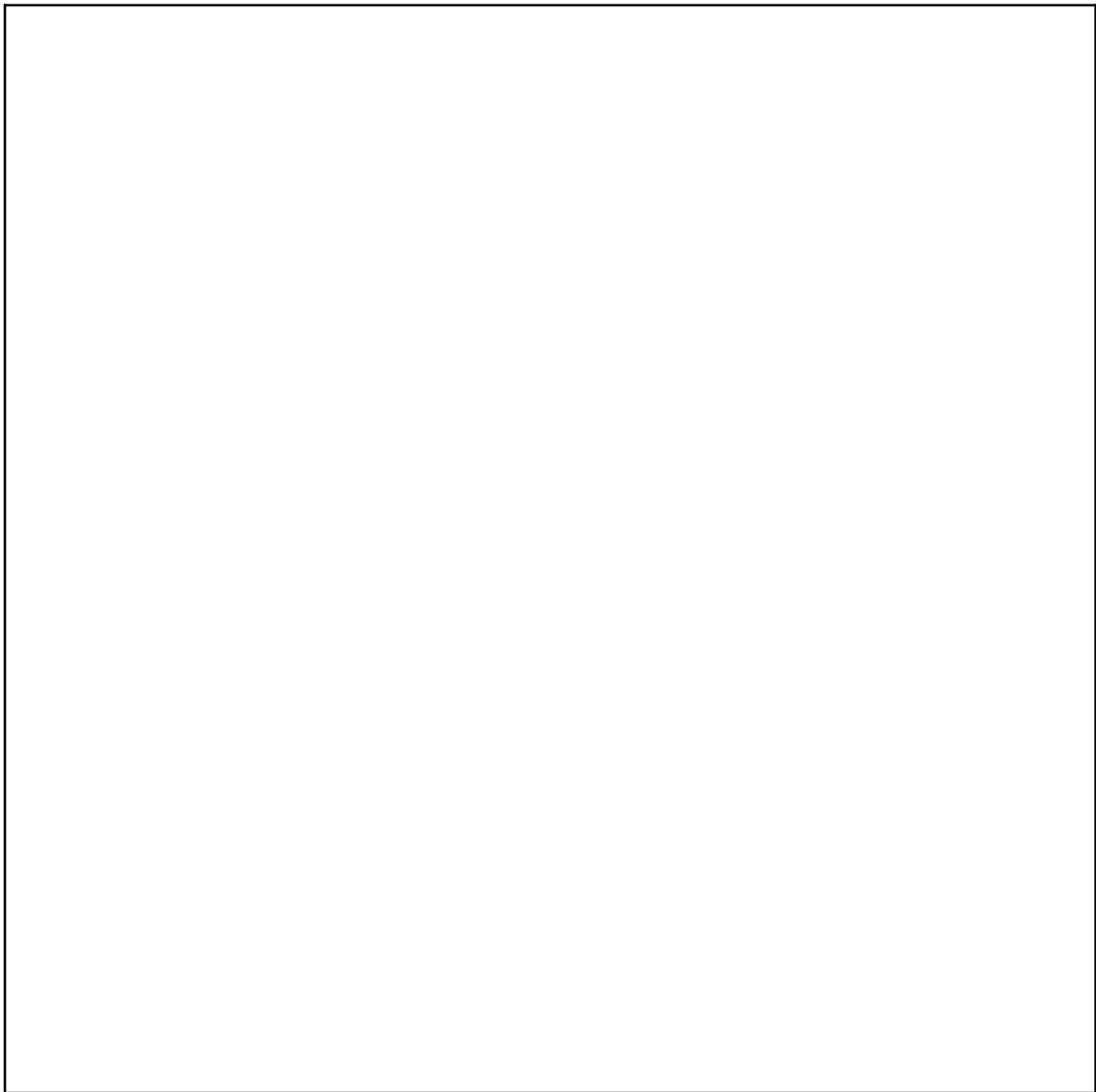


Fig 1.3.4 Environment Variable Set

To proceed, refer to Figure 1.3.4 for the "Edit system environment variables" option. Before setting the environment variable, copy your Python script's location. Access system settings, then "Edit system environment variables" . This step customizes variables for your Python app, ensuring accurate path information. Refer to Figure 1.3.4 for configuring system environment variables. Clicking "Settings" in the "Edit System Environment Variables" section reveals additional options. These settings let you customize variables, paths, and system configurations. Accessing "Settings" ensures your system aligns with your preferences,

including adjustments for Python app performance. This simplifies tailoring your environment.

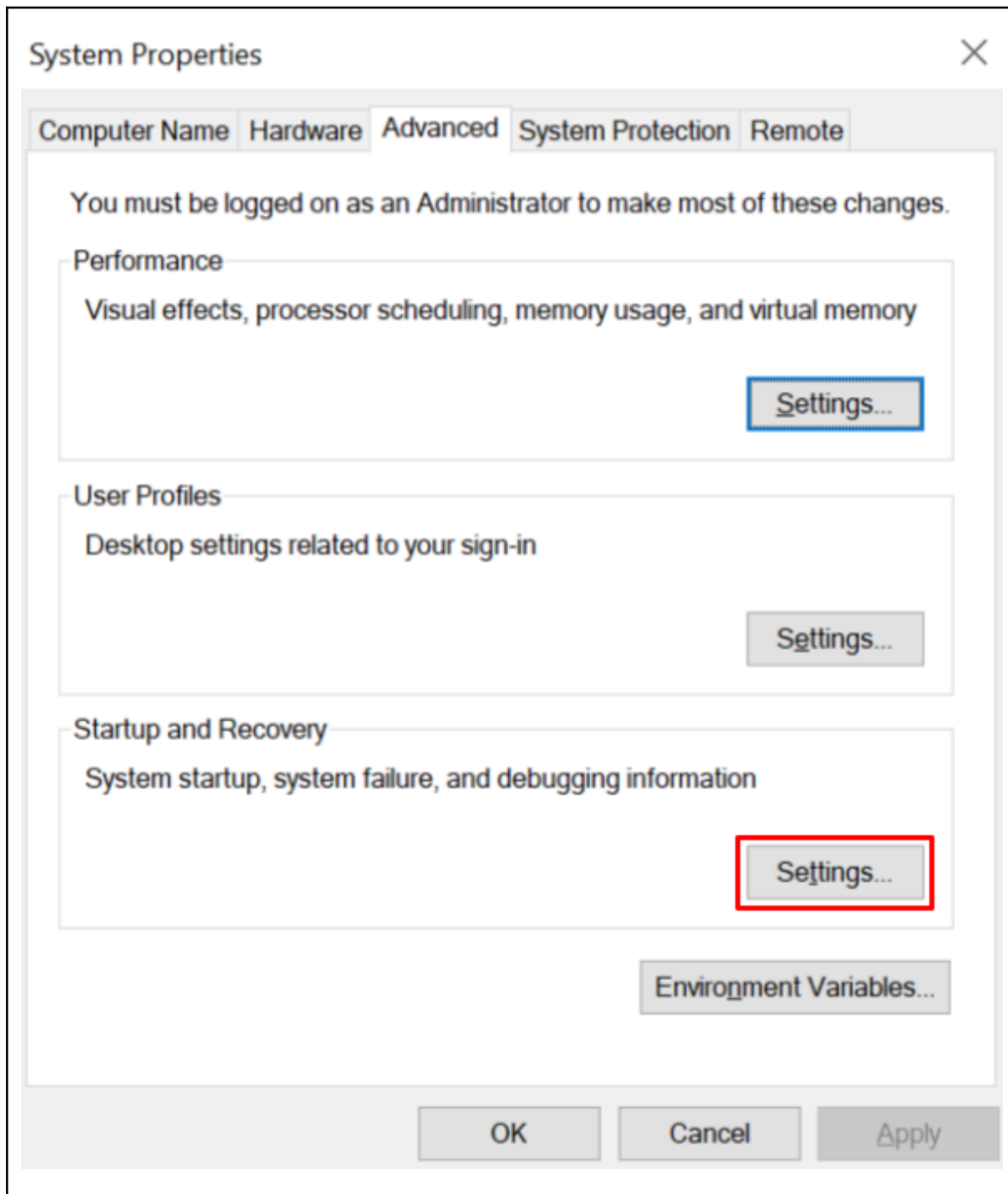


Fig 1.3.5 Environment Variable Set

Once you have accessed the "Edit System Environment Variables" section, a new tab will open, as illustrated in Figure 1.3.5. Within this tab, you will find a list of system variables that can be modified. To proceed, locate the "Path" variable among the system variables. Clicking on the "Path" variable, as depicted , will allow you to edit the system paths. This

step is crucial for configuring the environment specifically for your Python application. By selecting "Path," you can ensure that your application has access to the necessary directories and files, enabling it to function correctly.

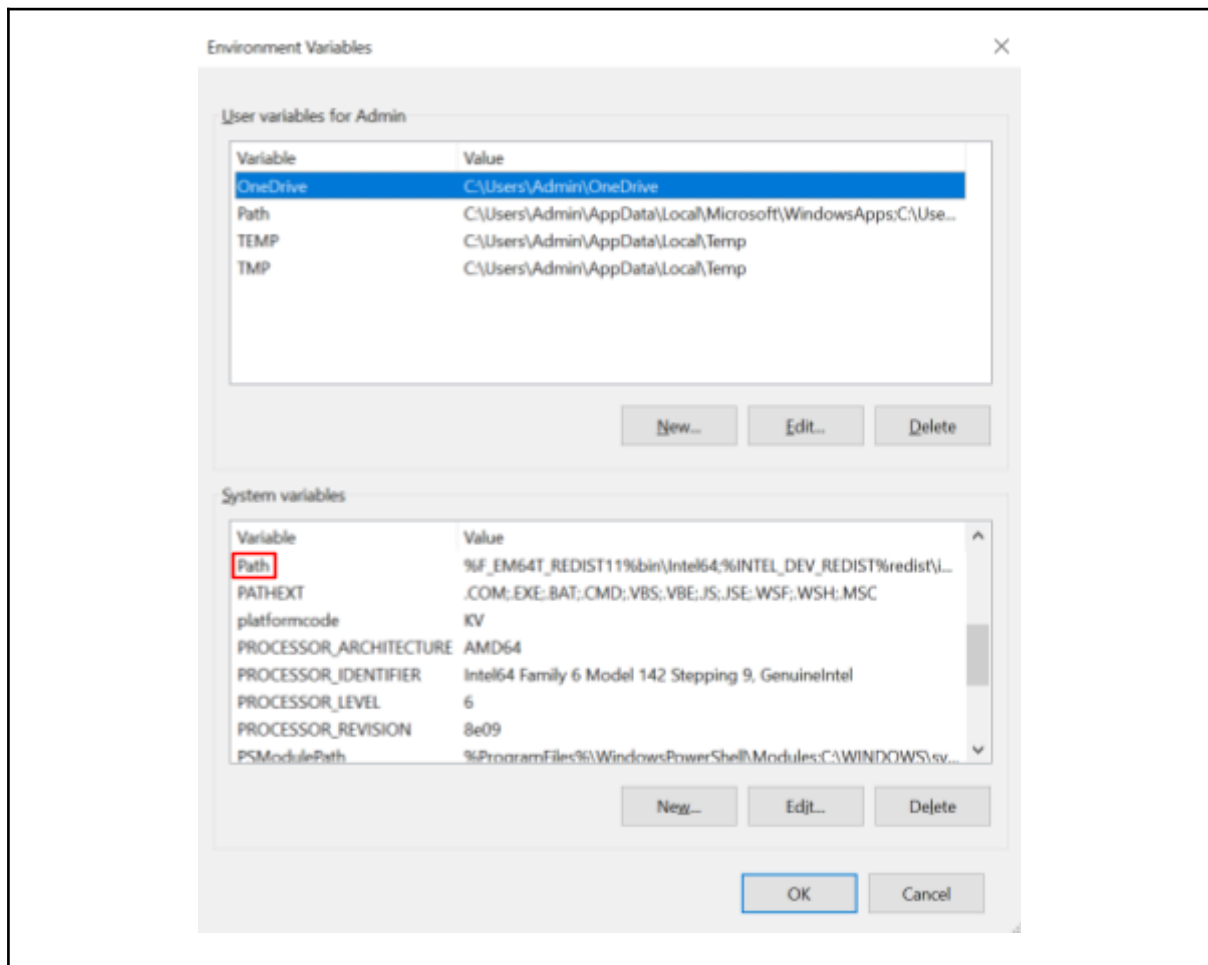


Fig 1.3.6 Environment Variable Set

Next, select "New," as shown in Figure 1.3.7, and paste the Python location and Python script location into the environment variable as mentioned in Figure 1.3.8

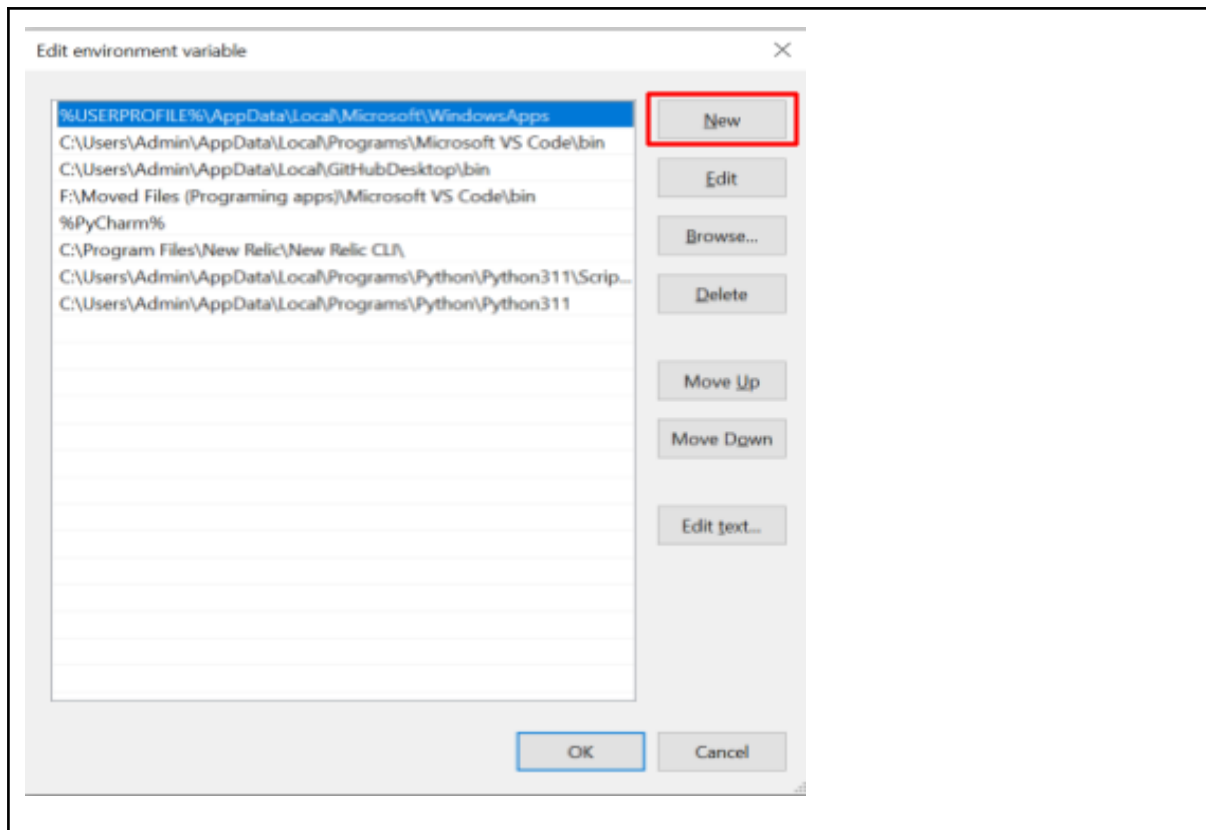


Fig1.3.7 Environment Variable Set

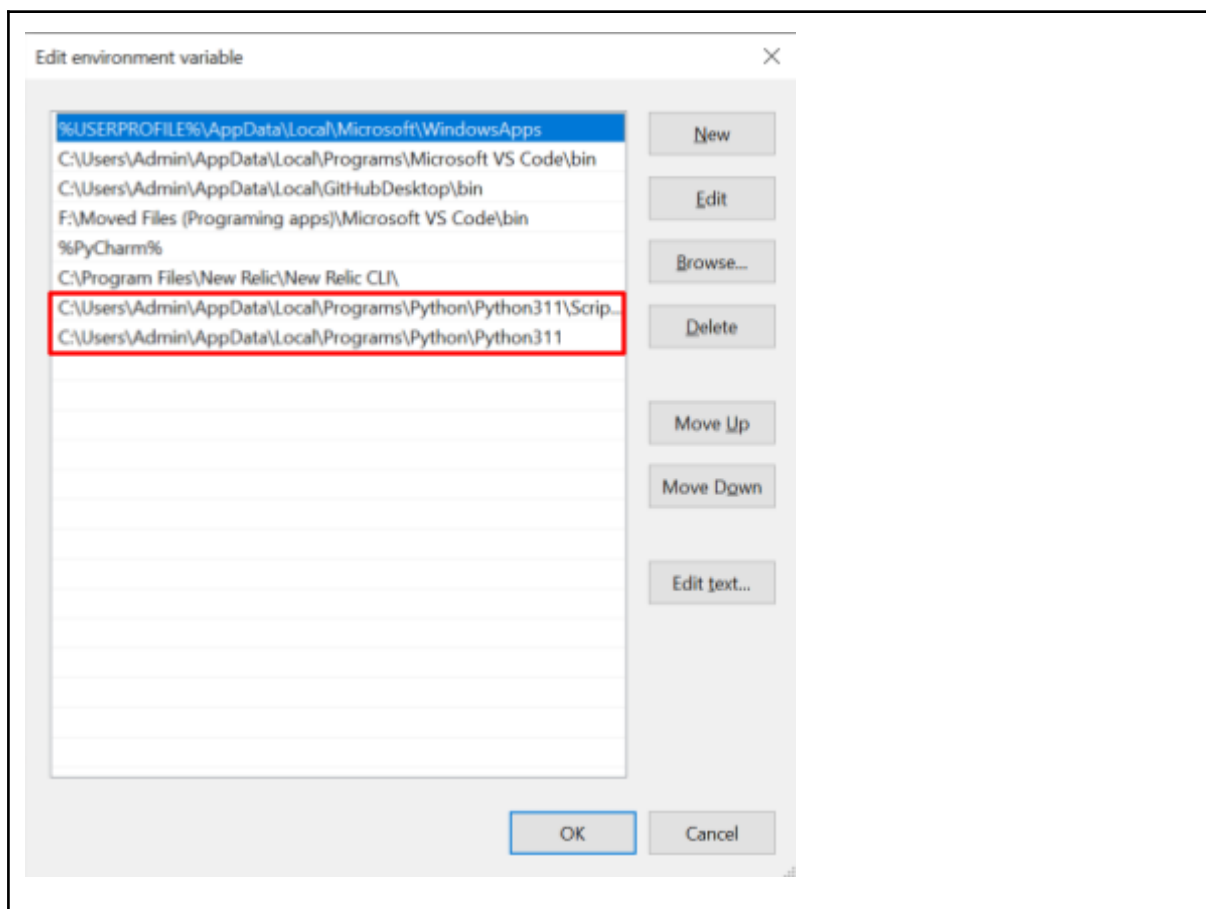


Fig 1.3.8 Environment Variable Set

Upon completing the previous steps, you can verify that the PATH environment variable has been successfully updated by accessing the System Properties dialog once again. Within this dialog, navigate to the Advanced tab and click on the Environment Variables button. Locate the PATH variable within the System Variables section and click on Edit. You should now observe that the Value field contains the recently added paths to both the Python installation directory and the directory containing your Python script. This confirmation validates that your system can now accurately locate and execute Python scripts from any directory. To confirm that you have successfully configured the environment variable for Python, open the Command Prompt (CMD) by searching for "cmd" in the Start menu or pressing Win + R and typing "cmd." mentioned in Figure 8. Once the Command Prompt window is open, simply type "python" and press Enter.



If Python is correctly added to your system's environment variables, launching the Python interpreter in the Command Prompt will open the Python shell, allowing you to run Python commands and scripts interactively, confirming system-wide access to Python.

If Python has been correctly added to your system's environment variables, the Python interpreter will launch, and you will be presented with the Python shell, as illustrated in Figure 9. This shell allows you to interactively execute Python commands and scripts directly within the Command Prompt, indicating that Python is now accessible system-wide.

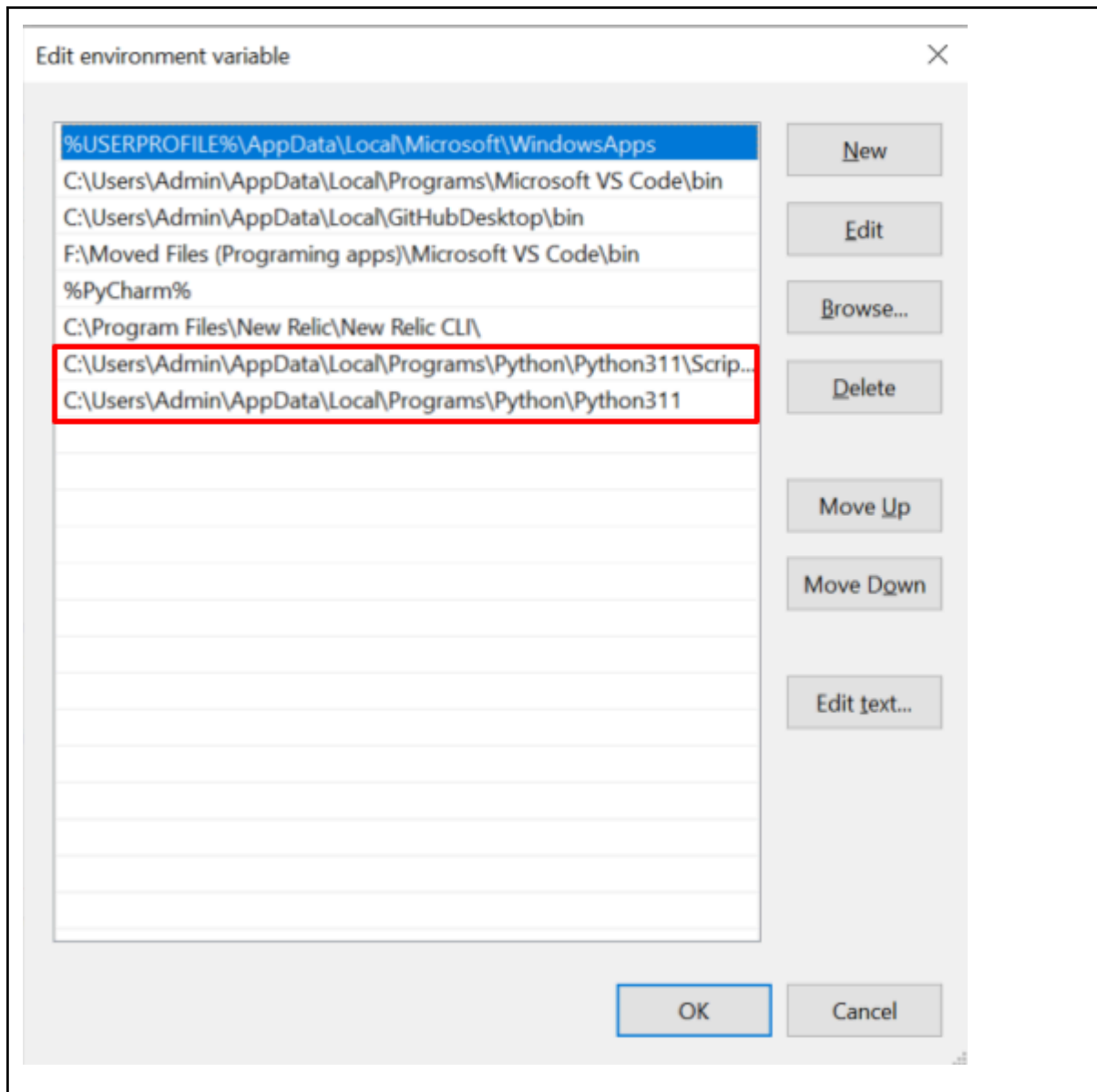


Fig 1.3.8 Environment Variable Set

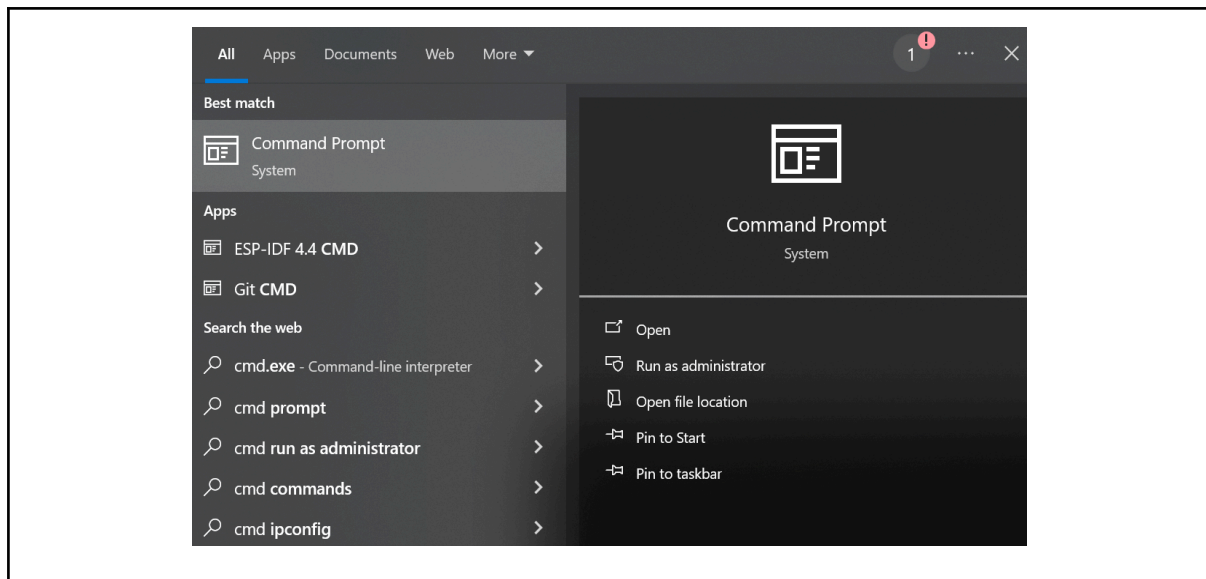


Fig 1.3.9 Python Installation Check

1.4) Working with libraries

In Python, libraries are collections of pre-written code and modules that provide a wide range of functionalities to simplify and enhance the development process. These libraries are designed to be reusable and save time for developers by offering ready-to-use functions and tools for various tasks. Python has a vast ecosystem of libraries catering to different domains, such as NumPy for numerical computations, Pandas for data manipulation and analysis, Matplotlib for data visualization, Requests for making HTTP requests, TensorFlow for machine learning, and Flask for web development, among many others. The ability to leverage these libraries empowers Python programmers to build complex applications efficiently, as they can tap into the collective effort of the community and benefit from the continuous improvement and updates provided by library maintainers.

Topic 2: Python Programming Fundamentals

2.1) Simple Arithmetic

Simple arithmetic is the fundamental branch of mathematics that deals with basic numerical operations.

It involves performing the four fundamental operations on numbers:

1. Addition
2. subtraction
3. Multiplication
4. division

Addition

Addition is the process of combining two or more numbers to obtain a total or sum. For example, adding 2 and 3 yields 5 ($2 + 3 = 5$). In this program, we have directly given two numbers and using + operator we add the two numbers (Operands) and we print the value. Here we store the value of 10 in the variable a and value 15 in the variable b. Then we add the a+b variables and put them in the c variable. So when we print the c variable, we have the value of $10 + 15 = 25$.

```
a = 10
b = 15
print(a+b)
```

Figure 2.1.1 add

Subtraction:

Subtraction is the process of finding the difference between two numbers. It represents the operation of taking away one number from another. For example, subtracting 3 from 7 gives 4 ($7 - 3 = 4$). mention as fig 2.1.2

```
a = 10
b = 5
c = a-b
print(c)
```

Fig.2.1.2 subtraction code

Multiplication:

Multiplication is the process of repeated addition or combining equal groups. It finds the total of multiple equal numbers. For example, multiplying 4 by 3 gives 12 ($4 * 3 = 12$).

```
a = 10
b = 5
c = a*b
print(c)
```

Fig 2.1.3 Multiplication code

Division:

Division is the process of dividing a quantity into equal parts or finding how many times one number can fit into another. For example, dividing 10 by 2 gives 5 ($10 \div 2 = 5$).

These basic arithmetic operations are the building blocks of more complex mathematical calculations.

```
a = 10
b = 5
c = a/b
print(c)
```

Fig 2.1.4 Division code

2.2) Introduction to Strings

A string is a fundamental data type in computer programming and is used to represent text or sequences of characters. It is one of the most commonly used data types in programming languages like Python, JavaScript, Java, C++, and many others. Strings are used to store and manipulate textual data, such as words, sentences, and characters. Here are some key characteristics and concepts related to strings:

Sequence of Characters

Strings are a sequence of characters, balancing the benefits of stability and security against the need for flexibility and adaptability.

Enclosed in Quotes

Enclosed in Quotes: Strings are typically enclosed in either single quotes (' '), double quotes (" "), or triple quotes (""" """) depending on the programming language.

For example:

Single-quoted string in Python: 'Hello, World!'

Double-quoted string in JavaScript: "Welcome to programming."

String Manipulation: You can perform various operations on strings, including concatenation (combining two or more strings), substring extraction, searching, replacing, and more.

Escape Sequences

Escape sequences are special character combinations in programming and text formatting used to represent characters that are otherwise difficult to include directly in a string. Typically starting with a backslash (\), escape sequences allow for the insertion of non-printable or reserved characters into a text string. Common examples include '\n' for a newline, '\t' for a tab, '\\' for a literal backslash, and '\"' for a double quote. These sequences are crucial in many programming languages as they enable developers to format strings more effectively and control the output of text, ensuring proper display and functionality in various contexts. For instance, in JSON or XML data, escape sequences are used to handle special characters that might otherwise disrupt the data structure. Overall, understanding and utilizing escape sequences is essential for effective string manipulation and data representation in software development.

String Length

String length in computer science refers to the number of characters in a string, including letters, numbers, symbols, and spaces. It's a fundamental aspect of string manipulation and is crucial for various operations, such as data validation, formatting, and storage. Accurately determining and handling string length is essential for tasks like ensuring user input meets required criteria, optimizing memory usage, and preventing buffer overflow vulnerabilities. In many programming languages, built-in functions or methods, such as 'strlen' in C, 'length' in Java, or 'len' in Python, are available to retrieve the length of a string efficiently. Understanding and managing string length is a key skill for developers in creating robust and efficient software applications.

Indexing

Indexing is a crucial process in information retrieval systems, databases, and search engines that enhances the speed and efficiency of data retrieval. By creating an index, a system organizes data in a structured manner, allowing for quick access and retrieval without scanning every piece of information. This process involves creating a reference or a pointer to data entries, similar to how an index at the back of a book helps locate specific content quickly. In databases, indexing can significantly improve query performance by reducing the amount of data that needs to be processed. Effective indexing strategies are essential for managing large volumes of data, ensuring that users can retrieve relevant information promptly and efficiently.

String Operations

String operators in programming are essential tools that facilitate the manipulation and handling of text data. These operators allow for various operations such as concatenation, comparison, slicing, and formatting of strings. Concatenation, typically achieved using the '+' operator, merges two or more strings into a single string. Comparison operators, like '==' and '!=', check for equality or inequality between strings. Slicing enables extracting specific portions of a string, using syntax like 'string[start:end]' to retrieve a substring. Additionally, formatting operators, such as '%' or the 'format()' method in Python, allow for the dynamic insertion of variables into strings, enhancing their flexibility and utility in programming. Overall, string operators are pivotal in efficiently managing and manipulating textual data across various programming languages.

String Interpolation

String interpolation is a programming technique used to embed variables directly within a string, simplifying the process of dynamically generating text. This method enhances code readability and maintainability by allowing developers to create more concise and expressive string expressions. In many modern programming languages, such as Python, JavaScript, and C#, string interpolation is implemented using specific syntax. For example, in Python, an f-string (formatted string) is used by prefixing the string with an 'f' and placing variables within curly braces ' {}'. This allows for the seamless integration of variables and expressions directly within the string, making the code more intuitive and easier to debug. Overall, string interpolation is a powerful feature that streamlines the construction of dynamic strings in various programming tasks.

Unicode Support

String Unicode support is crucial in modern computing as it enables the representation and manipulation of text from diverse languages and symbol sets in a standardized manner. Unicode, a universal character encoding standard, allows for the consistent encoding, representation, and handling of text. This comprehensive standard includes characters from almost all writing systems, as well as symbols, emojis, and control characters. By supporting Unicode, programming languages and software systems ensure that text data is interoperable across different platforms and locales, facilitating global communication and the development of internationalized applications. Consequently, Unicode support in strings is essential for maintaining data integrity, avoiding encoding errors, and providing a seamless user experience in a multilingual world.

For further understanding and examples on string manipulation and Unicode support in Python, you can refer to this

https://github.com/quantanics2018/Python/blob/main/Introduction_to_Strings.py

2.3) Indexing and Slicing with string

In this example, we have assigned the variable 'a' with the string "Adhiban". By slicing the string variable 'a', we can access specific portions of the string using index positions. The index position refers to the current position of an element within the string, starting at 0 and

ending at $n-1$, where n is the total number of characters in the string. For the given name "Adhiban", which consists of 7 characters, the index values range from 0 to 6.

0	1	2	3	4	5	6	7	8
C	#		C	O	R	N	E	R
-9	-8	-7	-6	-5	-4	-3	-2	-1

Fig 2.3.1 indexing

```
a = "Adhiban"
print(a[0])
print(a[6])
```

Fig 2.3.2 Indexing Code

Here the Letter A represented the first index position.

Index value six represents the last value

Here , We have printed the Letter A and n using their respective index value.

```
A
n
```

Fig 2.3.4 Indexing Code Output

Here , we use a negative index to access the element of a specific position.

Generally negative index starts from the end of the element as 0 th position is occupied by the first element

```
a = "Adhiban"  
print(a[-1])  
print(a[-4])
```

Fig 2.3.5 Negative Indexing Code

```
n  
i
```

Fig 2.3.6 Negative Indexing Code Output

2.4)String Properties and Method

Strings in Python are sequences of characters, and they come with a variety of properties and methods that allow you to manipulate and work with text data. Here are some common string properties and methods:

String Properties

String is a lightweight, strong, and flexible material with diverse properties making it indispensable in various applications. It can be made from natural fibers like cotton, silk, and wool, or synthetic fibers such as nylon and polyester, each type offering unique characteristics. Natural strings are typically biodegradable and have a pleasant texture, while synthetic strings are known for their durability and resistance to environmental factors. Strings can be elastic or inelastic, impacting their use in applications ranging from musical instruments, where precise tension is crucial, to sports equipment like tennis rackets, which require high strength and resilience. The versatility of string materials also extends to industrial uses, such as in the manufacturing of ropes, fishing lines, and sewing threads, highlighting their essential role in both everyday items and specialized tools.

String Methods

The users to manipulate, search, and analyze text data effectively. The code can be found at In Python, strings offer a variety of useful methods for manipulation and inspection. The

`lower()` and `upper()` methods convert a string to lowercase or uppercase, respectively. For capitalizing strings, `capitalize()` changes only the first letter of the string to uppercase, while `title()` capitalizes the first letter of each word. Whitespace removal is handled by `strip()`, `lstrip()`, and `rstrip()`, which remove whitespace from both ends, the beginning, or the end of the string, respectively. The method divides a string into a list of substrings based on a specified delimiter, and combines elements of an iterable (like a list) into a single string using the string as a delimiter. Substring replacement is achieved with `replace(old, new)`, which replaces occurrences of an old substring with a new one. To locate substrings, `find(substring)` and `index(substring)` return the index of the first occurrence of a substring, though `index()` raises an error if the substring is not found. The `count(substring)` method counts non-overlapping occurrences of a substring. Checking the start or end of a string can be done with `startswith(prefix)` and `endswith(suffix)`. For character type inspection, `isalpha()`, `isdigit()`, `isalnum()`, `isspace()`, `islower()`, and `isupper()` methods verify if the string contains alphabetic characters, digits, alphanumeric characters, whitespace, lowercase characters, or uppercase characters, respectively. The `len()` function returns the length of the string. Formatting is handled by `format()`, which substitutes placeholders with specified values. The `splitlines()` method splits a string into a list of lines based on newline characters, and `encode(encoding)` converts the string into bytes using a specified encoding.

This Python code offers a comprehensive exploration of fundamental string properties and methods. It starts by emphasizing the immutability of strings, showing how attempting to modify a character within a string leads to an error. The code then delves into string transformation methods, such as `.lower()`, `.upper()`, `.capitalize()`, and `.title()`, which convert text to lowercase, uppercase, capitalize the first letter, and capitalize the first letter of each word, respectively. String manipulation methods are demonstrated next, including `.strip()` for removing leading and trailing whitespaces and `.split(",")` for breaking a comma-separated string into a list. The reverse operation, joining a list into a string with a specified delimiter, is exemplified by `.join()`. The code also exhibits substring replacement using `.replace()`, followed by the exploration of substring search methods—`.find()` and `.index()`. While both methods find the index of the first occurrence of a substring, `.index()` raises an error when the substring isn't found. The code showcases `.count()` for counting occurrences of a character in a string and `.startswith()` and `.endswith()` for checking if a string begins or ends with a specific substring. Furthermore, the code examines string testing methods like `.isalnum()`, `.isdigit()`, `.isalpha()`, `.isspace()`, `.islower()`, and `.isupper()`, which verify various string

properties, such as alphanumeric characters, digits, alphabetic characters, whitespace, lowercase, and uppercase status. Other methods, such as `len()` for determining string length, `.format()` for dynamic string formatting, `.splitlines()` for splitting a multiline string into a list of lines, and `.encode("utf-8")` for encoding a string into bytes using a specified encoding, are also demonstrated. In essence, this code snippet serves as an educational resource for understanding the extensive capabilities of Python's string properties and methods, enabling https://github.com/quantanics2018/Python/blob/main/String_method.py

```

# String immutability
string_immutable = "Python is great!"
# This will result in an error:
# string_immutable[0] = 'J'

# String transformation methods
text = "Hello, World!"
lower_text = text.lower()
upper_text = text.upper()
capitalized_text = text.capitalize()
title_text = text.title()
print("Original:", text)
print("Lowercase:", lower_text)
print("Uppercase:", upper_text)
print("Capitalized:", capitalized_text)
print("Title:", title_text)

# String manipulation methods
whitespace_text = "    whitespace    "
stripped_text = whitespace_text.strip()
print("Stripped:", stripped_text)

comma_sep_text = "apple,banana,orange"
split_fruits = comma_sep_text.split(",")
print("Split:", split_fruits)

fruits = ["apple", "banana", "cherry"]
joined_text = ", ".join(fruits)
print("Joined:", joined_text)

replaced_text = text.replace("Hello", "Hi")
print("Replaced:", replaced_text)

index1 = text.find("World")
index2 = text.index("World")
print("Index (find):", index1)
print("Index (index):", index2)

count_a = text.count("o")
print("Count of 'o':", count_a)

# String testing methods
startsWithHello = text.startswith("Hello")
endsWithWorld = text.endswith("World")
print("Starts with 'Hello':", startsWithHello)
print("Ends with 'World':", endsWithWorld)

```

Fig 2.4.2 String Properties and Method Code

```
isAlphaNumeric = text.isalnum()
isDigit = text.isdigit()
isAlpha = text.isalpha()
isSpace = text.isspace()
isLower = text.islower()
isUpper = text.isupper()
print("isalnum:", isAlphaNumeric)
print("isdigit:", isDigit)
print("isalpha:", isAlpha)
print("isspace:", isSpace)
print("islower:", isLower)
print("isupper:", isUpper)

# Other string methods
string_length = len(text)
formatted_text = "My name is {}. I am {} years old.".format("Alice", 30)
split_lines_text = "Line 1\nLine 2\nLine 3".splitlines()
encoded_text = text.encode("utf-8")

print("String length:", string_length)
print("Formatted text:", formatted_text)
print("Split lines:", split_lines_text)
print("Encoded text:", encoded_text)
```

Fig 2.4.2 String Properties and Method Code

```
Original: Hello, World!
Lowercase: hello, world!
Uppercase: HELLO, WORLD!
Capitalized: Hello, world!
Title: Hello, World!
Stripped: whitespace
Split: ['apple', 'banana', 'orange']
Joined: apple, banana, cherry
Replaced: Hi, World!
Index (find): 7
Index (index): 7
Count of 'o': 2
Starts with 'Hello': True
Ends with 'World': False
isalnum: False
isdigit: False
isalpha: False
isspace: False
islower: False
isupper: False
String length: 13
Formatted text: My name is Alice. I am 30 years old.
Split lines: ['Line 1', 'Line 2', 'Line 3']
Encoded text: b'Hello, World!'
```

Fig 2.4.3 String Properties and Method Output

2.5)Print Formatting in Strings

Method 1: Using % Operator (Older method, less recommended in Python 3.6 and above)

In Python, formatting strings involves incorporating variables or values into a string template to produce formatted strings. There are several methods available to achieve this, including the use of the % operator, the str.format() function, and f-strings (formatted string literals introduced in Python 3.6). Each of these methods offers a way to create dynamic and customizable string outputs. The % operator, which is one of the older methods, uses a format specifier and a tuple to insert values into a string template. The str.format() function, introduced in Python 2.6 and enhanced in Python 3, allows more control and flexibility by using curly braces as placeholders within the string, which are then replaced by the values passed to the format() method. Finally, f-strings provide a more concise and readable way to format strings by prefixing the string with an 'f' and embedding expressions inside curly braces directly within the string. This method is not only more intuitive but also faster, making it the preferred choice for many Python developers. For more information and examples, you can refer to this https://github.com/quantanics2018/Python/blob/main/String_method_1.py

```
name = "John"
age = 30

# Using the % operator for string formatting
formatted_string = "My name is %s and I am %d years old." % (name, age)
print(formatted_string)
```

Fig 2.5.1 String Print Format Code

My name is John and I am 30 years old.

Fig 2.5.2 String Print Format Output

Method 2: Using str.format() Method (Recommended in Python 2.x and 3.x)

In Python, formatting strings involves incorporating variables or values into a string template to produce formatted strings. There are several methods available to achieve this, including the use of the % operator, the str.format() function, and f-strings (formatted string literals introduced in Python 3.6). Each of these methods offers a way to create dynamic and customizable string outputs. The % operator, which is one of the older methods, uses a format specifier and a tuple to insert values into a string template. The str.format() function, introduced in Python 2.6 and enhanced in Python 3, allows more control and flexibility by using curly braces as placeholders within the string, which are then replaced by the values passed to the format() method. For more information and examples on this method, you can refer to this :https://github.com/quantanics2018/Python/blob/main/String_method_2.py

Finally, f-strings provide a more concise and readable way to format strings by prefixing the string with an 'f' and embedding expressions inside curly braces directly within the string. This method is not only more intuitive but also faster, making it the preferred choice for many Python developers. For more information and examples, you can refer to this https://github.com/quantanics2018/Python/blob/main/String_method_2.py.

```
name = "John"
age = 30

# Using str.format() for string formatting
formatted_string = "My name is {} and I am {} years old.".format(name, age)
print(formatted_string)
```

Fig 2.5.3 String Print Format Code

```
My name is John and I am 30 years old.
```

Fig 2.5.6 String Print Format Output

Method 3: Using the f-string method in Python provides a highly efficient and readable way to format strings. Introduced in Python 3.6, f-strings, or formatted string literals, allow developers to embed expressions inside string literals using curly braces {}. By prefixing the string with an f or F, the expressions within the braces are evaluated at runtime and formatted accordingly. This method is not only more concise compared to older formatting techniques but also improves code readability and performance. For instance, instead of using the cumbersome "%s" % (value) or the verbose "{}".format(value), you can simply write f"{value}". F-strings also support more complex expressions, such as calculations and

method calls, directly within the string, making them a versatile tool for dynamic string creation. For more details and examples, you can refer to this https://github.com/quantanics2018/Python/blob/main/String_method_3.py.

```
name = "John"
age = 30

# Using f-strings for string formatting
formatted_string = f"My name is {name} and I am {age} years old."
print(formatted_string)
```

Fig 2.5.5 String Print Format Code

```
My name is John and I am 30 years old.
```

Fig 2.5.6 String Print Format output

Topic 3: Python Data Structures and Control Flow

3.1) Lists in Python

A list is a collection of items that are ordered and mutable, which means you can change, add, or remove items after the list is created. Lists are defined using square brackets [] and can contain items of different data types. This flexibility makes lists a fundamental and versatile data structure in Python. Whether you need to store a sequence of numbers, strings, or even a mix of various data types, lists can accommodate your needs. Additionally, the ability to modify lists by appending, inserting, or deleting elements makes them particularly useful for dynamic data handling. Lists support a variety of built-in methods, such as `append()`, `extend()`, `insert()`, `remove()`, and `pop()`, which provide powerful and convenient ways to manipulate the contained items. Their ordered nature allows for easy indexing and slicing, enabling straightforward access and modification of specific elements or subsets of the list.

This code demonstrates various operations that can be performed on lists in Python. It begins by initializing a list named `fruits` with four items: "apple," "banana," "orange," and "grape." The code then showcases accessing items using indexing, such as retrieving the first item with `fruits[0]` ("apple") and the third item with `fruits[2]` ("orange"). It also demonstrates

modifying items through indexing by changing the second item ("banana") to "kiwi." Adding items is illustrated using the append method to add "pear" to the end of the list and the insert method to place "pineapple" at the second position. The remove method removes "kiwi" from the list, while the pop method with index 1 removes and returns "pineapple," storing it in the variable popped_fruit. The index method finds the index of "orange," and the in keyword checks for the presence of "banana," returning False. The len function calculates the number of items in the list, and a for loop iterates through each item, printing them individually. In summary, the code illustrates common list operations, such as accessing, modifying, adding, and removing items, as well as iterating through a list using a loop, showcasing the versatility of lists in handling collections of items. For more details and the full code, you can refer to this <https://github.com/quantanics2018/Python/blob/main/List.py>.

```
# Creating a list
fruits = ["apple", "banana", "orange", "grape"]

# Accessing items in a list using indexing
print(fruits[0]) # Output: apple
print(fruits[2]) # Output: orange

# Modifying items in a list using indexing
fruits[1] = "kiwi"
print(fruits) # Output: ['apple', 'kiwi', 'orange', 'grape']

# Adding an item to the end of the list using the `append` method
fruits.append("pear")
print(fruits) # Output: ['apple', 'kiwi', 'orange', 'grape', 'pear']

# Inserting an item at a specific index using the `insert` method
fruits.insert(2, "pineapple")
print(fruits) # Output: ['apple', 'kiwi', 'pineapple', 'orange', 'grape', 'pear']

# Removing an item from the list using the `remove` method
fruits.remove("kiwi")
print(fruits) # Output: ['apple', 'pineapple', 'orange', 'grape', 'pear']

# Removing an item by index using the `pop` method
popped_fruit = fruits.pop(1)
print("Popped fruit:", popped_fruit) # Output: Popped fruit: pineapple
print(fruits) # Output: ['apple', 'orange', 'grape', 'pear']

# Finding the index of an item using the `index` method
index_orange = fruits.index("orange")
print("Index of 'orange':", index_orange) # Output: Index of 'orange': 1

# Checking if an item is in the list using the `in` keyword
print("banana" in fruits) # Output: False

# Getting the length of the list using the `len` function
print("Number of fruits:", len(fruits)) # Output: Number of fruits: 4

# Iterating through a list using a for loop
for fruit in fruits:
    print(fruit)
```

Fig 3.1.1 Python List Code

```
apple
orange
['apple', 'kiwi', 'orange', 'grape']
['apple', 'kiwi', 'orange', 'grape', 'pear']
['apple', 'kiwi', 'pineapple', 'orange', 'grape', 'pear']
['apple', 'pineapple', 'orange', 'grape', 'pear']
Popped fruit: pineapple
['apple', 'orange', 'grape', 'pear']
Index of 'orange': 1
False
Number of fruits: 4
apple
orange
grape
pear
```

Fig 3.1.2 Python List Output

3.2) Dictionaries in python

Dictionaries are mutable data structures that allow you to store key-value pairs. Dictionary can be created using the `dict()` constructor or curly braces `{}`. Once you have created a dictionary, you can add, remove, or update elements using the methods `dict.update()`, `dict.In`. In Python, dictionaries are essential data structures that facilitate flexible and efficient data storage and retrieval. Unlike lists, which rely on numerical indices, dictionaries use key-value pairs to organize information. Each key within a dictionary is unique and corresponds to a specific value, enabling data association through meaningful labels or names. The provided code exemplifies dictionary usage by initializing a dictionary named `student` with attributes like `name`, `age`, `major`, and `enrollment status`, each associated with specific values. Demonstrating access operations, the code prints out these attributes sequentially. It further illustrates how to modify dictionary values; for instance, updating the student's age from 18 to 21 and adding new attributes like `gender` ("Male"). Removal of key-value pairs, such as deleting `enrollment status`, is also showcased using the `del` keyword. Additionally, the code covers checking for key existence, retrieving lists of keys and values, and iterating through the dictionary using loops. Overall, it offers a comprehensive exploration of how dictionaries manage and organize data based on descriptive keys and their corresponding values in Python

<https://github.com/quantanics2018/Python/blob/main/Dictionary.py>

Dictionary Properties

A Python dictionary consists of a key and then an associated value. Duplicate keys are not allowed but values can be duplicated. Heterogeneous objects are allowed for both keys and values. Dictionaries are mutable (can change or update values). Indexing and slicing concepts are not applicable.

```
# Creating a dictionary
student = {
    "name": "Adhiban",
    "age": 18,
    "major": "Computer Science",
    "is_enrolled": True
}

# Accessing values using keys
print("Name:", student["name"])           # Output: Name: Adhiban
print("Age:", student["age"])             # Output: Age: 18
print("Major:", student["major"])         # Output: Major: Computer Science
print("Is Enrolled:", student["is_enrolled"]) # Output: Is Enrolled: True

# Modifying values using keys
student["age"] = 21
print("Updated Age:", student["age"])     # Output: Updated Age: 21

# Adding a new key-value pair
student["gender"] = "Male"
print("Gender:", student["gender"])       # Output: Gender: Male

# Removing a key-value pair using the `del` keyword
del student["is_enrolled"]
# The following line would raise a KeyError because the key "is_enrolled" no longer exists
# print("Is Enrolled:", student["is_enrolled"])

# Checking if a key exists in the dictionary
if "name" in student:
    print("Name exists in the dictionary")

# Getting a list of all keys using the `keys` method
keys_list = student.keys()
print("Keys:", keys_list)                 # Output: Keys: dict_keys(['name', 'age', 'major', 'gender'])

# Getting a list of all values using the `values` method
values_list = student.values()
print("Values:", values_list)             # Output: Values: dict_values(['John Doe', 21, 'Computer Science', 'Male'])

# Iterating through keys and values using a loop
for key, value in student.items():
    print(key, ":", value)
```

Fig 3.2.1 Dictionaries in Python Code

```
Name: Adhiban
Age: 18
Major: Computer Science
Is Enrolled: True
Updated Age: 21
Gender: Male
Name exists in the dictionary
Keys: dict_keys(['name', 'age', 'major', 'gender'])
Values: dict_values(['Adhiban', 21, 'Computer Science', 'Male'])
name : Adhiban
age : 21
major : Computer Science
gender : Male
```

Fig 3.2.2 Dictionaries In Python Output

3.3) Tuples in python

Tuples in Python are a type of data structure that allows for the storage of a collection of items. They are similar to lists but have a few key differences, the most notable being their immutability. This means that once a tuple is created, its contents cannot be changed. Tuples are defined using parentheses () and can contain items of different data types. This immutability makes tuples particularly useful for storing data that should not be altered, providing a layer of data protection within a program. Despite their fixed nature, tuples can still be used effectively in various programming scenarios, especially when paired with operations that do not require modification of the stored data.

```

# Creating a tuple
student_info = ("Adhiban", 18, "Computer Science")

# Accessing elements using indexing
print("Name:", student_info[0])      # Output: Name: Adhiban
print("Age:", student_info[1])       # Output: Age: 18
print("Major:", student_info[2])     # Output: Major: Computer Science

# Iterating through a tuple
for item in student_info:
    print(item)
# Output:
# Adhiban
# 18
# Computer Science

# Nested tuples
course1 = ("Math", 4)
course2 = ("History", 3)
courses = (course1, course2)

# Accessing elements in nested tuples
print("Course 1:", courses[0])       # Output: Course 1: ('Math', 4)
print("Course 2:", courses[1])       # Output: Course 2: ('History', 3)
print("Course 1 Name:", courses[0][0]) # Output: Course 1 Name: Math
print("Course 2 Credits:", courses[1][1]) # Output: Course 2 Credits: 3

```

Fig 3.3.1 Tuples In Python Code

This code provides a comprehensive overview of tuples in Python, demonstrating their creation, indexing, iteration, and nesting. It begins by creating a tuple named `student_info`, which encapsulates a student's name, age, and major. The elements within the tuple are enclosed in parentheses and separated by commas. Subsequent lines show how to access specific elements using indexing: `student_info[0]` retrieves the student's name "Alice," and `student_info[2]` retrieves their major "Biology." These values are printed with descriptive labels. The code then illustrates iterating through the `student_info` tuple using a `for` loop, where each element is printed on separate lines, showcasing the ordered nature of tuples. Next, the concept of nested tuples is introduced with two tuples, `course1` and `course2`, representing course names and their credit hours. These are combined into a larger tuple named `courses`. The code demonstrates accessing elements within these nested tuples using indexing, such as `courses[0]` for the tuple ("Math", 4) and `courses[1]` for the tuple ("History", 3). Additional indexing allows access to nested tuple elements, like `courses[0][0]` for the course name and `courses[1][1]` for credit hours. Overall, this code exemplifies the fundamental characteristics of tuples in Python, highlighting their immutability, ordered structure, and ability to hold diverse data types. It also provides insights into indexing for both single-level and nested tuples, as well as demonstrating the iterative capabilities of

tuples. For more details and the full code, you can refer to this <https://github.com/quantanics2018/Python/blob/main/Tuple.py>

```
Name: Adhiban
Age: 18
Major: Computer Science
Adhiban
18
Computer Science
Course 1: ('Math', 4)
Course 2: ('History', 3)
Course 1 Name: Math
Course 2 Credits: 3
'
```

Fig 3.3.2 Tuples In Python Code

3.4) Sets in python:

In Python, a set is a built-in data structure that represents an unordered collection of unique elements. Defined using curly braces {}, sets can contain various data types. They automatically ensure that each element appears only once and do not maintain any specific order. This makes sets particularly useful for tasks such as removing duplicate values from a collection or testing membership.

```

# Creating sets
fruits = {"apple", "banana", "orange"}
numbers = set([1, 2, 3, 3, 4, 4, 5]) # Creating a set from a list

# Adding and removing elements
fruits.add("grape")
fruits.remove("banana")
fruits.discard("watermelon")

# Set operations
set1 = {1, 2, 3}
set2 = {3, 4, 5}

union_set = set1 | set2 # Union using the | operator or .union() method
intersection_set = set1 & set2 # Intersection using the & operator or .intersection() method
difference_set = set1 - set2 # Difference using the - operator or .difference() method

# Checking membership
if "apple" in fruits:
    print("Apple is in the set")

# Iterating through sets
print("Fruits:")
for fruit in fruits:
    print(fruit)

# Set comprehensions
squared_numbers = {x**2 for x in range(1, 6)}

# Frozen sets
frozen = frozenset([1, 2, 3])

# Displaying results
print("Numbers:", numbers) # Output: Numbers: {1, 2, 3, 4, 5}
print("Union Set:", union_set) # Output: Union Set: {1, 2, 3, 4, 5}
print("Intersection Set:", intersection_set) # Output: Intersection Set: {3}
print("Difference Set:", difference_set) # Output: Difference Set: {1, 2}
print("Squared Numbers:", squared_numbers) # Output: Squared Numbers: {1, 4, 9, 16, 25}
print("Frozen Set:", frozen) # Output: Frozen Set: frozenset({1, 2, 3})

```

Fig 3.4.1 Sets In Python Code

This code offers a comprehensive illustration of how sets are used in Python, covering various aspects of their creation, manipulation, and operations. It begins by creating two sets, `fruits` and `numbers`. The `fruits` set is initialized with a few fruit names, while the `numbers` set is created from a list of integers, effectively removing any duplicate values due to the unique property of sets. The code then showcases adding and removing elements from the `fruits` set, where the `.add()` method adds the fruit "grape," and the `.remove()` method removes "banana." The `.discard()` method is also used to safely remove "watermelon" even if it isn't present in the set. Next, set operations are demonstrated using `set1` and `set2`, highlighting the union, intersection, and difference of these sets. The `union_set` contains all distinct elements from both sets, the `intersection_set` contains the common elements, and the `difference_set` holds elements present in `set1` but not in `set2`. Membership testing is exemplified with the `if "apple" in fruits:` statement, checking if "apple" exists in the `fruits` set. The code then moves on to iterating through the `fruits` set using a `for` loop, emphasizing that sets do not guarantee a

specific order of elements. Set comprehensions are introduced to create sets based on expressions and looping, demonstrated by generating `squared_numbers` using a comprehension. Lastly, the concept of frozen sets is introduced, showcasing their immutability and suitability as dictionary keys due to their unchanging nature. In summary, the code provides an all-encompassing view of sets in Python, from their creation to their various operations and uses, emphasizing their uniqueness and unordered nature, which are valuable for tasks such as eliminating duplicates, performing set operations, and testing membership. More detail and object file can be found here <https://github.com/quantanics2018/Python/blob/main/Sets.py>

```
Apple is in the set
Fruits:
grape
apple
orange
Numbers: {1, 2, 3, 4, 5}
Union Set: {1, 2, 3, 4, 5}
Intersection Set: {3}
Difference Set: {1, 2}
Squared Numbers: {1, 4, 9, 16, 25}
Frozen Set: frozenset({1, 2, 3})
```

Fig 3.4.2 output

3.5) Booleans in python

In Python, the Boolean data type represents one of two values: True or False. Booleans are fundamental for logical operations, comparisons, and control flow within programs. They are used to make decisions and perform conditional actions based on the truth value of expressions.

Boolean Values

There are two Boolean values in Python: True and False. They are case-sensitive, so writing them as true or false will result in errors.

Boolean Operators

Python supports three main Boolean operators: `'and'`, `'or'`, and `'not'`, which are used to combine or negate Boolean values. The `'and'` operator returns `'True'` if both operands are

`True`; otherwise, it returns `False`. The `or` operator returns `True` if at least one operand is `True`; otherwise, it returns `False`. The `not` operator negates the Boolean value, making `True` become `False` and vice versa. Additionally, Python provides several comparison operators for comparing values and returning Boolean results. These include `==` (equal to), `!=` (not equal to), `<` (less than), `>` (greater than), `<=` (less than or equal to), and `>=` (greater than or equal to).

Truthy and Falsy Values

In Python, certain values are considered truthy or falsy when evaluation in a Boolean context. For example, empty collections (empty strings, lists, dictionaries, etc.) are falsy, while non-empty collections are truthy. Similarly, 0 is falsy, while non-zero numbers are truthy.

Conditional Statements

Booleans play a crucial role in conditional statements like `if`, `elif`, and `else`. These statements determine the flow of the program based on the truth value of expressions.

```

# Boolean Values
is_student = True
has_passed_exam = False

# Boolean Operators
and_result = is_student and has_passed_exam # False
or_result = is_student or has_passed_exam # True
not_result = not is_student # False

# Comparison Operators
age = 20
is_adult = age >= 18 # True

# Truthy and Falsy Values
empty_list = []
non_empty_list = [1, 2, 3]
empty_string = ""
non_empty_string = "Hello"
zero = 0
non_zero = 42

# Conditional Statements
if is_adult:
    print("You are an adult.")
else:
    print("You are not an adult.")

if non_empty_list:
    print("The list is not empty.")
else:
    print("The list is empty.")

if not empty_string:
    print("The string is empty.")
else:
    print("The string is not empty.")

if non_zero:
    print("The number is non-zero.")
else:
    print("The number is zero.")

```

Fig 3.5.1 Booleans In Python Code

The provided code example serves as a comprehensive demonstration of Boolean usage in Python, covering a range of concepts related to Boolean values, operators, comparisons, truthy and falsy values, and their role in conditional statements. The code begins by assigning Boolean values to the variables `'is_student'` and `'has_passed_exam'`. These values, `'True'` and `'False'` respectively, are the fundamental building blocks of Boolean logic in Python. The three main Boolean operators – `'and'`, `'or'`, and `'not'` – are showcased. Using these operators, the code evaluates combinations of `'is_student'` and `'has_passed_exam'`, yielding results such as `'False'` for the `'and'` operation and `'True'` for the `'or'` operation. Additionally, the `'not'` operator is employed to invert the value of `'is_student'`.

Comparison operators, another fundamental concept, are demonstrated through the variable `'age'`. Using the `'>='` operator, `'is_adult'` is assigned the Boolean value `'True'` if the age is greater than or equal to 18, highlighting how these operators are used to make comparisons and derive Boolean outcomes. The concept of truthy and falsy values is illustrated through a series of examples. Empty collections, such as empty lists and strings, are considered falsy, while non-empty collections are considered truthy. Similarly, numeric values like 0 are falsy, whereas non-zero numbers are truthy. These concepts are crucial for conditional statements.

Conditional statements come into play as the program uses `'if'` and `'else'` to control the program's flow based on the truth value of expressions. The first conditional statement checks whether a person is an adult based on the `'is_adult'` Boolean value. Depending on the outcome, an appropriate message is printed. Subsequent conditional statements evaluate the truthiness of different variables and respond accordingly. In essence, this code example offers a comprehensive introduction to Booleans in Python. It effectively highlights the roles of Boolean values, operators, comparisons, truthy and falsy concepts, and their integration into conditional statements. Understanding these aspects is essential for creating logical and decision-making structures within Python programs. More detail and object file can be found here <https://github.com/quantanics2018/Python/blob/main/Booleans.py>

```
You are an adult.  
The list is not empty.  
The string is empty.  
The number is non-zero.
```

Fig 3.5.2 Booleans In Python Output

3.6) I/O with basic file in python

This code is a complete example that covers basic file input/output operations in Python . More detail and object file can be found here https://github.com/quantanics2018/Python/blob/main/IO_with_basic%20files.py

```
# Using double backslashes  
with open("F:\\HI.txt", "r") as file:  
    content = file.read()  
  
with open("F:\\HI.txt", "w") as file:  
    file.write("Hello, world!\\n")  
    file.write("This is a new line.")  
  
with open("F:\\HI.txt", "a") as file:  
    file.write("\\nThis is an appended line.")  
  
try:  
    with open("F:\\HI.txt", "r") as file:  
        content = file.read()  
except FileNotFoundError:  
    print("File not found.")
```

Fig 3.6.1 I/O With Basic File In Python Code

```
hii  
file not found
```

Fig 3.6.2 I/O With Basic File In Python Output

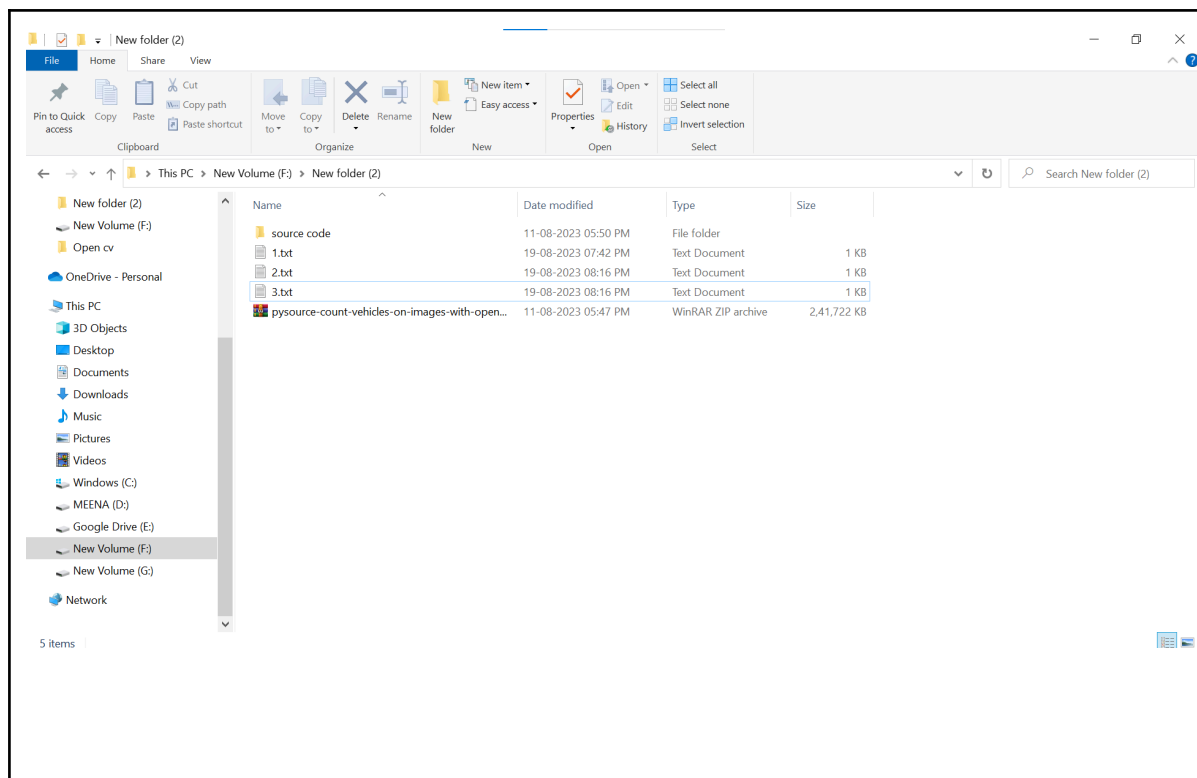


Fig 3.6.3 I/O With Basic File In Python Output

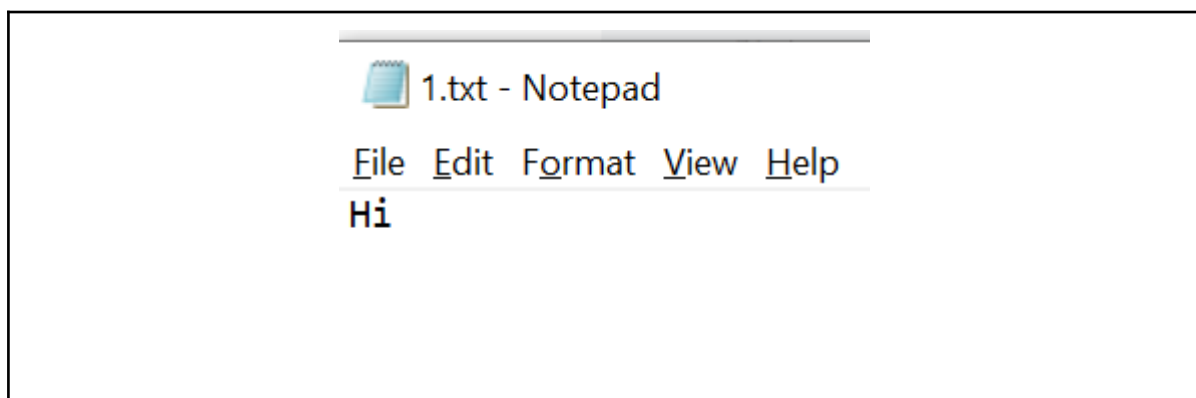


Fig 3.6.4 I/O With Basic File In Python Output

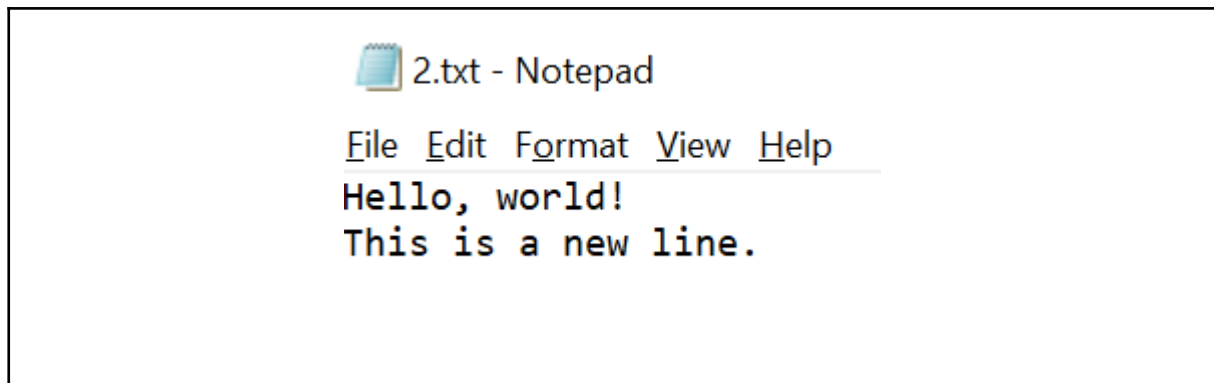


Fig 3.6.5 I/O With Basic File In Python Output

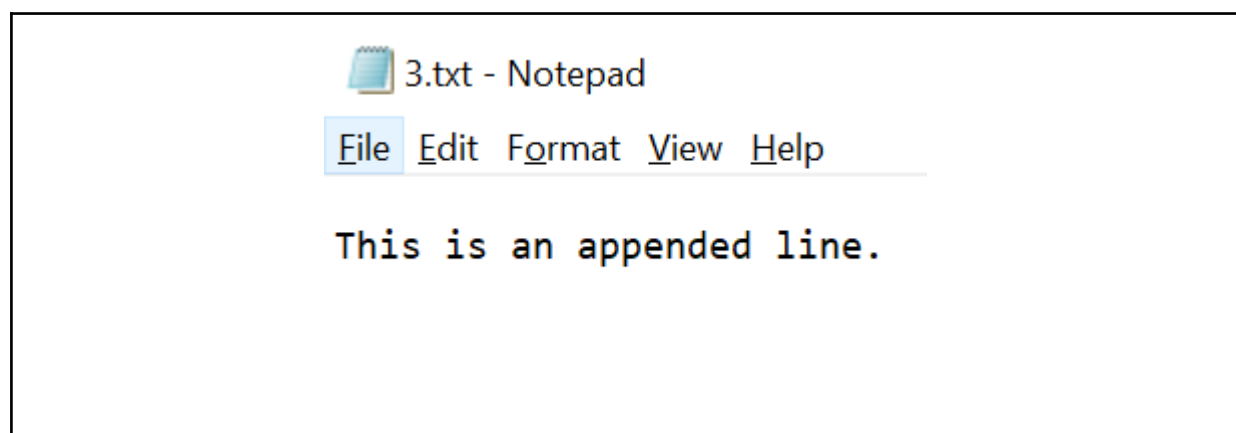


Fig 3.6.6 I/O With Basic File In Python Output

3.7) Python Object and Data Structure

In Python, objects and data structures are fundamental concepts that allow you to store, manipulate, and organize data in various forms. Understanding these concepts is crucial for effective programming. Here's an overview:

Python Objects

In Python, everything is an object, and objects have both data (attributes) and associated functions (methods). Python includes several built-in data types such as integers ('int'), floating-point numbers ('float'), strings ('str'), booleans ('bool'), lists ('list'), tuples ('tuple'), sets ('set'), dictionaries ('dict'), and more. Additionally, you can create custom objects by defining classes, which serve as blueprints that define the structure and behavior of these objects.

Python Objects

Everything is an Object: In Python, everything is an object. Objects have both data (attributes) and associated functions (methods).

Data Types: Python has several built-in data types, including integers (int), floating-point numbers (float), strings (str), booleans (bool), lists (list), tuples (tuple), sets (set), dictionaries (dict), and more.

Custom Objects: You can create your own custom objects by defining classes. Classes are blueprints that define the structure and behavior of objects.

Data Structures

Lists: Ordered, mutable collections of items. Lists are created using square brackets [].

Tuples: Ordered, immutable collections of items. Tuples are created using parentheses () .

Sets: Unordered collections of unique items. Sets are created using curly braces {} .

Dictionaries: Key-value pairs for efficient data retrieval. Dictionaries are created using curly braces {} with key-value pairs separated by colons.

Mutable vs. Immutable

Mutable: Objects that can be modified after creation. Lists, dictionaries, and sets are mutable.

Immutable: Objects that cannot be modified once created. Integers, floats, strings, and tuples are immutable.

Object-Oriented Programming (OOP):

Python supports Object-Oriented Programming (OOP). You can define classes with attributes and methods, creating reusable and organized code.

Built-in Functions

Python provides many built-in functions to work with objects, such as len(), type(), str(), int(), float(), and more.

Memory Management

Python handles memory management automatically. Objects are created, used, and discarded by the Python memory manager. The `del` statement can be used to delete references to objects and free up memory.

Comparison and Logical Operators

Comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) are utilized to compare values, yielding Boolean results. Logical operators (`and`, `or`, `not`) are employed to combine or negate Boolean values. A solid understanding of objects and data structures is crucial for writing efficient and organized code, as it enables the selection and effective use of appropriate data structures to address a variety of problems.

3.8) Comparison Operator in Python

In Python, comparison operators are used to compare two values and determine their relationship, returning Boolean values (True or False) based on the outcome of the comparison. The "equal to" (`==`) operator returns True if the values on both sides are equal, otherwise it returns False. The "not equal to" (`!=`) operator returns True if the values on both sides are not equal, otherwise it returns False. The "less than" (`<`) operator returns True if the value on the left is less than the value on the right, otherwise it returns False. The "greater than" (`>`) operator returns True if the value on the left is greater than the value on the right, otherwise it returns False. The "less than or equal to" (`<=`) operator returns True if the value on the left is less than or equal to the value on the right, otherwise it returns False. Finally, the "greater than or equal to" (`>=`) operator returns True if the value on the left is greater than or equal to the value on the right, otherwise it returns False..More detail and object file can be found here

https://github.com/quantanics2018/Python/blob/main/Comparison_operator_in_python.py

```
x = 10
y = 5

# Equal to
print(x == y)  # Output: False

# Not equal to
print(x != y)  # Output: True

# Less than
print(x < y)    # Output: False

# Greater than
print(x > y)    # Output: True

# Less than or equal to
print(x <= y)   # Output: False

# Greater than or equal to
print(x >= y)   # Output: True
```

Fig 3.8.1 Comparison Operator In Python Code

```
False
True
False
True
False
True
```

Fig 3.8.2 Comparison Operator In Python Output

3.9) Chaining Comparison Operator in Python with Logical Operator

Chaining comparison operators with logical operators in Python allows you to create more complex conditions by combining multiple comparisons and evaluating their results. Logical operators like and or are used to combine these conditions to create more intricate logic.

```
x = 5
y = 10
z = 15

# Using 'and' to combine multiple comparisons
if x < y and y < z:
    print("Both conditions are true.")
else:
    print("At least one condition is not met.")

# Using 'or' to combine multiple comparisons
if x > y or y < z:
    print("At least one condition is true.")
else:
    print("No condition is true.")
```

Fig 3.9.1 Comparison Operator Python Code

In this code, we have three variables: x, y, and z, each representing different values. Comparison operators (<, >) are used to compare these values. The first if statement combines two comparison conditions, $x < y$ and $y < z$, using the and logical operator. The and operator checks if both conditions are true. If both conditions are met, it prints "Both conditions are true." Otherwise, it prints "At least one condition is not met." The second if statement uses the or logical operator to combine two comparison conditions: $x > y$ and $y < z$. The or operator checks if at least one of the conditions is true. If at least one condition is true, it prints "At least one condition is true." Otherwise, it prints "No condition is true." By chaining comparison operators with logical operators, more complex conditions can be created, enabling sophisticated decision-making logic in Python programs. More details and object file can be found here https://github.com/quantanics2018/Python/blob/main/chaining_comparison_operator_in_python_with_logical_operator.py

```
Both conditions are true.
At least one condition is true.
```

Fig 3.9.2 Comparison Operator Python Output

3.10) If Elif and Else Statement in Python

The if, elif, and else statements are used in Python to control the flow of your program based on certain conditions. They allow you to execute different blocks of code depending on whether certain conditions are true or false.

```
# Example of if, elif, and else statements

x = 10

if x > 0:
    print("x is positive")
elif x == 0:
    print("x is zero")
else:
    print("x is negative")
```

Fig 3.10.1 If Elif and Else Statement in Python Code

In this example, we have a variable `x` with a value of 10. The program evaluates `x` against multiple conditions using if, elif (short for "else if"), and else statements. The if statement checks if `x` is greater than 0, and if true, it executes the indented block of code, printing "x is positive." If the if condition is false, the elif statement checks if `x` is equal to 0, and if true, it executes the corresponding block of code, printing "x is zero." If both the if and elif conditions are false, the else statement's block of code is executed, printing "x is negative." In this case, the output will be "x is positive" because `x` is indeed greater than 0. You can use as many elif statements as needed to handle different cases. If none of the conditions are true, the code in the else block will be executed. It is important to remember that if, elif, and else statements are designed to be mutually exclusive; once one condition is satisfied, the program will not evaluate subsequent conditions. This is crucial to keep in mind when organizing your code's logic. More details and object file can be found here https://github.com/quantanics2018/Python/blob/main/If_elif_and_else_statement_in_python.py

```
x is positive
```

Fig 3.10.2 If Elif and Else Statement in Python Output

3.11) For Loop in Python

The for loop in Python is used to iterate over a sequence (such as a list, tuple, string, or range) and perform some action on each element of the sequence. It's a powerful construct for performing repetitive tasks.

```
fruits = ["apple", "banana", "cherry"]  
  
for fruit in fruits:  
    print(fruit)
```

Fig 3.11.1 For Loop in Python Code

In this example, the for loop iterates over each element in the fruits list. On each iteration, the fruit variable takes the value of the current element, and the fruit variable takes the value of the current element, and the indented block of code (in this case, print(fruit)) is executed. The loop will run three times, printing each fruit's name. More details and object file can be found here [https://github.com/quantanics2018/Python/blob/main/for_loop\(1\).py](https://github.com/quantanics2018/Python/blob/main/for_loop(1).py)

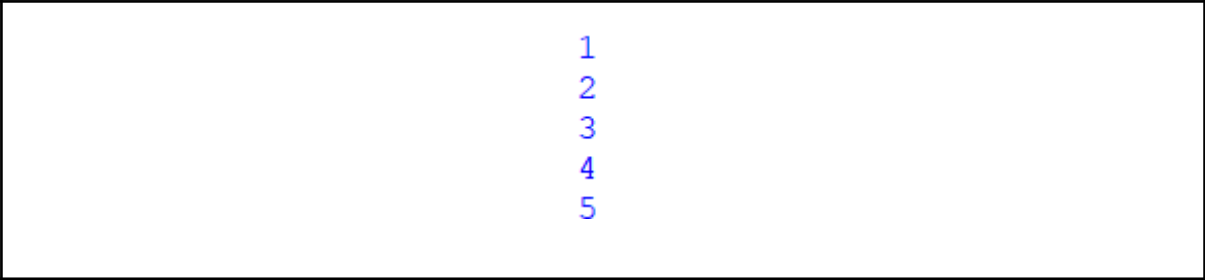
```
apple  
banana  
cherry
```

Fig 3.11.2 For Loop in Python Output

We can also use the range function in the loop.

```
for num in range(1, 6): # The range goes from 1 to 5 (6 is exclusive)  
    print(num)
```

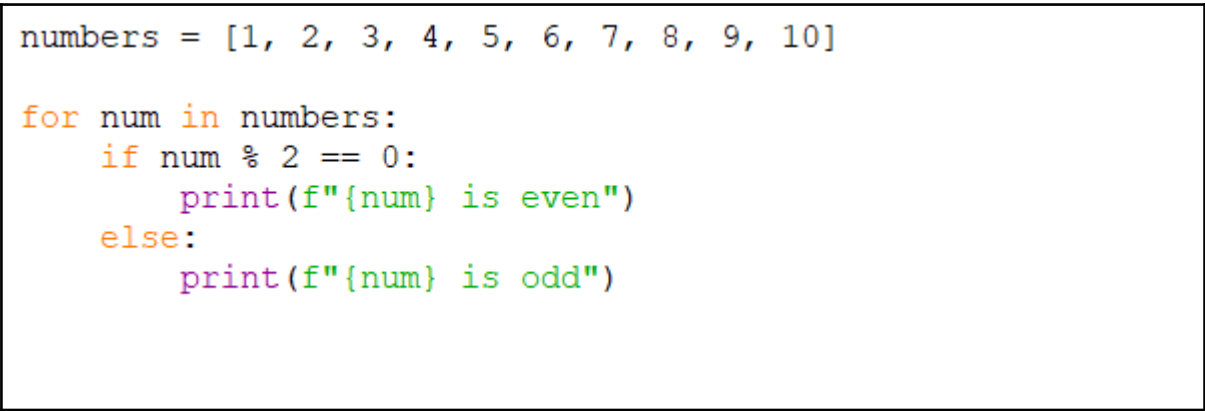
Fig 3.11.3 For Loop in Python Range Code



```
1
2
3
4
5
```

Fig 3.11.4 For Loop in Python Output

You can combine the for loop with other control flow statements like if, elif, and else to create more complex behaviors within the loop's body. More details and object file can be found here [https://github.com/quantanics2018/Python/blob/main/for_loop\(2\).py](https://github.com/quantanics2018/Python/blob/main/for_loop(2).py)



```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

for num in numbers:
    if num % 2 == 0:
        print(f"{num} is even")
    else:
        print(f"{num} is odd")
```

Fig 3.11.5 For Loop in Python Code

In this example, the loop iterates over each number in the numbers list. The if statement inside the loop checks if the number is even or odd and prints an appropriate message. The for loop is a fundamental construct in programming and is widely used for tasks that involve repetitive operations. More details and objects can be found here [https://github.com/quantanics2018/Python/blob/main/for_loop\(3\).py](https://github.com/quantanics2018/Python/blob/main/for_loop(3).py)

```
1 is odd
2 is even
3 is odd
4 is even
5 is odd
6 is even
7 is odd
8 is even
9 is odd
10 is even
```

Fig 3.11.6 For Loop in Python Output

3.12) While loop in python

The while loop in Python is used to repeatedly execute a block of code as long as a certain condition remains true. It's often used when the exact number of iterations is not known before hand.

```
count = 0

while count < 5:
    print("Count:", count)
    count += 1
```

Fig 3.12.1 While loop in python Code

In this example, the while loop continues to execute as long as the condition `count < 5` remains true. The code inside the loop prints the current value of `count` and then increments it by 1. The loop will run five times, printing the values 0 through 4. More details and object can be found here [https://github.com/quantanics2018/Python/blob/main/While\(1\).py](https://github.com/quantanics2018/Python/blob/main/While(1).py)

```
Count: 0  
Count: 1  
Count: 2  
Count: 3  
Count: 4
```

Fig 3.12.2 While loop in python Output

You need to be cautious when using while loops, as they can potentially lead to infinite loops if the condition never becomes false. More details and objects can be found here [https://github.com/quantanics2018/Python/blob/main/While\(2\).py](https://github.com/quantanics2018/Python/blob/main/While(2).py)

```
while True:  
    print("This is an infinite loop!")
```

Fig 3.12.3 While loop in python Code

```
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!  
This is an infinite loop!
```

Fig 3.12.4 While loop in python Output

3.13) Useful Operator in Python

Python offers a wide range of operators that are useful for various purposes. Here are some of the most commonly used and useful operators in Python:

Arithmetic Operators: Used for basic arithmetic operations.

1. + (addition)
2. - (subtraction)
3. * (multiplication)
4. / (division)
5. % (modulo - remainder of division)
6. ** (exponentiation)
7. // (floor division)

Comparison Operators: Used to compare values.

1. == (equal to)
2. != (not equal to)
3. < (less than)
4. > (greater than)
5. <= (less than or equal to)
6. >= (greater than or equal to)

Logical Operators: Used to perform logical operations.

1. and (logical AND)
2. or (logical OR)
3. not (logical NOT)

Assignment Operators: Used to assign values to variables.

1. = (simple assignment)
2. +=, -=, *=, /=, %=, **=, //= (compound assignment)

Membership Operators: Used to test membership in a sequence.

1. in (element exists in sequence)
2. not in (element does not exist in sequence)

Identity Operators: Used to compare object identities.

1. is (objects are the same)
2. is not (objects are not the same)

Bitwise Operators: Used to perform bitwise operations on integers.

1. & (bitwise AND)
2. | (bitwise OR)
3. ^ (bitwise XOR)
4. ~ (bitwise NOT)
5. << (left shift)
6. >> (right shift)

Ternary Operator: A shorthand way to write conditional expressions.

1. x if condition else y
2. These operators form the foundation of Python's capabilities and are used extensively in programming.
3. They allow you to perform mathematical calculations, make decisions, manipulate data, and control the flow of your program. Understanding and using these operators effectively is essential for writing efficient and clear Python code.

3.14) List Comprehensions in Python

In Python, list comprehensions provide a concise way to create lists. They are a syntactic construct that allows you to generate a new list by applying an expression to each item in an existing iterable (such as a list, tuple, or range) and optionally applying a filter condition.

```
# Original list of numbers
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# List comprehension to filter even numbers and calculate their squares
even_square_list = [x ** 2 for x in numbers if x % 2 == 0]

# Output
print(even_square_list)

# Original list of numbers
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

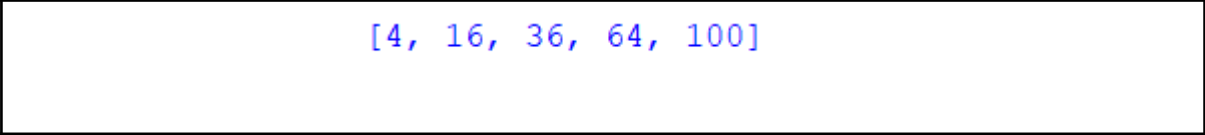
# List comprehension to filter even numbers and calculate their squares
even_square_list = [x ** 2 for x in numbers if x % 2 == 0]

# Output
print(even_square_list)
```

Fig 3.14.1 List Comprehensions in Python Code

In this example, we use a list comprehension to create a new list of squared even numbers from an original list ranging from 1 to 10. The list comprehension iterates through each number in the 'numbers' list, checking if each number is even with the condition `x % 2 == 0`. If the number is even, the expression `x ** 2` calculates its square, which is then added to the

new list 'even_square_list'. After iterating through all the numbers, the resulting list contains the squares of the even numbers from the original list. List comprehensions offer a compact and readable way to create lists by combining iteration and conditional filtering in a single line of code. More details and object can be found here https://github.com/quantanics2018/Python/blob/main/list_comprehensions_in_python.py

A screenshot of a Python list output, enclosed in a black rectangular border. The text inside the border is "[4, 16, 36, 64, 100]" in a blue, monospaced font, representing the result of a list comprehension operation.

```
[4, 16, 36, 64, 100]
```

Fig 3.14.2 List Comprehensions in Python Output

Topic 4: Python Functions and Classes

4.1) Methods and Python Documentation

In Python, methods are functions associated with objects, enabling various actions or operations on those objects. Each data type or object class in Python has its own set of methods providing specific functionalities. For more information on these methods and their usage, refer to the [Python documentation](<https://docs.python.org/>). This comprehensive resource offers detailed information on the Python programming language, including built-in functions, standard libraries, modules, classes, and more. It is an essential reference for Python developers to understand and utilize the language's diverse features effectively.

The official Python documentation is organized into sections like tutorials, library references, and language references. Built-in functions and methods documentation can also be accessed directly within your Python interpreter or code using the 'help()' or 'dir()' functions. Besides the official documentation, numerous online resources, tutorials, and community-driven websites, such as Stack Overflow and various Python blogs, provide explanations, examples, and usage tips for Python methods and libraries. Integrated development environments (IDEs) like PyCharm, Visual Studio Code, and Jupyter Notebook enhance this exploration with in-editor documentation and autocomplete features. When working with specific data types or classes, it is crucial to refer to their documentation to understand available methods and their effective use. This documentation typically includes explanations of parameters, return values, usage examples, and related methods.

4.2) Introduction to Function

A function in programming is a reusable block of code that performs a specific task, helping to organize code into smaller, manageable pieces, which makes it easier to read, maintain,

and debug. Functions encapsulate logic, promote code reusability, and enhance the overall structure of the code. For instance, consider defining a function named `calculate_rectangle_area` using the `def` keyword.

This function takes two parameters, `length` and `width`, representing the dimensions of a rectangle. The accompanying docstring describes the function's purpose, the parameters, and the return value. Within the function, the area of the rectangle is calculated by multiplying the `length` and `width` parameters, with the result stored in the variable `area`.

The `return` statement then sends the calculated area back to the caller, which assigns it to the variable `area_of_rectangle` when the function is called. By providing specific values for `rectangle_length` and `rectangle_width`, representing the dimensions of a rectangle, we call the `calculate_rectangle_area` function, passing these values as arguments. The function computes the area using the provided dimensions, and the calculated area is printed using a formatted string.

This example illustrates defining a function, providing input through arguments, performing a calculation, and returning a result. Functions improve code organization and readability by encapsulating reusable logic, and the docstring provides helpful information, making the function's purpose and usage easier to understand.

Use the keyword `def` to declare the function and follow this up with the function name. Add parameters to the function: they should be within the parentheses of the function. End your line with a colon.

```

# Function definition
def calculate_rectangle_area(length, width):
    """
    This function calculates the area of a rectangle.

    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.

    Returns:
    float: The calculated area of the rectangle.
    """
    area = length * width
    return area

# Using the function
rectangle_length = 5.0
rectangle_width = 3.0
area_of_rectangle = calculate_rectangle_area(rectangle_length, rectangle_width)

# Output
print(f"The area of the rectangle is: {area_of_rectangle}")

```

Fig 4.2.1 Introduction to Function python code

```

The area of the rectangle is: 15.0
|

```

Fig 4.2.2 Introduction to Function python code output

More details and the project files can be found here
<https://github.com/quantanics2018/Python/blob/main/Function.py>

4.3) Basics of Python Function

Function Definition: Functions are defined using the `def` keyword, followed by the function name and a pair of parentheses. The function name should be descriptive of its purpose. You can define parameters inside the parentheses.

```

def greet(name):
    print(f"Hello, {name}!")

```

Function Call: To use a function, you call it by using its name followed by parentheses. If the function has parameters, you provide values for those parameters inside the parentheses

```
greet("Alice") # calling the greet function with the argument "Alice"
```

Parameters and Arguments: Parameters are placeholders in the function definition that accept input values. Arguments are the actual values passed to a function during a function call. The order of arguments matters.

Return Statement: Functions can return values using the return statement. The returned value can be assigned to a variable or used directly in expressions.

```
def add(a, b):  
    return a + b  
  
result = add(3, 5) # Calling the add function and assigning its return value to 'result'  
print(result)
```

8

Default Parameters: You can provide default values for parameters when defining a function. If an argument is not provided during the function call, the default value will be used.

Scope: Functions create their own local scope, meaning variables defined within a function are only accessible within that function. Variables defined outside of a function are in the global scope and can be accessed from any part of the code.

Docstrings: A docstring is a multiline string placed within triple quotes at the beginning of a function definition. It provides information about the function's purpose, parameters, and usage. Use the `help()` function to access docstrings interactively.

Functions are essential for structuring your code and promoting code reusability. They allow you to break down complex tasks into smaller, manageable pieces, making your code more modular and maintainable. As you progress, you'll encounter more advanced topics related to functions, such as lambda functions, closures, and decorato

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Parameters_and_arguments.p_

4.4) Logic with Python Functions

```
def is_even(number):  
    """  
    This function checks if a number is even.  
  
    Parameters:  
    number (int): The number to check.  
  
    Returns:  
    bool: True if the number is even, False if it's odd.  
    """  
    if number % 2 == 0:  
        return True  
    else:  
        return False  
  
# Using the function  
num = 10  
if is_even(num):  
    print(f"{num} is even.")  
else:  
    print(f"{num} is odd.")
```

Fig 4.4.1 Python Functions code

We define a function named `is\_even` that takes a number as a parameter. Inside the function, an if-else statement checks if the remainder of the number divided by 2 equals 0. If true, the function returns `True`, indicating the number is even; otherwise, it returns `False`, indicating the number is odd. We then call the `is\_even` function with the number 10 as an argument. Using an if statement, we determine whether the returned value is `True` (even) or `False` (odd). Based on this result, we print whether the number is even or odd. This example shows how to use a Python function to determine if a number is even or odd. By encapsulating the logic within a function, the code becomes more modular and readable, allowing for the reuse of the same logic in multiple places without duplication.


```
10 is even.
```

Fig 4.4.2 Logic with Python Functions Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Logic_with_python.py

4.5) Tuple Unpacking with Python Function

Tuple unpacking allows you to assign the elements of a tuple to multiple variables in a single line, which is particularly useful when working with functions that return multiple values packed in a tuple. For example, consider the function `'get_student_info'`, which simulates obtaining information about a student and returns a tuple containing the student's name, age, and grade. After calling this function, we can use tuple unpacking to assign the returned tuple to three separate variables: `'student_name'`, `'student_age'`, and `'student_grade'`. The elements of the tuple are unpacked and assigned to these variables in the order they appear. We then print the unpacked values using the assigned variables. Tuple unpacking with functions allows you to work with multiple return values easily, enhancing code readability and simplifying the process of extracting values from tuples.

```

# Creating a tuple
student_info = ("Adhiban", 18, "Computer Science")

# Accessing elements using indexing
print("Name:", student_info[0])      # Output: Name: Adhiban
print("Age:", student_info[1])       # Output: Age: 18
print("Major:", student_info[2])     # Output: Major: Computer Science

# Iterating through a tuple
for item in student_info:
    print(item)
# Output:
# Adhiban
# 18
# Computer Science

# Nested tuples
course1 = ("Math", 4)
course2 = ("History", 3)
courses = (course1, course2)

# Accessing elements in nested tuples
print("Course 1:", courses[0])        # Output: Course 1: ('Math', 4)
print("Course 2:", courses[1])        # Output: Course 2: ('History', 3)
print("Course 1 Name:", courses[0][0]) # Output: Course 1 Name: Math
print("Course 2 Credits:", courses[1][1]) # Output: Course 2 Credits: 3

```

Fig 4.5.1 Tuple Unpacking With Python Function Code

```

Name: Adhiban
Age: 18
Major: Computer Science
Adhiban
18
Computer Science
Course 1: ('Math', 4)
Course 2: ('History', 3)
Course 1 Name: Math
Course 2 Credits: 3

```

Fig 4.5.2 Tuple Unpacking with Python Function Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Tuple.py>

4.6) *args and **Kwags in Python

*args and **kwargs are special syntax in Python used to pass a variable number of arguments to a function. They are often used when you want to create functions that can accept a flexible number of positional or keyword arguments. Here's how *args and **kwargs work

***args:**

The *args syntax allows you to pass a variable number of positional arguments to a function. These arguments are collected into a tuple inside the function.

```
def print_args(*args):  
    for arg in args:  
        print(arg)  
  
print_args(1, 2, 3)    # Output: 1 2 3  
print_args('hello', 'world')  # Output: hello world
```

Fig 4.6.1 Args Python Code

```
1  
2  
3  
hello  
world
```

Fig 4.6.2 Args Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/args.py>

****kwargs:**

The ****kwargs** syntax allows you to pass a variable number of keyword arguments to a function. These arguments are collected into a dictionary inside the function.

```
def print_kwargs(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_kwargs(name='Adhiban', age=18) # Output: name: Adhiban age: 18
print_kwargs(country='India', city='Madurai') # Output: country: India city: Madurai
```

Fig 4.6.3 Kwargs Python Code

```
name: Adhiban
age: 18
country: India
city: Madurai
```

Fig 4.6.4 Kwargs Python Output

Combining *args and **kwargs:

You can also use ***args** and ****kwargs** together in a function definition. When calling such a function, positional arguments are collected by ***args**, and keyword arguments are collected by ****kwargs**.

```
def mixed_args_kwargs(arg1, arg2, *args, **kwargs):
    print("First two arguments:", arg1, arg2)
    print("Additional positional arguments:", args)
    print("Additional keyword arguments:", kwargs)

mixed_args_kwargs(1, 2, 3, 4, name='Adhiban', age=18)
# Output:
# First two arguments: 1 2
# Additional positional arguments: (3, 4)
# Additional keyword arguments: {'name': 'Adhiban', 'age': 18}
```

Fig 4.6.5 Args and Kwargs Python Code

```
First two arguments: 1 2
Additional positional arguments: (3, 4)
Additional keyword arguments: {'name': 'Adhiban', 'age': 18}
```

Fig 4.6.6 Args and Kwargs Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Args%20and%20Kwargs.py>

4.7) Lambda Expression, Map and Filter Functions

Lambda Expressions:

Lambda expressions, also known as lambda functions, are small, anonymous functions that can be used for simple operations. They are created using the lambda keyword. Lambda functions are often used in places where a small, simple function is required, such as when passing a function as an argument to another function

```
# Regular function
def square(x):
    return x ** 2

# Equivalent lambda function
square_lambda = lambda x: x ** 2

print(square(4))          # Output: 16
print(square_lambda(4))   # Output: 16
```

Fig 4.7.1 Lambda Expressions Python Code

```
16
16
```

Fig 4.7.2 Lambda Expressions Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Lambda.py>

map() Function:

The map() function applies a given function to all items in an iterable (such as a list) and returns an iterator of the results.

```
numbers = [1, 2, 3, 4, 5]
squared_numbers = map(lambda x: x ** 2, numbers)

# Convert the iterator to a list
squared_numbers_list = list(squared_numbers)

print(squared_numbers_list) # Output: [1, 4, 9, 16, 25]
```

Fig 4.7.3 Map Function Python Code

```
[1, 4, 9, 16, 25]
```

Fig 4.7.4 Map Function Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Map.py>

:

The `filter()` function filters elements of an iterable based on a given function that returns either True or False. It returns an iterator containing the elements for which the function returned True.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = filter(lambda x: x % 2 == 0, numbers)

# Convert the iterator to a list
even_numbers_list = list(even_numbers)

print(even_numbers_list)  # Output: [2, 4, 6, 8, 10]
```

Fig 4.7.5 Filter Function Python Code

```
[2, 4, 6, 8, 10]
```

Fig 4.7.6 Filter Function Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Filter.py>

4.8) Attributes and Class Keywords

In Python, classes are a fundamental concept used for creating objects that encapsulate data and behavior. Classes allow you to define your own data types, organizing attributes (data) and methods (functions) that operate on those attributes. Let's explore attributes and the class keyword in Python.

Attributes:

Attributes are variables that belong to an object. They store data that describes the object's state. In Python classes, attributes are defined within the class and are accessed using the dot notation (object.attribute_name).

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

# Creating an object (instance) of the class
person1 = Person("Adhiban", 18)

# Accessing attributes
print(person1.name)    # Output: Adhiban
print(person1.age)     # Output: 18
```

Fig 4.8.1 Attributes Python Code

In this example, the Person class has two attributes: name and age. The `__init__` method is a special method (constructor) that initializes the object's attributes when an instance is created.

```
Adhiban
18
```

Fig 4.8.2 Attributes Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Attributes.py>

class Keyword:

The class keyword is used to define a class in Python. It is followed by the class name and a colon. The body of the class contains attribute and method definitions.


```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f"{self.name} says Woof!")

# Creating an object of the class
dog1 = Dog("Buddy", 3)

# Accessing attributes and calling methods
print(f"{dog1.name} is {dog1.age} years old.") # Output: Buddy is 3 years old.
dog1.bark() # Output: Buddy says Woof!
```

Fig 4.8.3 Class Keyword Python Code

In this example, the Dog class has attributes name and age, as well as a method bark() that prints a message. The self parameter is used to refer to the instance of the object within methods. Classes and objects are fundamental concepts in object-oriented programming (OOP), allowing you to create reusable and structured code by modeling real-world entities.

```
Buddy is 3 years old.
Buddy says Woof!
```

Fig 4.8.4 class Keyword Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Class.py>

4.9) Class Object Attributes and Methods

Certainly! Here's a comprehensive explanation along with a code example that covers class attributes, instance attributes, and methods in Python classes

Class Attributes:

Class attributes are attributes that are shared among all instances of a class. They are defined directly within the class but outside of any methods. Class attributes are the same for all instances of the class.

```
class Car:
    # Class attribute
    wheels = 4

# Accessing class attribute using the class name
print(Car.wheels)  # Output: 4

# Creating instances of the class
car1 = Car()
car2 = Car()

# Accessing class attribute using instances
print(car1.wheels)  # Output: 4
print(car2.wheels)  # Output: 4
```

Fig 4.9.1 Class Attributes Python Code

```
4
4
4
```

Fig 4.9.2 Class Attributes Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Class_Attributes.py

Instance Attributes:

Instance attributes are specific to each instance of a class. They are defined within the `__init__` method using the `self` parameter. Each instance can have different values for its instance attributes.

```
class Person:
    def __init__(self, name, age):
        # Instance attributes
        self.name = name
        self.age = age

# Creating instances of the class
person1 = Person("Person1", 25)
person2 = Person("Person2", 30)

# Accessing instance attributes
print(person1.name, person1.age) # Output: Person1 25
print(person2.name, person2.age) # Output: Person2 30
```

Fig 4.9.3 Instance Attributes Python Code

```
Person1 25
Person2 30
```

Fig 4.9.4 Instance Attributes Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Instance_Attributes.py

Methods:

Methods are functions defined within a class. They can perform actions using the attributes of the class. The self parameter refers to the instance on which the method is called.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    # Instance method
    def bark(self):
        print(f"{self.name} is barking!")

# Creating an instance of the class
dog1 = Dog("Buddy", 3)

# Calling an instance method
dog1.bark() # Output: Buddy is barking!
```

Fig 4.9.5 Methods Python Code

Buddy is barking!

Fig 4.9.6 Methods Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Methods.py>

In this example, the bark() method is an instance method that uses the instance attributes name to print a message. Classes, attributes, and methods together provide a way to create organized and reusable code structures. Class attributes hold shared information, instance attributes hold specific instance data, and methods perform actions using that data.

4.10) Inheritance and Polymorphism

Inheritance:

Inheritance is a mechanism in which a new class (subclass or derived class) can inherit properties and behaviors (attributes and methods) from an existing class (superclass or base class). This allows you to create a hierarchy of classes with shared attributes and methods, while also allowing subclasses to add or override behaviors.

```
# Base class
class Animal:
    def speak(self):
        pass # Placeholder for the method

# Subclasses inheriting from Animal
class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

# Creating instances of subclasses
dog = Dog()
cat = Cat()

# Calling the overridden methods
print(dog.speak()) # Output: Woof!
print(cat.speak()) # Output: Meow!
```

Fig 4.10.1 Inheritance Python Code

```
Woof!
Meow!
```

Fig 4.10.2 Inheritance Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Inheritance.py>

In this example, the Animal class is the base class, and both Dog and Cat are subclasses. The subclasses inherit the speak() method from the base class and override it to provide their own implementations.

Polymorphism:

In Python, polymorphisms refer to the occurrence of something in multiple forms. As part of polymorphism, a Python child class has methods with the same name as a parent class method. This is an essential part of programming.

Polymorphism is the ability of different classes to be treated as instances of the same class through a common interface. In other words, it allows objects of different classes to be used interchangeably based on shared methods or behaviors

In this example, both Circle and Rectangle classes inherit from the Shape class and provide their own area() methods. By treating instances of different classes as Shape objects, we achieve polymorphism and can calculate their areas using a common interface. Inheritance and polymorphism are powerful tools that enhance code organization, reusability, and extensibility in object-oriented programming.

They allow you to create complex and flexible class structures while maintaining a clear hierarchy of behaviors and attributes as shown in fig4.10.2.

```

# Base class
class Shape:
    def area(self):
        pass # Placeholder for the method

# Subclasses inheriting from Shape
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

# Creating instances of subclasses
circle = Circle(5)
rectangle = Rectangle(4, 6)

# Using polymorphism to calculate areas
shapes = [circle, rectangle]

for shape in shapes:
    print("Area:", shape.area())

```

Fig 4.10.2 Polymorphism Python Code

```

Area: 78.53975
Area: 24

```

Fig 4.10.2 Inheritance Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Polymorphism.py>

4.11) Special (Magic/Dunder) Methods

Special methods, also known as magic methods or dunder methods (short for "double underscore" methods), are predefined methods in Python classes that have special behavior when invoked. These methods are identified by their names enclosed in double underscores, such as `__init__`, `__str__`, `__add__`, and more. They allow you to customize the behavior of your objects and classes in various contexts. Here are some commonly used special methods:

1. `__init__(self, ...)`: The constructor method is called when an object is created. It initializes instance attributes and prepares the object for use.
2. `__str__(self)`: The string representation method is called when the `str()` function is used on an object. It should return a human-readable string representation of the object.
3. `__repr__(self)`: The representation method is called when the `repr()` function is used on an object. It should return a string that can be used to recreate the object.
4. `__len__(self)`: The length method is called when the `len()` function is used on an object. It should return the length of the object.
5. `__getitem__(self, key)`: The indexing method is called when you use square brackets to access an element in an object, like `obj[key]`.

6. `__setitem__(self, key, value)`: The assignment method is called when you use square brackets to assign a value to an element in an object, like `obj[key] = value`.

7. `__delitem__(self, key)`: The deletion method is called when you use the `del` statement to delete an element from an object, like `del obj[key]`.

8. `__add__(self, other)`: The addition method is called when the `+` operator is used with two objects.

9. `__sub__(self, other)`: The subtraction method is called when the `-` operator is used with two objects.

10. `__eq__(self, other)`: The equality method is called when the `==` operator is used with two objects.

11. `__lt__(self, other)`: The less than method is called when the `<` operator is used with two objects.

12. `__gt__(self, other)`: The greater than method is called when the `>` operator is used with two objects.

```

class Book:
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages

    def __str__(self):
        return f"{self.title} by {self.author}"

    def __len__(self):
        return self.pages

    def __add__(self, other):
        total_pages = self.pages + other.pages
        return Book(f"{self.title} + {other.title}", self.author, total_pages)

# Creating book objects
book1 = Book("Python Crash Course", "Eric Matthes", 560)
book2 = Book("Deep Learning", "Ian Goodfellow", 800)

# Using special methods
print(book1)          # Output: Python Crash Course by Eric Matthes
print(len(book1))     # Output: 560

combined_book = book1 + book2
print(combined_book)  # Output: Python Crash Course + Deep Learning by Eric Matthes
print(len(combined_book)) # Output: 1360

```

Fig 4.11.1

Special (Magic/Dunder) Methods Python Code

```

Python Crash Course by Eric Matthes
560
Python Crash Course + Deep Learning by Eric Matthes
1360

```

Fig 4.11.2 Special (Magic/Dunder) Methods Python Output

More details and the project files can be found here:

[https://github.com/quantanics2018/Python/blob/main/special%20\(magic%2Cdunder\)%20methods.py](https://github.com/quantanics2018/Python/blob/main/special%20(magic%2Cdunder)%20methods.py)

In this example, we've defined special methods `__str__`, `__len__`, and `__add__` in the `Book` class to customize string representation, length calculation, and addition behavior of book objects.

Topic 5: Advanced Python Concepts

5.1) Modules and packages

Modules and packages are fundamental concepts in Python that help organize and manage code in a modular and reusable way. Let's explore what modules and packages are and how to use them:

:

A module is a single file containing Python code that defines functions, classes, variables, or statements that can be imported and used in other Python scripts or modules. Modules allow you to break down your code into smaller, manageable pieces and promote code reusability.

Here's an example of a simple module named `math_operations.py`:

```
math_operations.py

# math_operations.py

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

Fig 5.1.1 `math_operations.py`

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Math_Opertation/math_operation

main_script.py

```
# main_script.py

import math_operations

result = math_operations.add(5, 3)
print(result) # Output: 8
```

Fig 5.1.2 math_operations.py

8

Fig 5.1.3 Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Math_Opertation/main.py

Packages:

A package is a collection of modules organized in directories and subdirectories. It allows you to create a hierarchical structure for your codebase. Packages make it easier to manage and distribute large projects. In this example, my_package is a package containing modules module1.py and module2.py. It also contains a subpackage named subpackage with its own modules module3.py. The __init__.py files are necessary to make directories behave as Python packages. To use modules and submodules from packages, you can import them in a similar way as modules: In addition to regular packages, Python also provides standard libraries and third-party libraries that are packaged as modules and packages. These libraries offer a wide range of functionalities that you can leverage in your projects. Using modules and packages enhances code organization, promotes reusability, and simplifies collaboration in larger projects.

5.2) name and “main”

In Python, the `__name__` and `__main__` attributes are commonly used for controlling the execution of code in modules and scripts. They provide a way to determine whether a Python file is being run as the main program or being imported as a module into another program.

`__name__` Attribute:

The `__name__` attribute is a special variable in Python that is automatically set for each module or script. When a Python script is executed, the `__name__` attribute is set to `"__main__"` if the script is the main program being run. If the script is being imported as a module into another program, the `__name__` attribute is set to the name of the module (i.e., the filename without the `.py` extension).

In this example, if you run `module_example.py` as the main program, the code within the `if __name__ == "__main__":` block will be executed. However, if you import `module_example.py` as a module into another script, the code within the block will not be executed.

```
# module_example.py

def say_hello():
    print("Hello from module_example!")

print("Module's name:", __name__)

if __name__ == "__main__":
    say_hello()
```

Fig 5.2.1 name and “main” Python Code

```
Module's name: __main__
Hello from module_example!
```

Fig 5.2.2 name and “main” Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Name_And_main/module_example.py

Using `if __name__ == "__main__":` for Script Execution:

This construct is often used to include code that should only run when the script is executed as the main program and not when it's imported as a module. It's commonly used to create executable scripts that can be run from the command line or other environments.

```
# main_script.py

import module_example

print("Main script's name:", __name__)
```

Fig 5.2.3 Using `if __name__ == "__main__":`: Python Code

Module's name: `module_example`

Main script's name: `__main__`

Fig 5.2.4 Using `if __name__ == "__main__":`: Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Name_And_main/main_script1.py

Advantages of Using `if __name__ == "__main__":`:

Allows a script to be both a standalone program and an importable module. Prevents unintended code execution when a script is imported. Enables modular and organized code structure.

Summary:

The `__name__` and `__main__` attributes are important for distinguishing between the main program and imported modules in Python. By using the `if __name__ == "__main__":` construct, you can control the execution of code in scripts to ensure that certain parts of the code are only executed when the script is run as the main program.

5.3) Error and exception handling

Error and exception handling in Python allows you to handle unexpected situations and errors that can occur during the execution of a program. Python provides a structured way to catch, handle, and manage exceptions, ensuring that your program doesn't crash abruptly and providing helpful error messages for debugging. Here's how error and exception handling works:

Types of Errors:

In Python, errors are divided into three main categories:

Syntax Errors: These occur when you violate the syntax rules of the Python language. They are usually detected by the Python interpreter during the parsing phase.

Runtime Errors (Exceptions): These occur during the execution of a program. They are typically caused by issues like division by zero, attempting to access an index out of bounds, or calling a method on an incompatible object.

Logical Errors: Also known as semantic errors, these occur when the code is logically incorrect and produces unintended or unexpected results. These errors are harder to detect because the program runs successfully, but the output is not as expected.

Exception Handling:

Python provides a `try-except` block to catch and handle exceptions. This allows you to gracefully handle errors without abruptly stopping the program's execution. Here's a basic example:

```
try:
    numerator = 10
    denominator = 0
    result = numerator / denominator # Raises ZeroDivisionError
except ZeroDivisionError:
    print("Error: Division by zero")
```

Fig 5.3.1 Exception Handling Python Code

```
Error: Division by zero
```

Fig 5.3.2 Exception Handling Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Exception_handling.py

In this example, the try block contains code that might raise an exception (division by zero). If an exception of the specified type (ZeroDivisionError) occurs, the except block is executed, and an error message is printed.

5.4) Pylint Overview

Pylint is a popular static code analysis tool for Python that helps you identify and fix potential issues, coding style violations, and errors in your Python code. It enforces coding standards and best practices, improving the quality, readability, and maintainability of your codebase. Pylint examines your code and provides feedback on various aspects, such as syntax errors, naming conventions, code complexity, and more.

Key Features of Pylint:

Code Style Enforcement: Pylint checks your code against the PEP 8 style guide, which is the official style guide for Python code.
Syntax and Semantic Checks: Pylint identifies syntax errors, name errors, undefined variables, and more.
Code Complexity Analysis: Pylint calculates code complexity metrics such as Cyclomatic Complexity and maintains a balance

between code complexity and maintainability. Potential Bugs Detection: Pylint identifies potential bugs, such as using variables before assignment or comparing variables with None without using the is operator. Consistency Checks: Pylint enforces consistency in your code by checking for consistent indentation, whitespace, and other formatting issues.

Docstring Checks: Pylint helps you maintain good documentation by checking the quality of your docstrings. Customizable Configuration: Pylint's behavior can be customized using configuration files or command-line arguments to suit your project's coding style and requirements.

Installing Pylint:

You can install Pylint using the following pip command:

```
pip install pylint
```

Running Pylint:

To run Pylint on your Python code, use the following command: Here instead of youtube_download.py, Replace with how you have saved the python file.

```
pylint youtube_download.py
```

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/pylint.py>

Suppressing Warnings:

You can suppress specific warnings using inline comments or configuration files. For example, if you want to ignore a specific warning, you can use the # pylint: disable=warning-id comment in your code.

Using Pylint in IDEs:

Pylint is integrated with many popular integrated development environments (IDEs) like Visual Studio Code, PyCharm, and more. These IDEs can highlight issues detected by Pylint directly within the editor, helping you fix issues in real-time.

Pylint is a powerful tool that can significantly improve code quality and maintainability. By using it regularly, you can develop cleaner, more consistent, and less error-prone Python code.

5.5) Decorators with Python Overview

Decorators are a powerful and flexible feature in Python that allow you to modify or enhance the behavior of functions or methods without changing their actual code. Decorators are often used for tasks such as logging, timing, authentication, and code reuse. They provide a clean and elegant way to extend the functionality of functions or methods

In Python, functions are first-class objects, which means they can be passed around and used as arguments to other functions. Decorators leverage this property to wrap a target function with additional functionality by defining a higher-order function (a function that takes another function as an argument). The higher-order function typically returns a modified version of the original function.

Here's a basic example of how a decorator is defined and used:

```
def my_decorator(func):  
    def wrapper():  
        print("Something is happening before the function is called.")  
        func()  
        print("Something is happening after the function is called.")  
    return wrapper  
  
@my_decorator  
def say_hello():  
    print("Hello!")  
  
say_hello()
```

Fig 5.5.1 Defining a Decorator Python Code

```
Something is happening before the function is called.  
Hello!  
Something is happening after the function is called.
```

Fig 5.5.2 Defining a Decorator Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Defining_Decorator.py

In this example, `my_decorator` is a decorator function that takes another function (`func`) as an argument and returns a new function (`wrapper`). The `wrapper` function adds behavior before and after calling the original function. The `@my_decorator` syntax is a shorthand for applying the decorator to the `say_hello` function.

Decorator Chaining:

You can apply multiple decorators to a single function, and they will be applied in the order they appear. In this example, the `repeat` decorator accepts an argument (`num`) and applies the decorated function multiple times. Decorators are a powerful tool for enhancing the behavior of functions in a reusable and organized way. They are commonly used to separate concerns, add cross-cutting functionality, and maintain cleaner code. As shown in the fig 5.5.2

```
def uppercase_decorator(func):  
    def wrapper():  
        result = func()  
        return result.upper()  
    return wrapper  
  
def exclamation_decorator(func):  
    def wrapper():  
        result = func()  
        return result + "!"  
    return wrapper  
  
@exclamation_decorator  
@uppercase_decorator  
def greet():  
    return "hello"  
  
print(greet()) # Output: HELLO!
```

Fig 5.5.2 Defining a Decorator chaining Python Code

HELLO!

Fig 5.5.4 Decorator Chaining Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Passing_Argument_to_decorator.py

In this example, the repeat decorator accepts an argument (num) and applies the decorated function multiple times. Decorators are a powerful tool for enhancing the behavior of functions in a reusable and organized way. They are commonly used to separate concerns, add cross-cutting functionality, and maintain cleaner code.

5.6) Generators with Python

Generators are a special type of iterable in Python that allow you to create iterators in a more memory-efficient and concise manner. They are a way to generate values on-the-fly, rather than storing them in memory all at once. Generators are particularly useful when working with large datasets or when you want to generate values dynamically without consuming excessive memory.

Creating a Generator:

Generators are created using generator functions or generator expressions.

Generator Functions:

Generator functions are defined like regular functions, but instead of using the return statement, they use the yield keyword to yield a value one at a time.

```
def countdown(n):  
    while n > 0:  
        yield n  
        n -= 1  
  
# Using the generator function  
for i in countdown(5):  
    print(i) # Output: 5, 4, 3, 2, 1
```

Fig 5.6.1 Generator Functions Python Code

```
5  
4  
3  
2  
1
```

Fig 5.6.2 Generator Functions Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Generator_function.py

Generator Expressions:

Generator expressions are similar to list comprehensions, but they use parentheses instead of square brackets. They generate values one at a time as you iterate over them.

```
even_squares = (x ** 2 for x in range(10) if x % 2 == 0)

for value in even_squares:
    print(value)  # Output: 0, 4, 16, 36, 64
```

Fig 5.6.3 Generator Expressions Python Code

```
0
4
16
36
64
```

Fig 5.6.4 Generator Expressions Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Generator_expression.py

Advantages of Generators:

Memory Efficiency: Generators generate values one at a time and do not store the entire sequence in memory, making them suitable for large datasets.

Lazy Evaluation: Values are generated on-the-fly when requested, which is useful for infinite sequences or when you don't need all values at once.

Simplicity: Generator functions are often simpler to write and understand compared to writing custom iterator classes.

Performance: Generators can be more efficient than creating and managing temporary lists or sequences

Built-in Generator Functions:

- Python provides built-in functions that return generators:
- `range()`: Generates a sequence of numbers.
- `map()`: Applies a function to each item in an iterable.
- `filter()`: Filters elements based on a function's condition.

5.7) Python Collection Modules

The collections module in Python provides specialized container data types that enhance the functionality of built-in data types like lists, tuples, and dictionaries. These specialized data types offer additional features and optimizations for various use cases. Let's explore some of the key classes provided by the collections module:

Counter: The Counter class is used for counting occurrences of elements in a collection, typically in a dictionary-like structure. It's particularly useful for creating frequency distributions

```
from collections import Counter

text = "python programming"
counter = Counter(text)

print(counter) # Output: Counter({'p': 2, 'r': 2, 'o': 2, 'g': 2, 'm': 2, 'y': 1, 't': 1, ' ': 1, 'a': 1, 'i': 1, 'n': 1})
```

Fig 5.7.1 Counter Python Code

```
Counter({'p': 2, 'o': 2, 'n': 2, 'r': 2, 'g': 2, 'm': 2, 'y': 1, 't': 1, 'h': 1, ' ': 1, 'a': 1, 'i': 1})
```

Fig 5.7.2 Counter Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Counter.py>

defaultdict: The defaultdict class is a subclass of the built-in dict class that provides a default value for missing keys.

```
from collections import defaultdict

name_age = defaultdict(int)
name_age['Person1'] = 20

print(name_age['Person1']) # Output: 20
print(name_age['Bob'])     # Output: 0 (default value for int)
```

Fig 5.7.3 defaultdict Python Code

```
20
0
```

Fig 5.7.4 defaultdict Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Default_dict.py

deque: The deque class (short for "double-ended queue") provides a data structure similar to lists but with fast $O(1)$ appends and pops from

```
from collections import deque

queue = deque()
queue.append(1)
queue.append(2)
queue.append(3)

print(queue.popleft()) # Output: 1 (FIFO order)
```

Fig 5.7.5 deque Python Code

1

Fig 5.7.6 deque Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Dequeue.py>

namedtuple: The namedtuple class creates a new tuple subclass with named fields. It provides more readable and self-documenting code when dealing with tuples

```
from collections import namedtuple

Person = namedtuple('Person', ['name', 'age'])
person = Person(name='Person1', age=25)

print(person.name, person.age)  # Output: Person1 25
```

Fig 5.7.7 namedtuple Python Code

Person1 25

Fig 5.7.8 namedtuple Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Name_tuple.py

OrderedDict: The OrderedDict class maintains the order of keys as they were inserted. Unlike the built-in dict class, an OrderedDict remembers the order of items.

```
from collections import OrderedDict

data = OrderedDict()
data['first'] = 1
data['second'] = 2
data['third'] = 3

print(data) # Output: OrderedDict([('first', 1), ('second', 2), ('third', 3)])
```

Fig 5.7.9 OrderedDict Python Code

```
OrderedDict([('first', 1), ('second', 2), ('third', 3)])
```

Fig 5.7.10 OrderedDict Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Ordered_Dict.py

These are just a few examples of the classes provided by the collections module. These specialized data types help you tackle various programming challenges with enhanced performance and readability.

5.8) Opening and Reading Files and Folders

python datetime module

The datetime module in Python provides classes for working with dates, times, and time intervals. It allows you to manipulate, format, and calculate dates and times with ease. Let's explore some of the key classes and functions provided by the datetime module:

1. datetime Class:

The datetime class represents a combination of date and time. It has attributes like year, month, day, hour, minute, second, and microsecond.

```
from datetime import datetime

current_datetime = datetime.now()
print(current_datetime) # Output: Current date and t
print(current_datetime.year)
print(current_datetime.month)
print(current_datetime.day)
print(current_datetime.hour)
print(current_datetime.minute)
print(current_datetime.second)
```

Fig 5.8.1 datetime Class Python Code

```
2024-06-19 10:15:09.516390
2024
6
19
10
15
9
```

Fig 5.8.2 datetime Class Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Datetime.py>

2. date Class:

The date class represents a date (year, month, and day) without time information.

```
from datetime import date

today = date.today()
print(today) # Output: Current date
print(today.year)
print(today.month)
print(today.day)
```

Fig 5.8.3 date Class Python Code

```
2024-06-19
2024
6
19
```

Fig 5.8.4 date Class Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Date.py>

3. time Class:

The time class represents time without date information.

```
from datetime import time

current_time = time(hour=12, minute=30, second=45)
print(current_time) # Output: 12:30:45
print(current_time.hour)
print(current_time.minute)
print(current_time.second)
```

Fig 5.8.5 time Class Python Code

```
12:30:45
12
30
45
```

Fig 5.8.6 time Class Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Time.py>

4. Formatting Dates and Times:

You can format dates and times using the `strftime()` method. Here are some common format codes:

- %Y: Year (four digits)
- %m: Month (zero-padded)
- %d: Day of the month (zero-padded)
- %H: Hour (24-hour clock)
- %M: Minute
- %S: Second

5. Parsing Dates and Times:

You can parse strings containing dates and times using the `strptime()` method.

6. Timedelta:

The `timedelta` class represents the difference between two dates or times. The `datetime` module provides a comprehensive set of tools for handling dates and times. It's useful for tasks ranging from simple date formatting to complex date calculations.

5.9) Python Math and Random Module

The Python `math` and `random` modules are part of the Python Standard Library and provide functions for mathematical operations and generating random numbers. Let's explore these modules in more detail:

1. math Module:

The math module provides various mathematical functions and constants that allow you to perform common mathematical operations. Here are some key features of the math module:

```
import math

# Mathematical constants
print(math.pi)           # Output: 3.141592653589793
print(math.e)            # Output: 2.718281828459045

# Basic mathematical functions
print(math.sqrt(25))      # Output: 5.0
print(math.pow(2, 3))     # Output: 8.0
print(math.exp(2))        # Output: 7.38905609893065
print(math.log(10))       # Output: 2.302585092994046

# Trigonometric functions (radians)
print(math.sin(math.pi/2)) # Output: 1.0
print(math.cos(math.pi))  # Output: -1.0
```

Fig 5.9.1 math Module Python Code

```
3.141592653589793
2.718281828459045
5.0
8.0
7.38905609893065
2.302585092994046
1.0
-1.0
```

Fig 5.9.2 math Module Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Math_module.py

2. random Module:

The Random Function returns a number selected at random within the range indicated by the low and high values. RAND(low, high) low. Lowest possible random value.

The random module in Python defines a series of functions for generating or manipulating random integers. The import random loads the random module, which contains a number of random number generation-related functions.

Python Random module generates random numbers in Python. These are pseudo-random numbers means they are not truly random.

This module can be used to perform random actions such as generating random numbers, printing random a value for a list or string, etc. It is an in-built function in Python

The random module provides functions to generate random numbers, sequences, and perform random selections. Here are some common uses of the random module:

```
import random

# Generate random float in [0.0, 1.0)
print(random.random())

# Generate random integer in a range [a, b]
print(random.randint(1, 10))

# Random choice from a sequence
my_list = [1, 2, 3, 4, 5]
print(random.choice(my_list))

# Shuffling elements of a sequence
random.shuffle(my_list)
print(my_list)

# Generating random sample without replacement
random_sample = random.sample(my_list, 3)
print(random_sample)
```

Fig 5.9.2 math Module Python Code

```
0.7146964408814545
2
5
[3, 4, 1, 2, 5]
[2, 3, 1]
```

Fig 5.9.4 random Module Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Random_module.py

The random module is useful for tasks like simulations, games, randomization, and testing. Keep in mind that the random numbers generated are pseudorandom and are based on an initial seed value. You can use `random.seed()` to set the seed explicitly for reproducibility.

5.10) Python Debugger

The Python debugger, often referred to as "pdb" (Python Debugger), is a built-in module that allows you to interactively debug your code by setting breakpoints, inspecting variables, and stepping through your code's execution. It's a powerful tool for identifying and fixing errors in your programs. Here's an overview of how to use the Python debugger:

1. Importing the Debugger:

To use the debugger, you need to import the pdb module at the beginning of your script.

```
import pdb
```

2. Setting Breakpoints:

You can set breakpoints in your code using the `pdb.set_trace()` function. When the program reaches this point, it will pause, and you can start interacting with the debugger.

```
import pdb

def my_function():
    x = 10
    y = 20
    pdb.set_trace()
    result = x + y
    print(result)

my_function()
```

Fig 5.10.1 Setting Breakpoints Python Code

3. Debugger Commands:

When the debugger is active, you can use various commands to navigate through your code and inspect variables. Here are some common commands:

- c: Continue execution until the next breakpoint.
- n: Continue execution until the next line in the current function.
- s: Step into the current function.
- q: Quit the debugger.
- p variable_name: Print the value of the specified variable.
- l: List the source code around the current line.
- b line_number: Set a breakpoint at the specified line number.
- h: Display a list of available debugger commands.

4. Exiting the Debugger:

You can exit the debugger by typing q or using the Ctrl+D keyboard shortcut.

5. Running the Script with the Debugger:

To run your script with the debugger, you can execute it using the Python interpreter with the -m pdb flag followed by the script name:

```
python -m pdb your_script.py
```

```

> d:\python\pop_2.py(7)my_function()
-> result = x + y
(Pdb) h

Documented commands (type help <topic>):
=====
EOF      c          d          h          list       q          rv         undisplay
a        cl         debug      help       ll         quit      s          unt
alias    clear      disable    ignore     longlist   r         source    until
args     commands  display    interact   n          restart   step      up
b        condition down       j          next       return    tbreak    w
break    cont       enable     jump       p          retval    u          whatis
bt       continue  exit       l          pp         run       unalias   where

Miscellaneous help topics:
=====
exec  pdb

(Pdb)

```

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Python_Debugger.py

Alternatively, you can insert the `pdb.set_trace()` function directly into your script, as shown in the example above. Using the debugger can be immensely helpful in diagnosing and fixing issues in your code. It allows you to see the state of variables, step through code execution, and gain a deeper understanding of how your program works.

5.11) Python Regular Expressions

Regular expressions (regex or regexp) in Python are powerful tools for working with patterns in strings. They allow you to search, match, and manipulate text based on specified patterns. The `re` module in Python provides functions to work with regular expressions. Here's an overview of how regular expressions work in Python:

1. Importing the re Module:

You need to import the re module before using regular expressions.

```
import re
```

Fig 5.11.1 Importing the re Module

2. Basic Pattern Matching:

The re.search() function is used to search for a specified pattern within a string.

```
import re
pattern= r"world"
text = "Hello, world!"
match = re.search(pattern, text)
if match():
    print("Pattern found:",match.groups())
else:
    print("Pattern is not found")
```

Fig 5.11.2 Basic Pattern Matching

```
Pattern found: world
```

Fig 5.11.3 re Python Code

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Basic_patten_matchng.py

3. Matching and Extracting:

You can use capturing groups () to extract parts of a pattern.

```
pattern = r"(\d{2})-(\d{2})-(\d{4})"
text = "Date: 21-08-2023"
match = re.search(pattern, text)
if match:
    print("Full match:", match.group(0))
    print("Day:", match.group(1))
    print("Month:", match.group(2))
    print("Year:", match.group(3))
```

Fig 5.11.4 Matching and Extracting Python Code

```
Full match: 21-08-2023
Day: 21
Month: 08
Year: 2023
```

Fig 5.11.5 Matching and Extracting Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Basic_patten_matchng.py

4. Finding All Matches:

The `re.findall()` function returns a list of all non-overlapping matches of the pattern in the string

```
pattern = r"\b\w+\b"
text = "Hello, world! How are you today?"
matches = re.findall(pattern, text)
print(matches) # Output: ['Hello', 'world', 'How', 'are', 'you', 'today']
```

Fig 5.11.6 Finding All Matches Python Code

```
['Hello', 'world', 'How', 'are', 'you', 'today']
```

Fig 5.11.7 Finding All Matches Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Finding_all_matches.py

5. Substitution:

You can use the `re.sub()` function to replace matches with a specified string.

```
pattern = r"\d+"
text = "There are 123 apples and 456 bananas."
new_text = re.sub(pattern, "X", text)
print(new_text) # Output: There are X apples and X bananas.
```

Fig 5.11.8 Substitution Matches Python Code

```
There are X apples and X bananas.
```

Fig 5.11.9 Substitution Matches Python Output

More details and the project files can be found here:

<https://github.com/quantanics2018/Python/blob/main/Substitution.py>

6. Flags and Modifiers:

You can use flags to modify how the regular expressions work, such as making them case-insensitive.

```
pattern = r"apple"
text = "An APPLE and an Apple."
match = re.search(pattern, text, re.IGNORECASE)
if match:
    print("Pattern found:", match.group())
```

Fig 5.11.10 Flags and Modifiers Python Code

```
Pattern found: APPLE
```

Fig 5.11.11 Flags and Modifiers Python Output

More details and the project files can be found here:

https://github.com/quantanics2018/Python/blob/main/Flags_and_modifiers.py

These are just some basic examples of using regular expressions in Python. Regular expressions provide a powerful way to work with text patterns, allowing you to search, extract, and manipulate strings effectively. Keep in mind that regular expressions can become complex, and it's a good practice to test and experiment with your patterns before using them in production code.

5.12) Timing Your Python Code

Timing your Python code allows you to measure the execution time of specific parts of your program. This can help you identify performance bottlenecks and optimize your code. There are multiple ways to measure execution time in Python. Let's explore a few techniques:

1. Using the time Module:

The time module provides functions for working with time-related operations. You can use the `time.time()` function to measure the time before and after code execution, allowing you to calculate the time difference. More details and object can be found be here https://github.com/quantanics2018/Python/blob/main/Using_the_time_module.py

```

import time

start_time = time.time()
#Replacw with your code
a=10
b=10
print(a+b)
end_time = time.time()
elapsed_time = end_time - start_time
print(f"Elapsed time: {elapsed_time} seconds")

```

Fig 5.12.1 Using the time Module Python Code

```

20
Elapsed time: 0.0 seconds

```

Fig 5.12.2 Using the time Module Python Output

2. Using the time Module

The timeit module is designed specifically for timing small code snippets. It provides a simple and accurate way to measure execution time. More details and object can be found here https://github.com/quantanics2018/Python/blob/main/Using_the_timeit_module.py

```

import timeit

code_to_measure = """
# Your code here
a=10
b=10
c=a+b
print(c)
"""

elapsed_time = timeit.timeit(stmt=code_to_measure, number=1000)
print(f"Average time per loop: {elapsed_time / 1000} seconds")

```

Fig 5.12.3 Using the timeit Module Python Output


```
20
20
20
20
20
20
20
20
20
20
20
20
20
20
Average time per loop: 0.007177167299902067 seconds
```

Fig 5.12.4 Using the time it Module Python Output

3. Using the datetime Module:

You can also use the datetime module to measure execution time. The datetime module provides a higher level of accuracy for measuring small time intervals.

More details and objects can be found here

https://github.com/quantanics2018/Python/blob/main/Using_the_datetime_module.py

```
from datetime import datetime

start_time = datetime.now()

# Your code here
a=10
b=20
print(a+b)
end_time = datetime.now()
elapsed_time = end_time - start_time
print(f"Elapsed time: {elapsed_time}")
```

Fig 5.12.5 Using the datetime Module Python Code

```
Elapsed time: 0:00:00
```

Fig 5.12.6 Using the datetime Module Python Output

4.:

For more in-**Using Profiling Tools**depth analysis of code performance, you can use profiling tools like cProfile or external tools like line_profiler and memory_profiler.

```
import cProfile

def my_function():
    # Your code here

cProfile.run("my function()")
```

Fig 5.12.7 Using Profiling Tools Python Code

```
4 function calls in 0.000 seconds

Ordered by: standard name

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000   0.000   0.000   0.000 <string>:1(<module>)
1      0.000   0.000   0.000   0.000 datetime module.py:3(my_function)
1      0.000   0.000   0.000   0.000 {built-in method builtins.exec}
1      0.000   0.000   0.000   0.000 {method 'disable' of '_lsprof.Profiler' objects}
```

Fig 5.12.8 Using Profiling Tools Python Output

When using profiling tools, you can get detailed information about the time each function takes and the number of times it's called. More details and objects can be found here https://github.com/quantanics2018/Python/blob/main/Using_profiling_tools.py

5.13) Zipping and Unzipping files using Python

Zipping and unzipping files in Python can be accomplished using the zipfile module, which is part of the Python standard library. This module provides classes and methods to work with ZIP archive files. Here's how you can zip and unzip files using the zipfile module:

1. Zipping Files

To create a new ZIP archive and add files to it, you can use the ZipFile class.

```
import zipfile

# List of files to zip
files_to_zip = ['child.html', 'resume.html']

# Create a new ZIP archive
with zipfile.ZipFile('my_archive.zip', 'w') as zipf:
    for file in files_to_zip:
        zipf.write(file)
```

Fig 5.13.1 Zipping Files Python Code

```
>>> |=====
```

Fig 5.13.2 Zipping Files Python Output

 resume.html	07-06-2023 09:55 AM	Chrome HTML Docu...	2 KB
 unins000.dat	14-10-2022 04:49 PM	DAT - MPEG video file	20,614 KB
 real.java	22-07-2023 12:54 PM	JAVA File	2 KB
 esp_idf.json	14-10-2022 04:50 PM	JSON Source File	1 KB
 idf-env.json	14-10-2022 04:49 PM	JSON Source File	1 KB
 AI_ML (1).pptx	08-02-2023 11:46 AM	Microsoft PowerPoint...	5,003 KB
 Blockchain (1).pptx	08-02-2023 12:11 PM	Microsoft PowerPoint...	800 KB
 Blockchain.pptx	07-02-2023 01:07 PM	Microsoft PowerPoint...	527 KB
 Cloud Computing.pptx	07-02-2023 01:06 PM	Microsoft PowerPoint...	1,429 KB
 iot-james ready (1).pptx	07-02-2023 12:43 PM	Microsoft PowerPoint...	4,028 KB
 iot-james ready (2).pptx	07-02-2023 12:43 PM	Microsoft PowerPoint...	4,487 KB
 iot-james ready.pptx	07-02-2023 12:33 PM	Microsoft PowerPoint...	4,028 KB
 Presentation (1) (1).pptx	07-02-2023 01:21 PM	Microsoft PowerPoint...	4,708 KB
 ESP32 LED Blink Code.docx	17-10-2022 05:40 PM	Microsoft Word Doc...	3,177 KB
 Leaf disease.docx	15-11-2022 07:25 PM	Microsoft Word Doc...	16 KB
 To Send a message in python using gmail.docx	13-11-2022 09:49 PM	Microsoft Word Doc...	1,873 KB
 2022-10-18 17-54-14.mkv	18-10-2022 05:54 PM	MKV File	1,574 KB
 1.mp4	21-07-2023 03:13 PM	MP4 File	4,554 KB
 ddc.py	09-06-2023 04:20 PM	Python File	7 KB
 Zippung.py	04-09-2023 06:11 PM	Python File	1 KB
 idf_cmd_init.bat	19-08-2022 05:37 PM	Windows Batch File	4 KB
 Initialize-Idf.ps1	19-08-2022 05:37 PM	Windows PowerShell ...	6 KB
 my_archive.zip	04-09-2023 06:05 PM	WinRAR ZIP archive	3 KB

Fig 5.13.3 Zipping Files Python Output

In this example, a new ZIP archive named `my_archive.zip` is created, and the files listed in `files_to_zip` are added to the archive. More details and object can be found here https://github.com/quantanics2018/Python/blob/main/Zipping_Files.py

2. Unzipping Files

To extract files from a ZIP archive, you can use the `extractall()` method of the `ZipFile` class. This will extract all the files from the `my_archive.zip` archive into the `extracted_files` directory.

3. Zipping a Directory

To zip an entire directory, you can use the `shutil` module in combination with the `zipfile` module. More details and objects can be found [here](https://github.com/quantanics2018/Python/blob/main/Zippping_a_directory.py)

```
import zipfile
import shutil
import os

# Directory to zip
directory_to_zip = 'F:\basicjava'

# Create a new ZIP archive
with zipfile.ZipFile('my_directory.zip', 'w') as zipf:
    for foldername, subfolders, filenames in os.walk(directory_to_zip):
        for filename in filenames:
            file_path = os.path.join(foldername, filename)
            arcname = os.path.relpath(file_path, directory_to_zip)
            zipf.write(file_path, arcname)
```

Fig 5.13.4 UnZippping Files Python Code

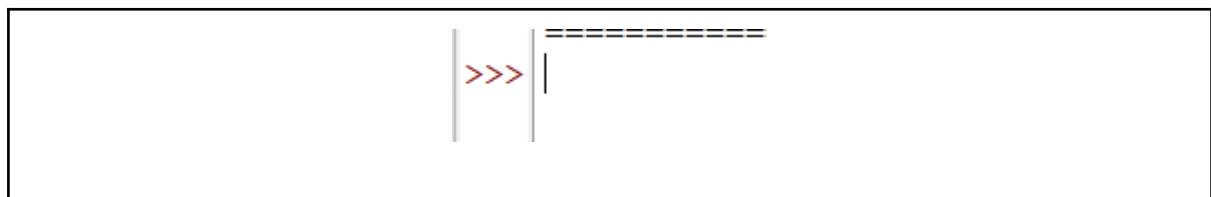


Fig 5.13.5 UnZippping Files Python Output

Note: Don't forget to import the necessary modules (`zipfile` and `shutil`) and handle exceptions appropriately when working with files and directories.

	unins000.dat	14-10-2022 04:49 PM	DAT - MPEG video file	20,614 KB
	real.java	22-07-2023 12:54 PM	JAVA File	2 KB
	esp_idf.json	14-10-2022 04:50 PM	JSON Source File	1 KB
	idf-env.json	14-10-2022 04:49 PM	JSON Source File	1 KB
	AI_ML (1).pptx	08-02-2023 11:46 AM	Microsoft PowerPoint...	5,003 KB
	Blockchain (1).pptx	08-02-2023 12:11 PM	Microsoft PowerPoint...	800 KB
	Blockchain.pptx	07-02-2023 01:07 PM	Microsoft PowerPoint...	527 KB
	Cloud Computing.pptx	07-02-2023 01:06 PM	Microsoft PowerPoint...	1,429 KB
	iot-james ready (1).pptx	07-02-2023 12:43 PM	Microsoft PowerPoint...	4,028 KB
	iot-james ready (2).pptx	07-02-2023 12:43 PM	Microsoft PowerPoint...	4,487 KB
	iot-james ready.pptx	07-02-2023 12:33 PM	Microsoft PowerPoint...	4,028 KB
	Presentation (1) (1).pptx	07-02-2023 01:21 PM	Microsoft PowerPoint...	4,708 KB
	ESP32 LED Blink Code.docx	17-10-2022 05:40 PM	Microsoft Word Doc...	3,177 KB
	Leaf disease.docx	15-11-2022 07:25 PM	Microsoft Word Doc...	16 KB
	To Send a message in python using gmail.docx	13-11-2022 09:49 PM	Microsoft Word Doc...	1,873 KB
	2022-10-18 17-54-14.mkv	18-10-2022 05:54 PM	MKV File	1,574 KB
	1.mp4	21-07-2023 03:13 PM	MP4 File	4,554 KB
	ddc.py	09-06-2023 04:20 PM	Python File	7 KB
	Zippung.py	04-09-2023 06:11 PM	Python File	1 KB
	idf_cmd_init.bat	19-08-2022 05:37 PM	Windows Batch File	4 KB
	Initialize-Idf.ps1	19-08-2022 05:37 PM	Windows PowerShell ...	6 KB
	my_archive.zip	04-09-2023 06:05 PM	WinRAR ZIP archive	3 KB
	my_directory.zip	04-09-2023 06:11 PM	WinRAR ZIP archive	1 KB

Fig 5.13.6 UnZipping Files Python Output

The zipfile module provides various other functionalities, such as adding files to an existing archive, extracting specific files, and working with password-protected archives. Refer to the official Python documentation for more advanced usage and options: [zipfile - Python Standard Library](<https://docs.python.org/3/library/zipfile.html>)

5.14) Setting Up Web Scraping Libraries

Setting up web scraping libraries in Python involves installing and using tools that allow you to extract information from websites. Two popular libraries for web scraping are BeautifulSoup and requests.

```
pip install beautifulsoup4
```

```
pip install requests
```

5.15) Grabbing a Title

To grab the title of a webpage using web scraping, you can use the BeautifulSoup library to parse the HTML content and extract the title tag.

Here's a step-by-step example:

```
from bs4 import BeautifulSoup
import requests

url = "https://www.w3schools.com/js/default.asp"
response = requests.get(url)
if response.status_code == 200:
    html_content = response.content
    soup = BeautifulSoup(html_content, "html.parser")
    title = soup.title.text
    paragraphs = soup.find_all("p")

    print("Title:", title)
    print("Paragraphs:")
    for p in paragraphs:
        print(p.text)
```

Fig 5.15.1 Grabbing a Title Python Code

```
Title: JavaScript Tutorial
```

Fig 5.15.2 Grabbing a Title Python Output

More details and objects can be found here

https://github.com/quantanics2018/Python/blob/main/Grabbing_a_title.py

In this example, the script sends an HTTP request to the specified URL using the requests library. If the response status code is 200 (OK), it means the request was successful. The HTML content of the webpage is then parsed using BeautifulSoup. The `soup.find("title")` line finds the title tag within the HTML content, and the `title_tag.text` extracts the text content of the title tag. Remember to replace the url variable with the actual URL of the webpage you want to scrape. Keep in mind that the structure of web pages can vary, and the example assumes that the title tag is present and follows standard HTML structure. In more complex cases, you might need to handle different HTML structures and handle exceptions for missing or unexpected elements.

5.16) Grabbing an Image

To grab an image from a webpage using web scraping, you can use the BeautifulSoup library to parse the HTML content and extract the image tags. Here's an example of how to do this:

```
from bs4 import BeautifulSoup
import requests

url = "https://www.w3schools.com/js/default.asp"
response = requests.get(url)
if response.status_code == 200:
    html_content = response.content
    soup = BeautifulSoup(html_content, "html.parser")
    title = soup.title.text
    paragraphs = soup.find_all("p")

    print("Title:", title)
    print("Paragraphs:")
    for p in paragraphs:
        print(p.text)
```

Fig 5.16.1 Grabbing a Image Python Code


```
Image Source: /images/colorpicker2000.png
Image Source: /signup/lynxlogo.svg
Image Source: /images/mylearning.png
Image Source: /images/img_falilla_up_300.png
Image Source: /images/colorpicker2000.png
```

Fig 5.16.2 Grabbing a Image Python Output

More details and object can be found here
https://github.com/quantanics2018/Python/blob/main/Grabbing_a_image.py

In this example, the script sends an HTTP request to the specified URL using the requests library. If the response status code is 200 (OK), it means the request was successful. The HTML content of the webpage is then parsed using BeautifulSoup. The `soup.find_all("img")` line finds all image tags within the HTML content, and the `img_tag.get("src")` extracts the `src` attribute, which contains the URL of the image. Remember to replace the `url` variable with the actual URL of the webpage you want to scrape. Keep in mind that the structure of web pages can vary, and the example assumes that the image tags are present and follow standard HTML structure. Additionally, the URLs obtained might be relative URLs, so you might need to construct absolute URLs if necessary.

5.17) Book Examples

Sure, here are a few well-known and highly recommended books for learning Python programming at various levels, from beginners to more advanced learners:

1. Python Crash Course by Eric Matthes:

This book is great for beginners. It covers Python fundamentals and includes practical projects to reinforce your learning.

2. Automate the Boring Stuff with Python" by Al Sweigart:

This book focuses on using Python to automate everyday tasks. It's suitable for beginners and covers practical applications.

3. Python for Data Analysis" by Wes McKinney:

If you're interested in data analysis and manipulation with Python, this book is a fantastic resource. It covers libraries like pandas and numpy.

4. Fluent Python" by Luciano Ramalho:

This book is for those who want to deepen their understanding of Python's advanced features and idioms. It covers topics like metaclasses, decorators, and generators.

5. Learning Python" by Mark Lutz:

Known for its comprehensive coverage, this book is suitable for beginners and provides a solid foundation for further Python learning.

6. Python Cookbook" by David Beazley and Brian K. Jones:

This book offers a collection of solutions to common programming problems in Python. It's aimed at intermediate to advanced Python developers.

7. Effective Python" by Brett Slatkin:

This book provides 59 specific ways to improve your Python code. It's ideal for those who want to write more idiomatic and efficient Python.

8. Dive into Python 3" by Mark Pilgrim:

A free online book that focuses on Python 3. It's concise and comprehensive, making it a good resource for both beginners and experienced programmers.

9. Python Testing with pytest" by Brian Okken:

If you're interested in software testing with Python, this book dives into using the pytest framework to write effective tests.

These are just a few examples, and there are many more Python books available that cater to various interests and skill levels. Remember that learning Python involves both reading and practical coding, so make sure to apply what you learn through hands-on coding exercises and projects.

5.18) Introduction to Images with Python

Working with images in Python involves using libraries that allow you to manipulate, process, and analyze images. Two popular libraries for image manipulation are PIL (Python Imaging Library) and its successor, Pillow. Here's an introduction to working with images using the Pillow library:

1. Installing Pillow

You can install the Pillow library using the following command:

```
pip install Pillow==10.0.0
```

2. Importing the Library

```
from PIL import image
```

Fig 5.18.1 Importing the Library

3. Opening and Displaying Images

You can open an image using the `Image.open()` method and display it using the `.show()` method:

```
image = image.open("image.jpg")  
image.show
```

Fig 5.18.2 Opening and Displaying Images

4. Image Properties

You can access various properties of the image, such as its size and format:

```
width,height = image.size
format = image.format
print("image size:",width,"x",height)
print("image format:",format)
```

Fig 5.18.3 Image Properties

5. Manipulating Images

The Pillow library provides various methods for manipulating images, including resizing, cropping, rotating, and filtering:

```
#Resize the image
resized_image = image.resize((width//2,height//2))

#Crop a region of the image
Cropped_image = image.crop((100,100,300,300))

#Rotated the image
rotated_image = image.rotate(90)

#Apply a filter(e.g.,blur)
from PIL import ImageFilter
blurred_image = image.filter(image.Filter.BLUR)
```

Fig 5.18.4 Manipulating Image

6. Saving Images

You can save modified images using the `.save()` method:

```
resized_image.save("resized_image.jpg")
cropped_image.save("cropped_image.jpg")
rotated_image.save("rotated_image.jpg")
blurred_image.save("blurred_image.jpg")
```

Fig 5.18.5 Saving Images

These are just a few basic operations you can perform with the Pillow library. It offers a wide range of functionality for working with images, including more advanced features like image

enhancement, color manipulation, and text overlay. Make sure to refer to the official Pillow documentation for detailed information and examples:

[Pillow Documentation](<https://pillow.readthedocs.io/en/stable/index.html>)

5.19) Working with CSV files in python

Working with CSV (Comma-Separated Values) files in Python is a common task for reading and writing tabular data. Python provides the csv module, which makes it easy to handle CSV files. Here's an overview of how to work with CSV files in Python:

1. Reading CSV Files

To read data from a CSV file in Python, you can utilize the csv.reader object provided by the csv module. Here's an example:

```
import csv

# Open the CSV file for reading
with open(r'C:\Users\nares\Downloads\gas_try (1).csv', 'r') as file:
    csv_reader = csv.reader(file)

    # Iterate over each row in the CSV file
    for row in csv_reader:
        print(row)
```

Fig 5.19.1 Reading CSV Files Python Code

By importing the csv module with 'import csv', you gain access to functions tailored for handling CSV files. Within a 'with' statement, 'open('output.csv', 'r')' opens the file 'output.csv' in read mode ('r'), facilitating data reading operations. The file object is assigned to the variable 'file' for ease of reference. Subsequently, 'csv_reader = csv.reader(file)' initializes a CSV reader object 'csv_reader' associated with the opened file, enabling sequential access to rows of data. Using a 'for' loop like 'for row in csv_reader:', each iteration retrieves and processes a single row from the CSV file. The 'print(row)' statement within the loop outputs each row of data as a list, where each element corresponds to a field (or column) in the CSV file. Upon execution, this script reads 'output.csv' and displays each row of data on the console Fig 5.19.2.

```
['Name', 'Age', 'Country']  
['Alice', '25', 'USA']  
['Bob', '30', 'Canada']  
['Carol', '22', 'Australia']
```

Fig 5.19.2 Reading CSV Files Python Output

For further information and additional functionalities, refer to the csv module documentation.

https://github.com/quantanics2018/Python/blob/main/Reading_CSV_file.py

2. Writing CSV Files:

To write data to a CSV file using Python's csv module, you start by importing the module with `import csv`. The data you want to write is structured as a list of lists (`data`), where each inner list represents a row of data with elements corresponding to columns. Within a `with` statement, `open('output.csv', 'w', newline='')` opens or creates a file named 'output.csv' in write mode ('w'), ensuring proper handling of line endings for different platforms. The file object `file` is created for reference. Subsequently, `csv_writer = csv.writer(file)` initializes a CSV writer object linked to `file`, which facilitates writing data to the CSV file. Iterating through each `row` in `data` with `for row in data:`, `csv_writer.writerow(row)` writes each row of data to the CSV file. This method formats each row with elements separated by commas and appends a newline character, ensuring a correctly structured CSV file. Upon execution, this script generates 'output.csv' in the current directory, containing the formatted data from `data`.

https://github.com/quantanics2018/Python/blob/main/Writing_CSV_file.py

In Fig 5.19.3, the Python code demonstrates how to write the data list to a CSV file named 'output.csv'. Each inner list in `data` represents a row in the CSV file, and `csv_writer.writerow(row)` is used within a loop to write each row to the file.

```
import csv

data = [
    ['Name', 'Age', 'Country'],
    ['Alice', 25, 'USA'],
    ['Bob', 30, 'Canada'],
    ['Carol', 22, 'Australia']
]

# Open the CSV file for writing
with open('output.csv', 'w', newline='') as file:
    csv_writer = csv.writer(file)

    # Write each row of data
    for row in data:
        csv_writer.writerow(row)
```

Fig 5.19.3 Writing CSV Files Python Code

Fig 5.19.4 shows the actual content written to 'output.csv' after executing the code. Each row from the data list is formatted with elements separated by commas, adhering to the CSV format, and each line ends with a newline character. This output illustrates how the data is structured in the resulting CSV file.

	A	B	C
1	Name	Age	Country
2	Alice	25	USA
3	Bob	30	Canada
4	Carol	22	Australia
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			
16			
17			

Fig 5.19.4 Writing CSV Files Python Output

3. Reading CSV Files with DictReader:

The `csv.DictReader` class allows you to read CSV files as dictionaries, where each row is represented by a dictionary with column names as keys.

```
import csv

# Open the CSV file for reading
with open(r"C:\Users\nares\Downloads\gas_try (1).csv", 'r') as file:
    csv_dict_reader = csv.DictReader(file)

    # Iterate over each row in the CSV file
    for row in csv_dict_reader:
        print(row['Name'], row['Age'], row['Country'])
```

Fig 5.19.5 Reading CSV Files with DictReader Python Code

import csv: This line imports the `csv` module, which provides functionality for working with CSV files.

Inside the with statement

The line `open('output.csv', 'r')` opens a file named 'output.csv' in read mode ('r'), preparing the file for reading. The file object is then assigned to the variable `file` for easier reference. Following this, `csv.DictReader(file)` creates a `DictReader` object associated with the opened file, which reads the data from the CSV file and interprets the first row as column headers (field names).

The loop `for row in csv_dict_reader:` iterates over each row in the CSV file, with the `csv_dict_reader` object returning each row as a dictionary where the keys are the column headers and the values are the corresponding values in each row. Inside the loop, `print(row['Name'], row['Age'], row['Country'])` prints specific columns ('Name', 'Age', 'Country') from each row by accessing the values using the column names as keys. After

running this code, it will read the 'output.csv' file, interpret the first row as column headers, and print the specified columns for each row.

More details and objects can be found here :

https://github.com/quantanics2018/Python/blob/main/Reading_CSV_file.py

```
Alice 25 USA
Bob 30 Canada
Carol 22 Australia
```

Fig 5.19.6 Reading CSV Files with DictReader Output

4. Writing CSV Files with DictWriter:

The csv.DictWriter class allows you to write dictionaries as rows to a CSV file.

```
import csv

fieldnames = ['Name', 'Age', 'Country']
data = [
    {'Name': 'Alice', 'Age': 25, 'Country': 'USA'},
    {'Name': 'Bob', 'Age': 30, 'Country': 'Canada'},
    {'Name': 'Carol', 'Age': 22, 'Country': 'Australia'}
]

# Open the CSV file for writing
with open(r"C:\Users\nares\Downloads\gas_try (1).csv", 'w', newline='') as file:
    csv_dict_writer = csv.DictWriter(file, fieldnames=fieldnames)

    # Write the header
    csv_dict_writer.writeheader()

    # Write the data rows
    for row in data:
        csv_dict_writer.writerow(row)
```

Fig 5.19.7 Writing CSV Files with DictReader Python Code

```
>>> |=====
```

Fig 5.19.8 Writing CSV Files with DictReader Output

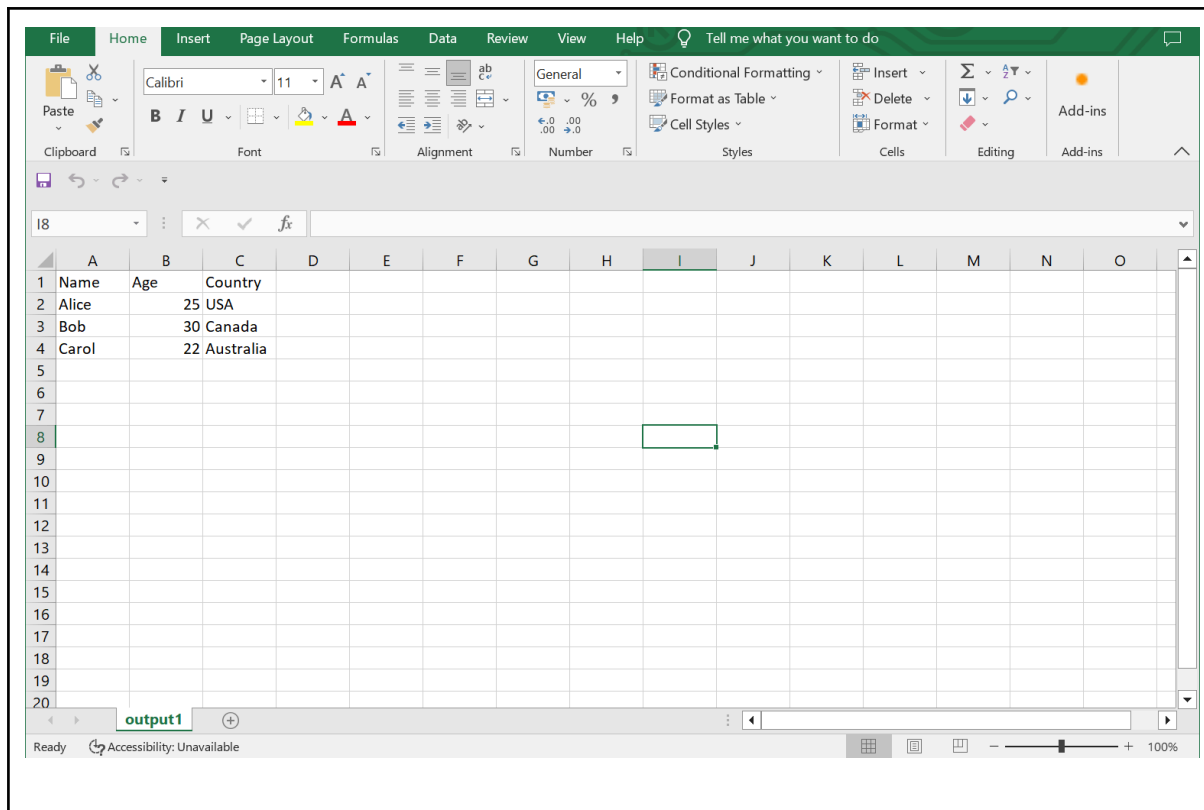


Fig 5.19.9 Writing CSV Files with DictReader Output

More details and objects can be found here :

https://github.com/quantanics2018/Python/blob/main/Reading_CSV_Files_with_DictReader.py

5.20) Working with PDF Files in Python

Working with PDF files in Python involves using libraries that allow you to manipulate, extract information, and generate PDF documents. The PyPDF2 library is a popular choice for working with PDF files in Python. Here's an introduction to using PyPDF2:

1. Installing PyPDF2: You can install the PyPDF2 library using the following command:

```
pip install PyPDF2
```

2. Reading PDF Files:

```
import pdfplumber

# Specify the PDF file path
pdf_path = r"C:\Users\nares\Downloads\Untitled document (2).pdf"

# Open the PDF file
with pdfplumber.open(pdf_path) as pdf:
    # Get the number of pages
    num_pages = len(pdf.pages)
    print("Number of pages:", num_pages)

    # Extract text from the first page
    first_page = pdf.pages[0]
    page_text = first_page.extract_text()
    print("Text from first page:", page_text)
```

Fig 5.20.1 Reading PDF Files Python Code

```
Number of pages: 1
Text from first page: A computer language, also known as a programming language,
is a formal language
comprised of a set of instructions that produce various kinds of output. These
languages are used by developers to communicate with computers and create
software applications, scripts, and other sets of instructions for machines to f
ollow.
Programming languages provide the necessary tools to write algorithms, manage
data structures, and control hardware. There are many types of programming
languages, each with its unique syntax, semantics, and use cases. Some, like
Python, are known for their simplicity and readability, making them ideal for
beginners, while others, like C++, offer more control over hardware and are used
for
performance-critical applications. Languages such as JavaScript are essential fo
r
web development, allowing for dynamic and interactive user experiences. Over tim
e,
programming languages have evolved to support more abstract and powerful
constructs, enabling developers to build complex systems more efficiently. The
choice of a programming language often depends on the specific requirements of a
project, including performance, ease of use, and the problem domain it aims to
address.
```

Fig 5.20.2m Reading PDF Files Output

More details and objects can be found here
:https://github.com/quantanics2018/Python/blob/main/Reading_pdf_files.py

3. Extracting Text from Multiple Pages

You can extract text from all pages of a PDF using a loop. More details and objects can be found here :

https://github.com/quantanics2018/Python/blob/main/Extracting_text_from_multiple_pages.py

```
import pdfplumber

# Specify the PDF file path
pdf_path = r"C:\Users\nares\Downloads\Untitled document (2).pdf"

# Open the PDF file using pdfplumber
with pdfplumber.open(pdf_path) as pdf:
    # Iterate over each page in the PDF file
    for page_num in range(len(pdf.pages)):
        page = pdf.pages[page_num]
        page_text = page.extract_text()
        print(f"Text from page {page_num + 1}:", page_text)
```

Fig 5.20.3 Extracting Text from Multiple Pages Python Code

Text from page 1: A computer language, also known as a programming language, is a formal language comprised of a set of instructions that produce various kinds of output. These languages are used by developers to communicate with computers and create software applications, scripts, and other sets of instructions for machines to follow. Programming languages provide the necessary tools to write algorithms, manage data structures, and control hardware. There are many types of programming languages, each with its unique syntax, semantics, and use cases. Some, like Python, are known for their simplicity and readability, making them ideal for beginners, while others, like C++, offer more control over hardware and are used for performance-critical applications. Languages such as JavaScript are essential for web development, allowing for dynamic and interactive user experiences. Over time, programming languages have evolved to support more abstract and powerful constructs, enabling developers to build complex systems more efficiently. The choice of a programming language often depends on the specific requirements of a project, including performance, ease of use, and the problem domain it aims to address.

Fig 5.20.4 Extracting Text from Multiple Pages Output

4. Merging PDFs

You can use the PdfReader class to read multiple PDF files and the PdfWriter class to merge them.

```
import PyPDF2

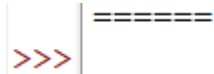
pdf_writer = PyPDF2.PdfWriter()

with open('doo.pdf', 'rb') as pdf1, open('re.pdf', 'rb') as pdf2:
    pdf_reader1 = PyPDF2.PdfReader(pdf1)
    pdf_reader2 = PyPDF2.PdfReader(pdf2)

    for page in pdf_reader1.pages:
        pdf_writer.add_page(page)
    for page in pdf_reader2.pages:
        pdf_writer.add_page(page)

with open('merged.pdf', 'wb') as output_file:
    pdf_writer.write(output_file)
```

Fig 5.20.5 Merging PDFs Python Code



```
>>> |=====
```

Fig 5.20.6 Merging PDFs Output

Doo.pdf

Merger/pdf

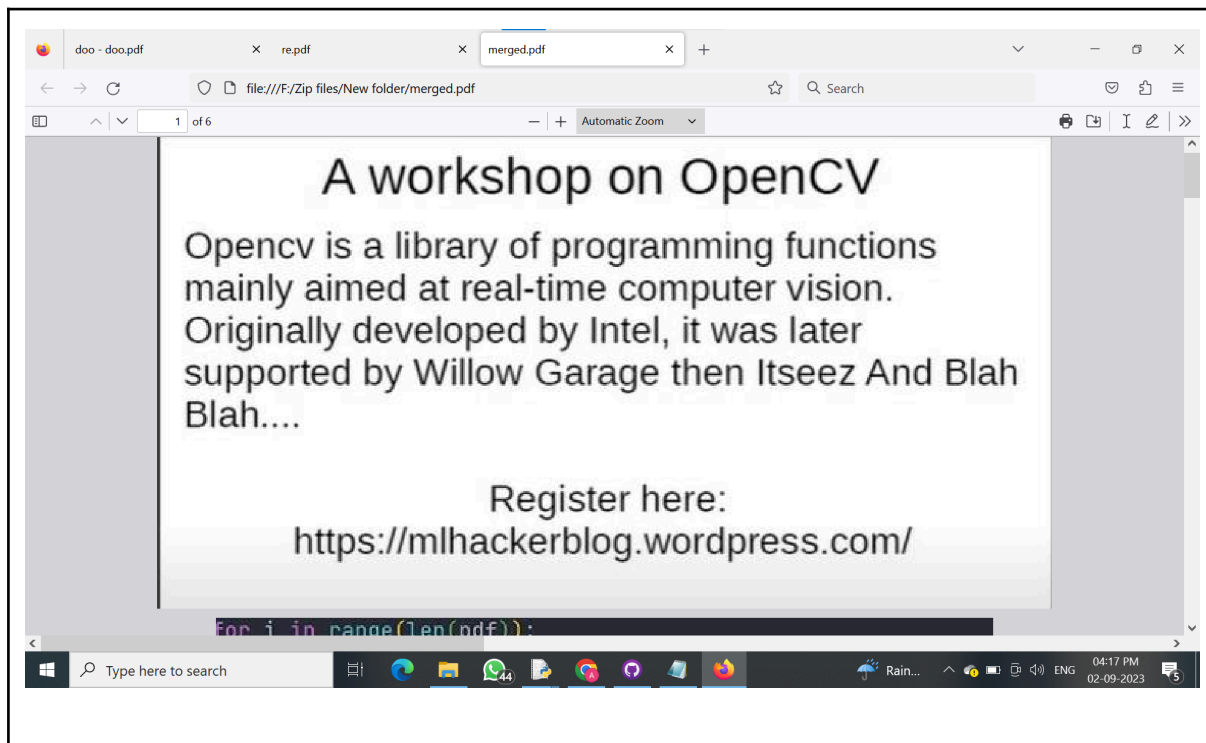


Fig 5.20.9 Merged PDF

More details and objects can be found here :
https://github.com/quantanics2018/Python/blob/main/Merging_two_pdf.py

These are just some basic operations you can perform using the PyPDF2 library. Keep in mind that PyPDF2 may not handle all PDF features, such as complex layouts and interactive forms. For more advanced PDF manipulation, you might consider using other libraries like reportlab or fpdf.

5.21) Sending Emails with Python

In this program first we need to turn on two step verification to unlock the app password. Inorder To turn on two step verification go to Google>>Google my Account>>Security>>Two Step Verification.

Search google my account in google.

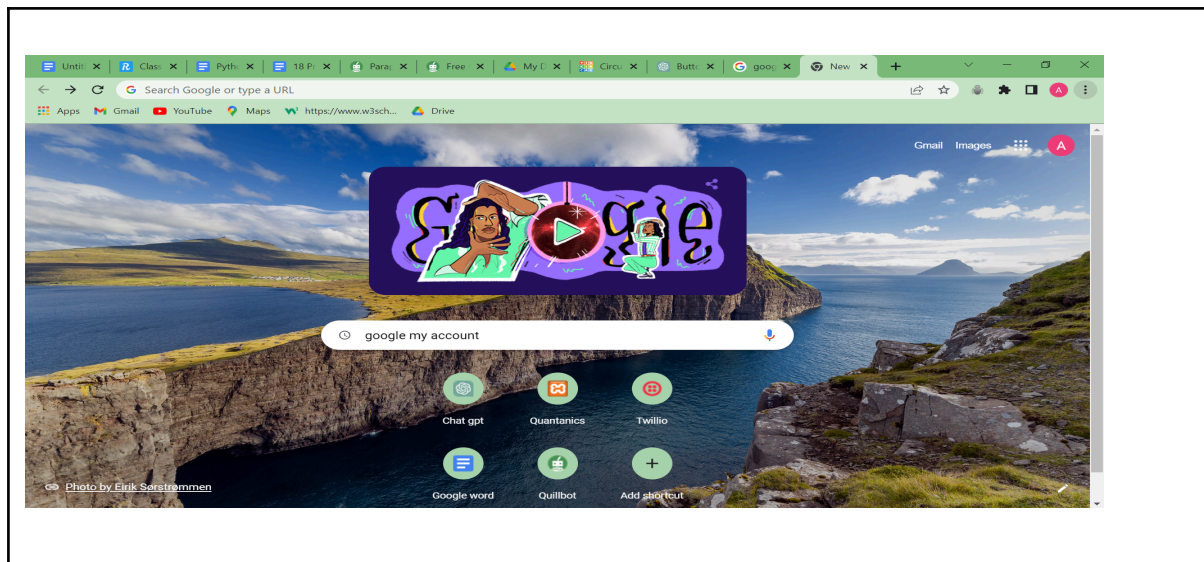


Fig 5.21.1 Google My Account Searching

Then click the first link .

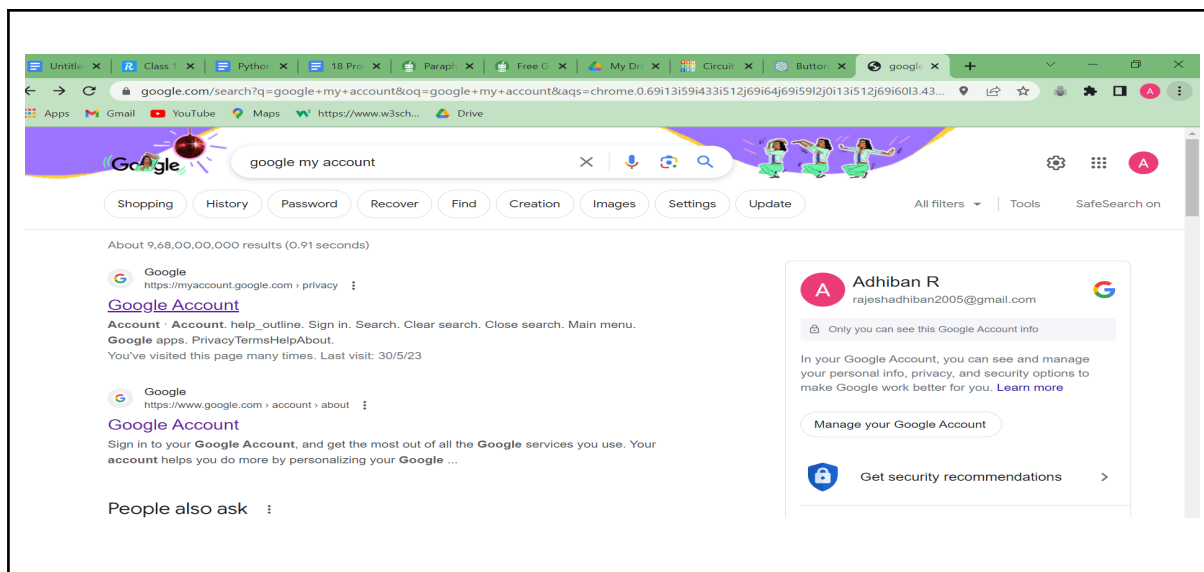


Fig 5.21.2 Google My Account Website

Then your google account will open like this , in that click Security.

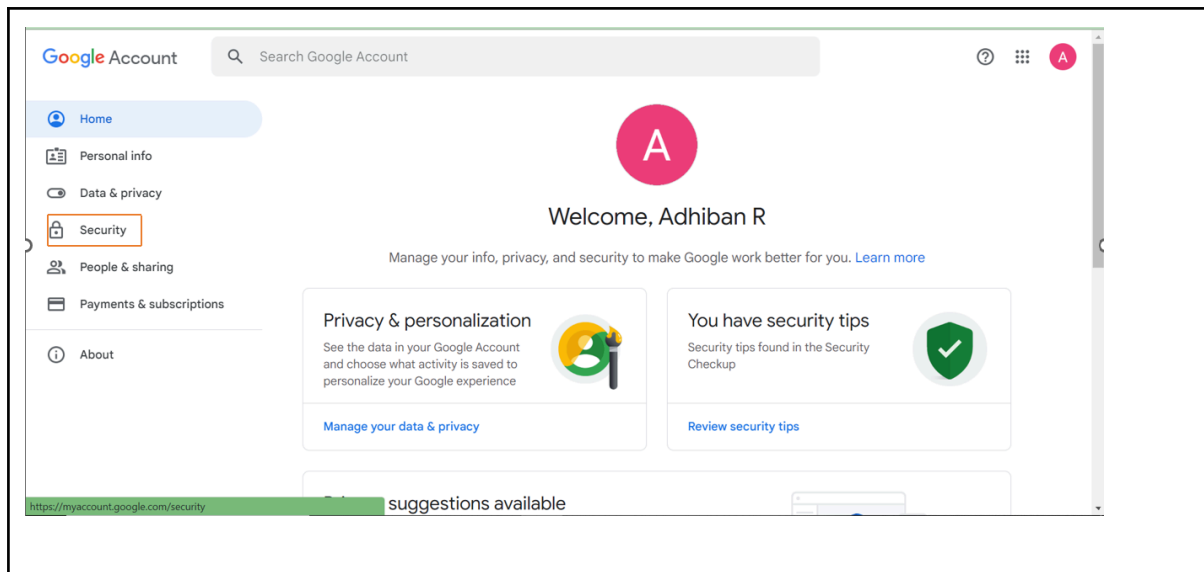


Fig 5.21.3 Google My Account Security

Then you will see the security tab and scroll down and then you will see the 2-Step Verification and turn it on.

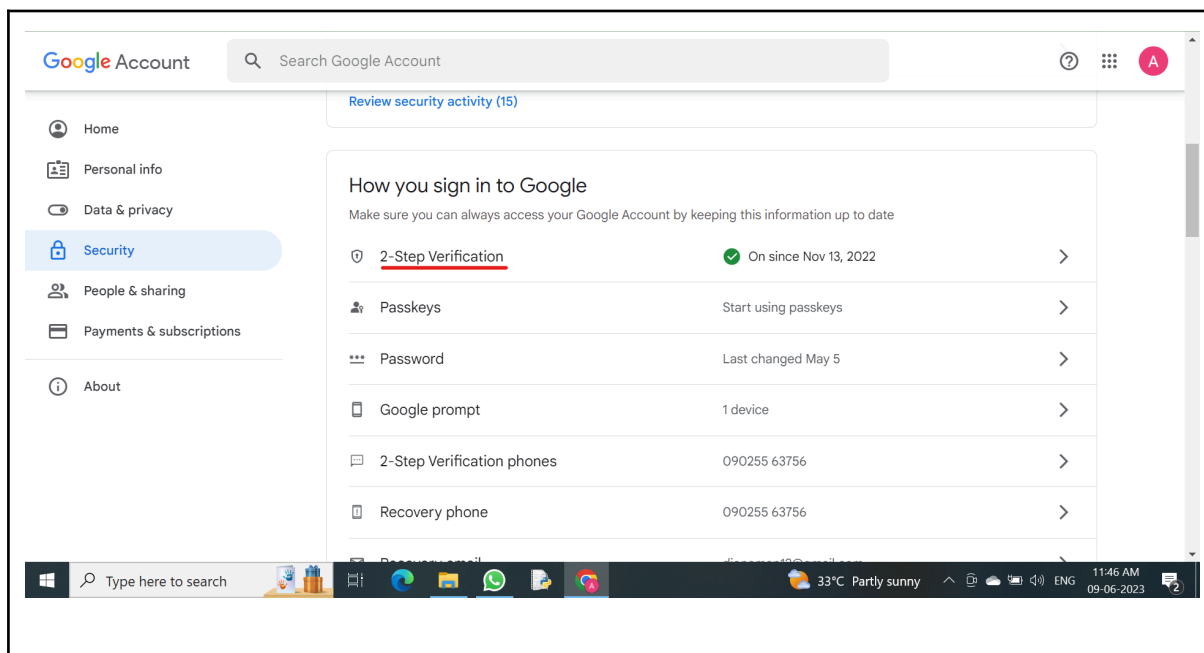


Fig 5.21.4 Google My Account Two Step Verification

After turning on 2-Step Verification you will see a screen like this and scroll down.

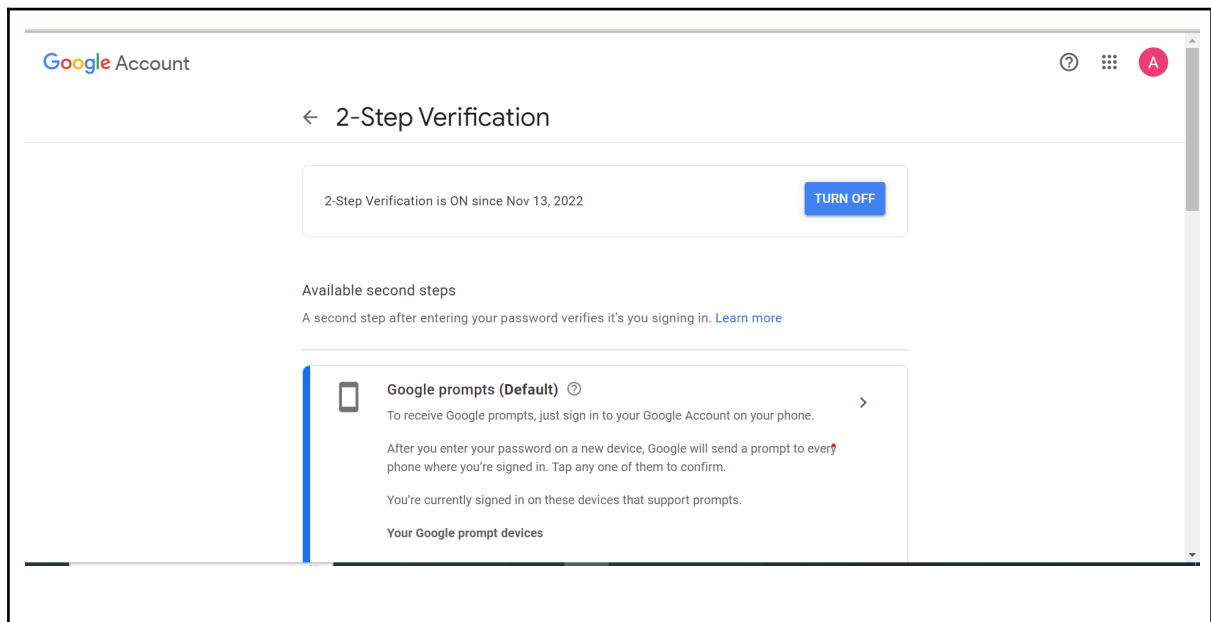


Fig 5.21.5 Turn On

Then it will show app password you have already been created

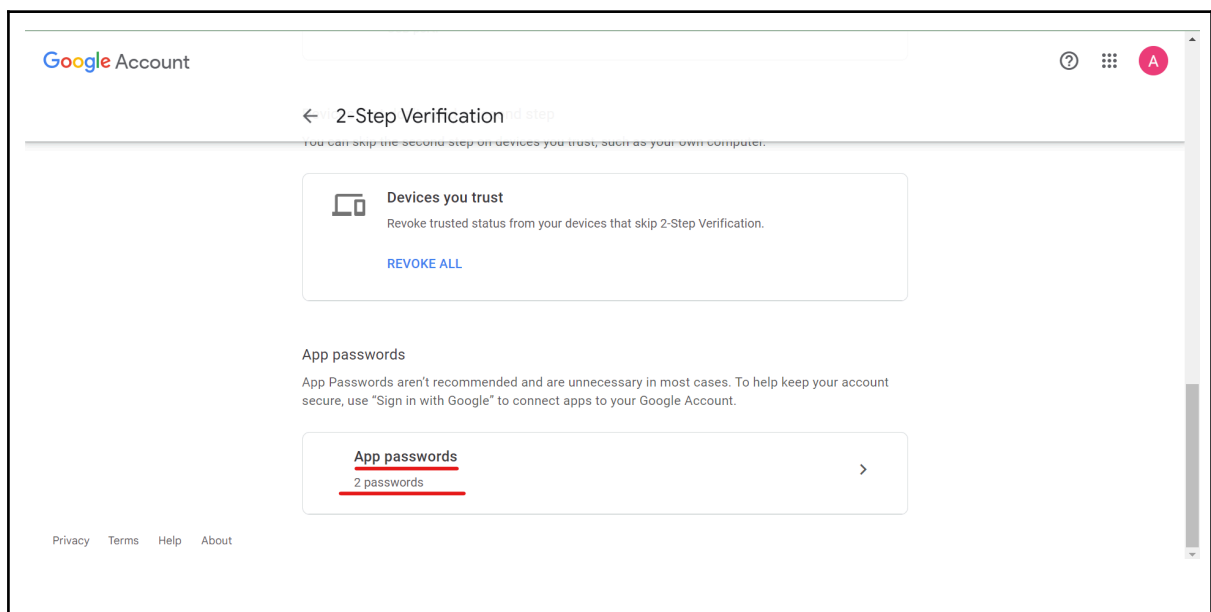


Fig 5.21.6 App Password Creation

If you click the AppPassword, it will show a screen where you can Generate app password and see Already generated App Password.

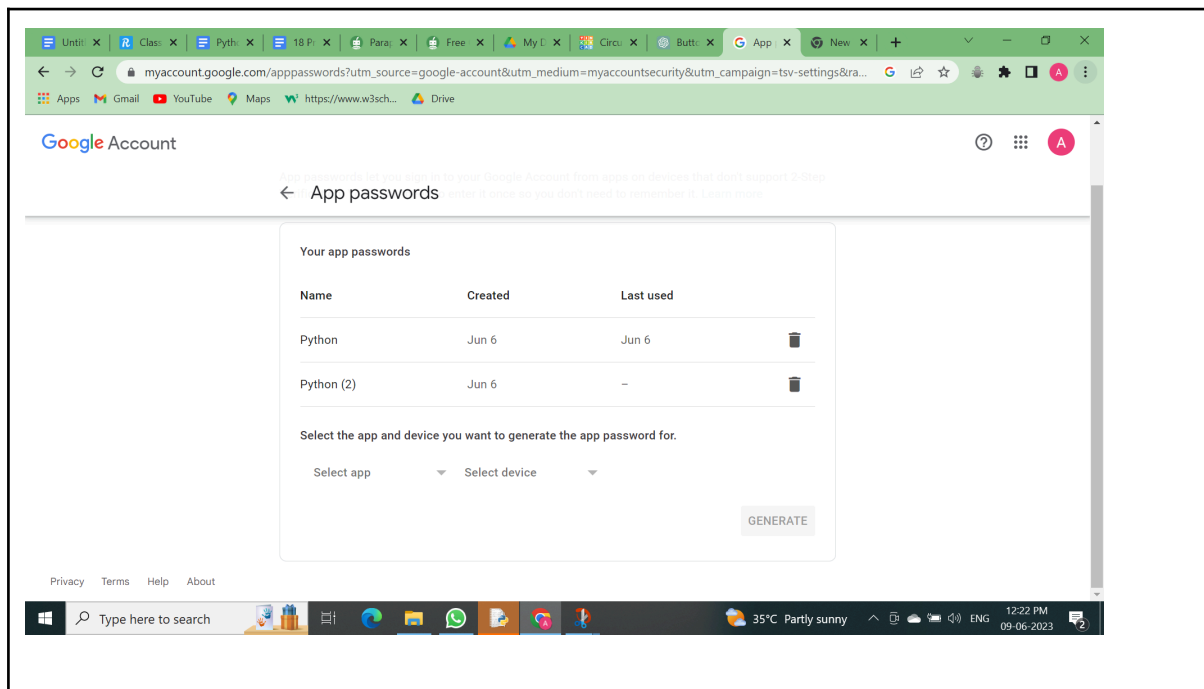


Fig 5.21.7 App Passwords Already Created

Then click Select app and click Other(Custom name)

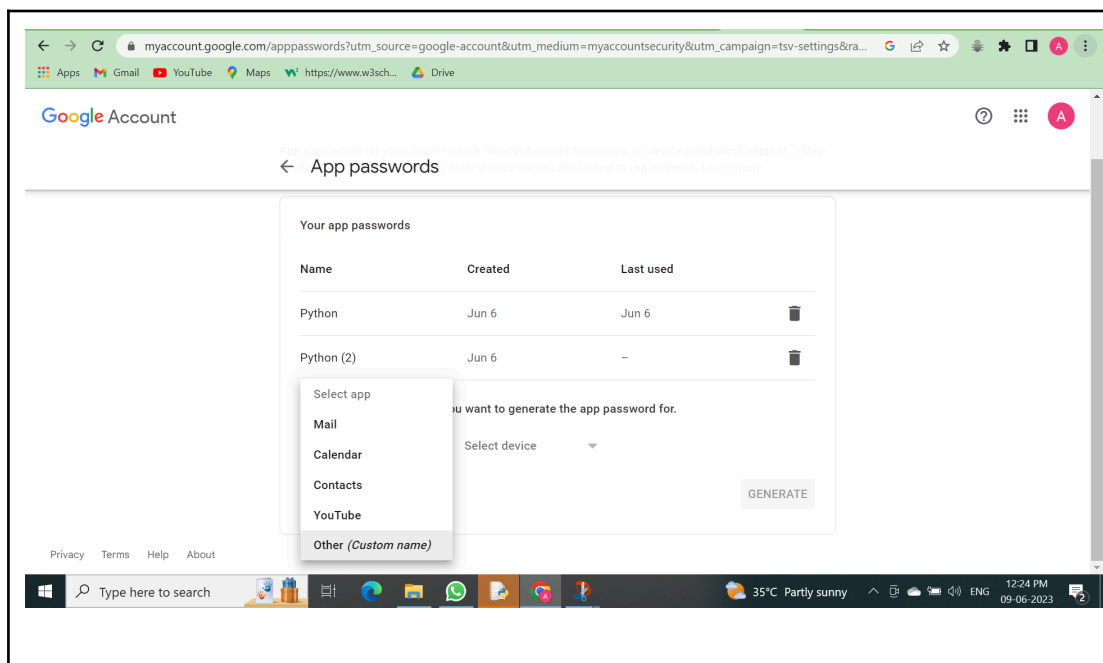


Fig 5.21.8 Typing the Name Python For Reference

Then type Python in it .

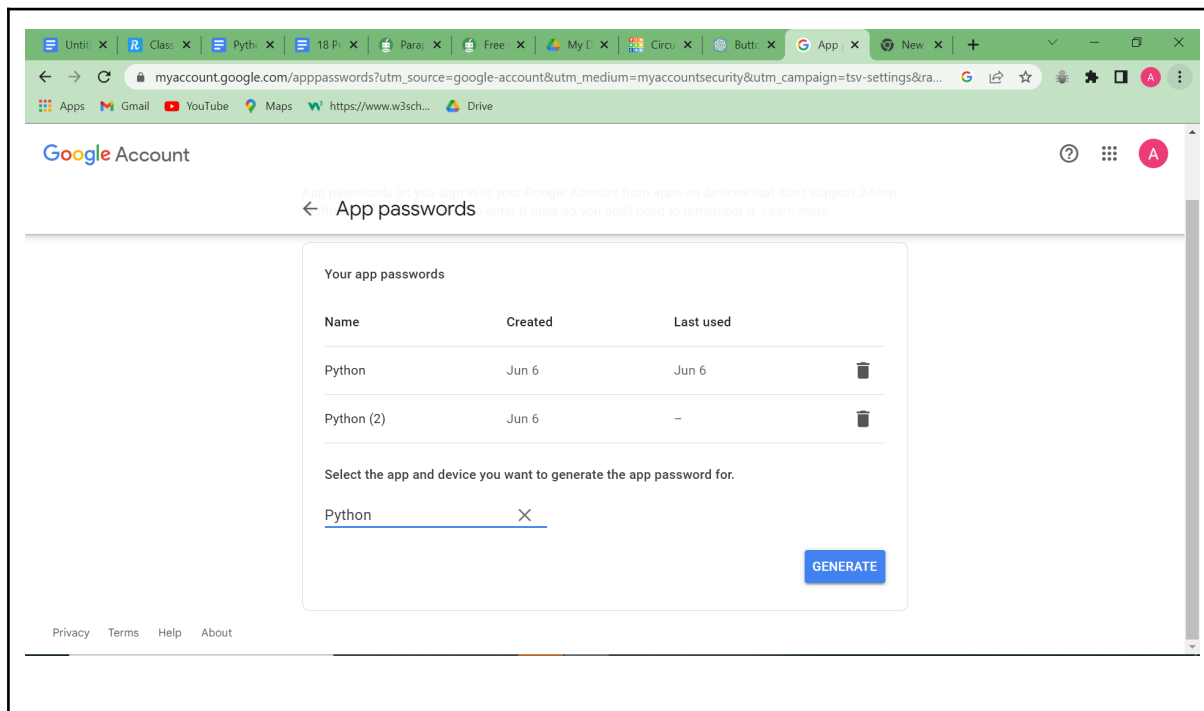


Fig 5.21.9 Typing the Name Python For Reference

Then it will generate an App password for Python.

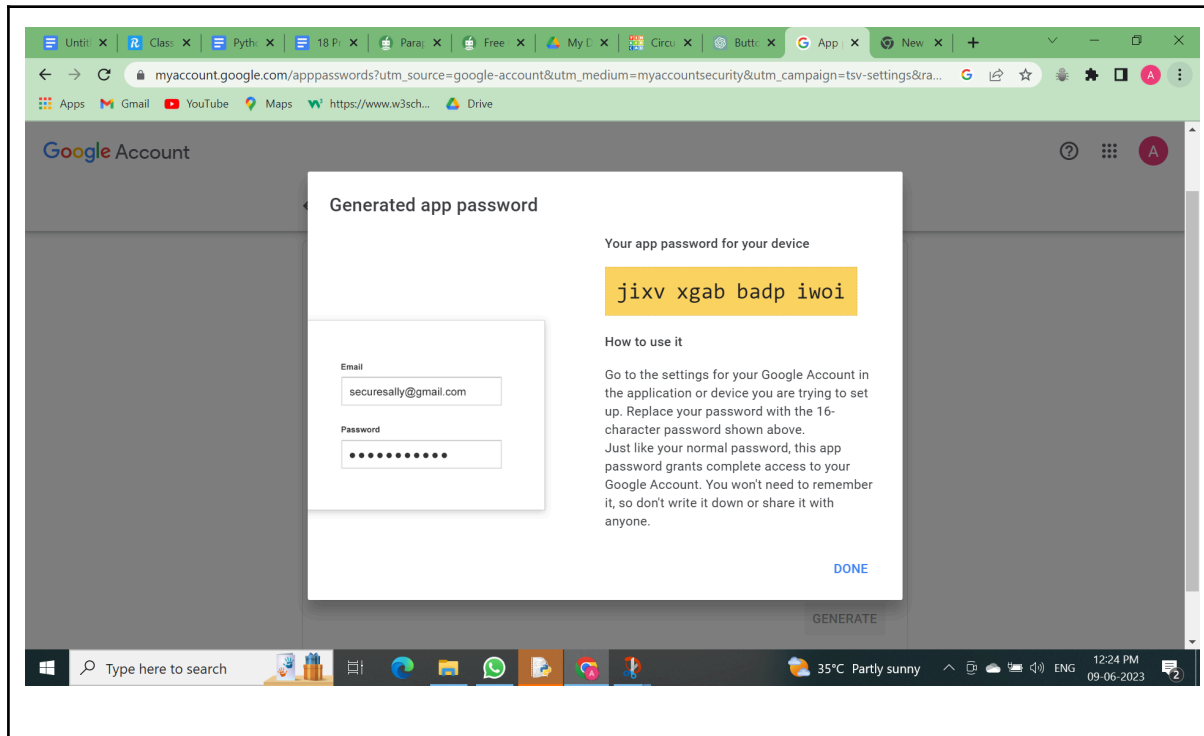


Fig 5.21.10 App Password Created

You must enter your own mail address, the app password you created, and the message you want to send before sending it to the recipient using the Python smtplib package.

Smtp refers to Simple Mail Transfer Protocol which is used to send and receive email.

SSL refers to Secure Socket Layer which is used to create safe connection between client and server.

Since I erased the app password, the code below won't work. Instead, try setting your own app password in your Google account.

Code explanation

Import required modules: The ssl module is imported to create a secure SSL context, the smtplib module is used to send emails, and EmailMessage from email.message is used to create email messages.

Define email sender, receiver, subject, and body: You specify the email sender's and receiver's addresses, the subject of the email, and the body of the email.

Create an EmailMessage instance: You create an EmailMessage object em and set its sender, receiver, subject, and content (body).

Create an SSL context: You create a secure SSL context using ssl.create_default_context().

Connect to the SMTP server: You use smtplib.SMTP_SSL to connect to Gmail's SMTP server on port 465, which is the SSL port.

Log in and send email: You log in to the sender's email account using the provided email and password, and then use the sendmail method to send the email using the as_string() method of the EmailMessage object.

Print success message: If the email is sent successfully, you print a success message.

More details and object can be found here

https://github.com/quantanics2018/Python/blob/main/Sending_emails_using_python.py

```

import ssl
import smtplib
from email.message import EmailMessage

# Your email sender details
email_sender = "rajeshadhiban2005@gmail.com"
email_password = "zswwbvqddcmewtrz" # This is an example password, use your actual password
email_receiver = "dianoman12@gmail.com"
subject = "Testing from Adhiban"
body = """
This is my Python program for Sending email checking
"""

# Create an EmailMessage instance
em = EmailMessage()
em['From'] = email_sender
em['To'] = email_receiver
em['Subject'] = subject
em.set_content(body)

# Create an SSL context for secure connection
context = ssl.create_default_context()

# Connect to the Gmail SMTP server using SSL
with smtplib.SMTP_SSL('smtp.gmail.com', 465, context=context) as smtp:
    # Log in to the sender's email account
    smtp.login(email_sender, email_password)

    # Send the email
    smtp.sendmail(email_sender, email_receiver, em.as_string())

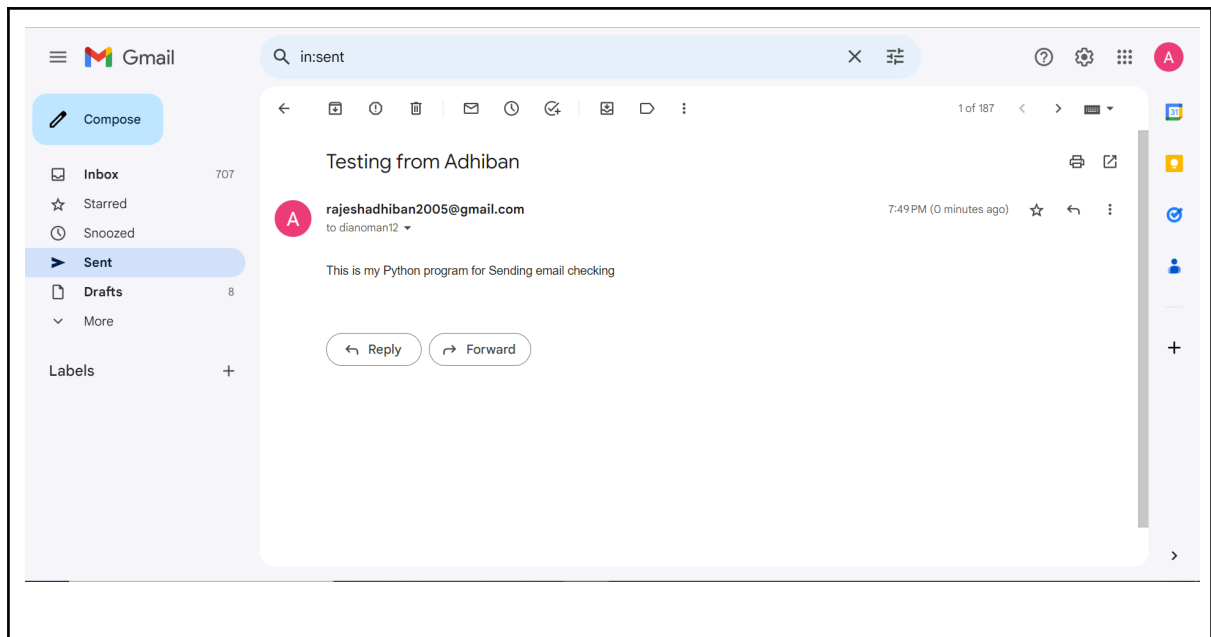
# Print a success message
print("Mail sent successfully")

```

Fig 5.21.11 Python code

```
Mail sent succesfully
```

Fig 5.21.12 output



More details and object can be found here

https://github.com/quantanics2018/Python/blob/main/Sending_emails_using_python.py

5.22)Receiving Gmail with Python

```

import imaplib
import email

user = "rajeshadhiban2005@gmail.com"
password = "zswwbvqdcemewtrz" #App password
imap_url = 'imap.gmail.com'
# Function to get email content part i.e its body part
def get_body(msg):
    if msg.is_multipart():
        return get_body(msg.get_payload(0))
    else:
        return msg.get_payload(None, True)
# Function to search for a key value pair
def search(key, value, con):
    result, data = con.search(None, key, '{}'.format(value))
    return data
# this is done to make SSL connection with GMAIL
con = imaplib.IMAP4_SSL(imap_url)
con.login(user, password)
# calling function to check for email under this label
con.select('Inbox')
# Search for the most recent email from the sender
sender_email = "dianoman12@gmail.com"
messages = search('FROM', sender_email, con)[0].split()
if messages:
    latest_message = messages[-1] # Get the ID of the latest email
    # Fetch the latest email
    typ, msg_data = con.fetch(latest_message, '(RFC822)')
    msg = email.message_from_bytes(msg_data[0][1])
    # Extracting required information
    sender = msg["From"]
    recipient = msg["To"]
    date = msg["Date"]
    subject = msg["Subject"]
    print("Sender:", sender)
    print("Recipient:", recipient)
    print("Date:", date)
    print("Subject:", subject)
    body = get_body(msg)
    print("Body:", body)
else:
    print("No recent email found from the sender.")

```

Fig 5.22.1 Gmail receiving code

```

Sender: Diano man<dianoman12@gmail.com>
Recipient: rajeshadhibhan2005@gmail.com
Date: thu, 31 Aug 2023 16:31:39 +0530
Subject: Python
Body: <email body content>

```

Fig 5.22.2 Output

Certainly! The code you've provided is a Python script that connects to a Gmail account using IMAP to retrieve the most recent email from a specific sender. It then extracts and prints information about the email such as sender, recipient, date, subject, and body. More details and objects can be found here

https://github.com/quantanics2018/Python/blob/main/Receiving_gmails_using_python.py

Here's an explanation of the code and the packages used:

1. import imaplib and import email: These are Python standard libraries used to interact with IMAP email servers and parse email content, respectively.

2. user and password: These variables store your Gmail credentials (username and password). Note that using your password directly in the code is not recommended for security reasons. Instead, consider using an App Password if you have two-step verification enabled.

3. imap_url: This variable contains the IMAP server URL for Gmail.

4. get_body(msg): This is a recursive function that extracts the content of the email body. If the email is multipart, it recursively calls itself to extract the body from the first part. If the email is not multipart, it directly returns the payload content.

5. search(key, value, con): This function searches for emails based on a key-value pair. In this code, it's used to search for emails from a specific sender.

6. con: This variable holds the IMAP connection object created using `imaplib.IMAP4_SSL(imap_url)`. It establishes a secure connection to the Gmail IMAP server.

7. Logging in: The script logs in to the Gmail account using the provided credentials.

8. `con.select('Inbox')`: It selects the 'Inbox' mailbox for further operations.

9. Searching for the most recent email: The code searches for emails from the specified sender and retrieves the message IDs of the matching emails.

10. Processing the most recent email: If there are matching emails, the script fetches the content of the most recent email, then extracts and prints the sender's email address, recipient, date, subject, and body.

11. Printing the results: The extracted information is printed to the console.

5.23) Creating a BMI Calculator

Creating BMI Calculator Web APP Using Python And PyWebIO

A Body Mass Index (BMI) calculator is a tool used to determine whether an individual has a healthy body weight for a given height. It is commonly used for the following purposes:

- Assessing Weight Status:** By calculating BMI, individuals can determine if they are underweight, normal weight, overweight, or obese. This can help in identifying potential health risks associated with their weight category.
- Health Risk Screening:** BMI is used as a screening tool to identify individuals who may be at risk for certain health conditions such as heart disease, diabetes, and hypertension. High or low BMI values can indicate an increased risk of these conditions.
- Monitoring Weight Changes:** Individuals can use BMI calculators to track their weight changes over time, particularly if they are following a weight loss or weight gain program. This helps in assessing the effectiveness of their efforts and making necessary adjustments.
- Public Health and Research:** Public health officials and researchers use BMI data to monitor the health status of populations, identify trends, and develop interventions to address obesity and related health issues.
- Nutritional Guidance:** Dietitians and healthcare providers use BMI to provide personalized dietary and lifestyle recommendations to help individuals achieve and maintain a healthy weight.

CODE:

```
# -*- coding: utf-8 -*-
"""
Created on Sun Apr  4 12:16:54 2021

@author: win10
"""
### Create a BMI Calculation Application

from pywebio.input import *
from pywebio.output import *

def bmicalculator():
    height=input("Please enter the height in cm",type=FLOAT)
    weight=input("Please enter the weight in Kg",type=FLOAT)

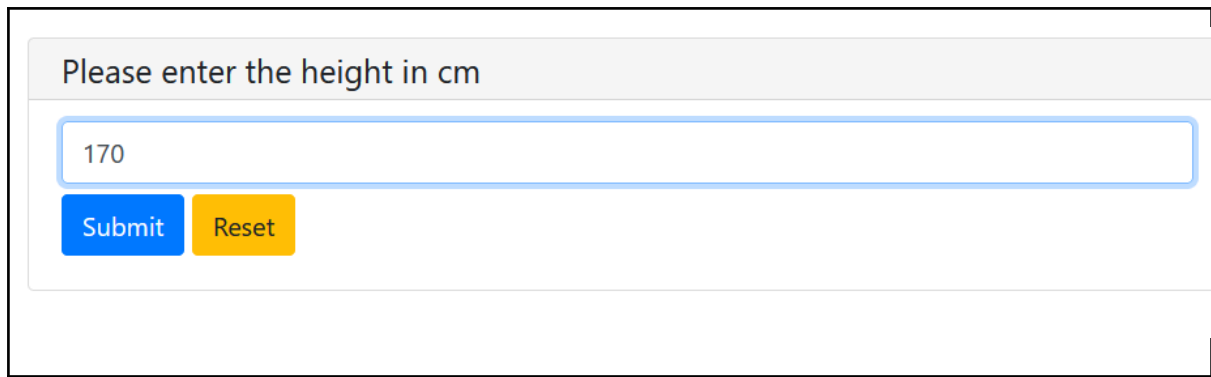
    bmi=weight/(height/100)**2

    bmi_output=[(16, 'Severely underweight'), (18.5, 'Underweight'),
                (25, 'Normal'), (30, 'Overweight'),
                (35, 'Moderately obese'), (float('inf'), 'Severely obese')]

    for tuple1,tuple2 in bmi_output:
        if bmi<=tuple1:
            put_text('Your BMI is :%.1f and the person is :%s'%(bmi,tuple2))
            break

if __name__=='__main__':
    bmicalculator()
```

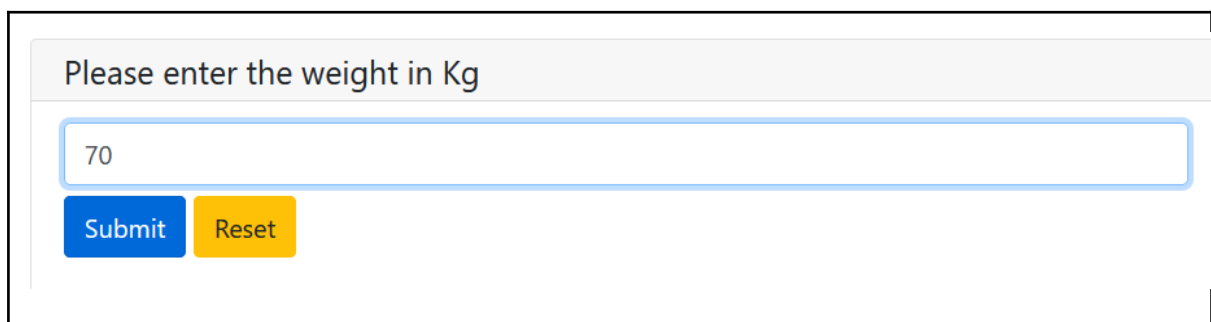
fig 5.23.1 BMI Calculator python code



Please enter the height in cm

Submit Reset

fig 5.23.2 output 1



Please enter the weight in Kg

Submit Reset

fig 5.23.3 output 2

Your BMI is :24.2 and the person is :Normal

fig 5.23.4 final output

More details and objects can be found here

https://github.com/quantanics2018/Python/tree/main/BMI_Calculator

5.24) Creating a Password generator

A password generator is a tool or program that creates strong, random passwords to enhance security. These passwords are typically complex, consisting of a mix of letters (both uppercase and lowercase), numbers, and special characters. This complexity makes the passwords difficult to guess or crack, thereby improving the security of accounts and systems. Using a password generator offers several advantages, including enhanced security through randomness, meeting complexity requirements of various systems, and ensuring

uniqueness, which reduces the risk of password reuse across different platforms. For instance, a Python-based password generator can be easily implemented to create secure passwords by randomly selecting characters from predefined sets, thereby guaranteeing robust protection for user accounts and sensitive data.

code:

```
from tkinter import *
import random
import string
import pyperclip

root = Tk()
root.geometry("400x400")
root.resizable(0,0)
root.title("DataFlair - PASSWORD GENERATOR")

pass_label = Label(root, text='PASSWORD LENGTH', font='arial 10 bold')
pass_label.pack()

pass_len = IntVar()
length = Spinbox(root, from_=8, to=32, textvariable=pass_len, width=15)
length.pack()

pass_str = StringVar()

def Generator():
    length = pass_len.get()
    if length < 4:
        length = 4
        pass_len.set(4)

    password = [
        random.choice(string.ascii_uppercase),
        random.choice(string.ascii_lowercase),
        random.choice(string.digits),
        random.choice(string.punctuation)
    ]

    if length > 4:
        password += random.choices(string.ascii_uppercase + string.ascii_lowercase + string.digits + string.punctuation, k=length - 4)

    random.shuffle(password)
    pass_str.set(''.join(password))

generate_button = Button(root, text="GENERATE PASSWORD", command=Generator)
generate_button.pack(pady=5)

password_entry = Entry(root, textvariable=pass_str)
password_entry.pack()
```

fig 5.25.1 password generator



fig 5.25.2 output

More details and objects can be found [here](#)

https://github.com/quantanics2018/Python/tree/main/Password_Generator

5.25) Hangman Game in Python

Going back to our school days, pen-paper games like Hangman were always a top choice for leisure time. Besides various chit games, Hangman involved guessing words based on determined guesses. Once all wrong guesses were exhausted, the player was "hanged"—not literally, but on paper.

This traditional way of playing Hangman has evolved with technology, allowing us to enjoy the game on our computers without needing another player. In this project, we will implement the Hangman game using Python. This doesn't require any specific modules beyond random and time.

Python loops and functions suffice to build this game. This project requires a good knowledge of Python, including defining functions and managing for/while loops. The functions we use contain arguments defined in a global scope, which can be further utilized in other functions to enhance game quality.

This approach also allows for different steps to be executed upon conditions defined by for and while loops. Here are the steps to build the Hangman game in Python. First, import the random and time modules. Next, define functions with specific global arguments. Then, implement loops to execute the program. Finally, pass the function in the program to run.

CODE:

More details and objects can be found here

https://github.com/quantanics2018/Python/tree/main/Hang_man

5.26) Creating a Calculator using Code

A calculator is a machine which allows people to do math operations more easily. For example, most calculators will add, subtract, multiply, and divide. Some also do square roots, and more complex calculators can help with calculus and draw function graphs. Calculators are found everywhere. A smartphone or other computer can also act as a calculator. Some calculators, like the abacus, will work without electricity. Others, like the electronic calculator, use batteries. Some calculators are simple, and can only add, subtract, multiply and divide, and sometimes take square roots. Scientific calculators can do many other things, such as use exponents (or powers or indices) and trigonometry functions.

Code:

```
from tkinter import *
from math import factorial, pi

root = Tk()
root.title('DataFlair - Calculator')
root.geometry('400x600') # Set a fixed window size

# Font settings for a larger display and buttons
display_font = ('Arial', 20)
button_font = ('Arial', 15)

# Adding the input field with a larger font
display = Entry(root, font=display_font, borderwidth=2, relief="solid")
display.grid(row=0, column=0, columnspan=5, sticky=N+E+W, padx=10, pady=10)

# Define a button creation function to avoid repetition
def create_button(text, row, col, command, rowspan=1, colspan=1):
    button = Button(root, text=text, font=button_font, command=command, height=2, width=5)
    button.grid(row=row, column=col, rowspan=rowspan, columnspan=colspan, sticky=N+S+E+W, padx=5, pady=5)

# Adding number buttons
create_button("1", 1, 0, lambda: get_variables(1))
create_button("2", 1, 1, lambda: get_variables(2))
create_button("3", 1, 2, lambda: get_variables(3))
create_button("4", 2, 0, lambda: get_variables(4))
create_button("5", 2, 1, lambda: get_variables(5))
create_button("6", 2, 2, lambda: get_variables(6))
create_button("7", 3, 0, lambda: get_variables(7))
create_button("8", 3, 1, lambda: get_variables(8))
create_button("9", 3, 2, lambda: get_variables(9))
create_button("0", 4, 1, lambda: get_variables(0))

# Adding operation buttons
create_button("+", 1, 3, lambda: get_operation("+"))
create_button("-", 2, 3, lambda: get_operation("-"))
create_button("*", 3, 3, lambda: get_operation("*"))
create_button("/", 4, 3, lambda: get_operation("/"))
create_button("pi", 1, 4, lambda: get_operation(str(pi)))
create_button("%", 2, 4, lambda: get_operation("%"))
create_button("(", 3, 4, lambda: get_operation("("))
create_button(")", 4, 4, lambda: get_operation("))"))
create_button("exp", 1, 5, lambda: get_operation("**"))
create_button("^2", 2, 5, lambda: get_operation("**2"))
create_button("x!", 3, 5, lambda: fact())
create_button(".", 4, 2, lambda: get_variables("."))
create_button("AC", 4, 0, lambda: clear_all())
```

Fig 5.26.1 python code

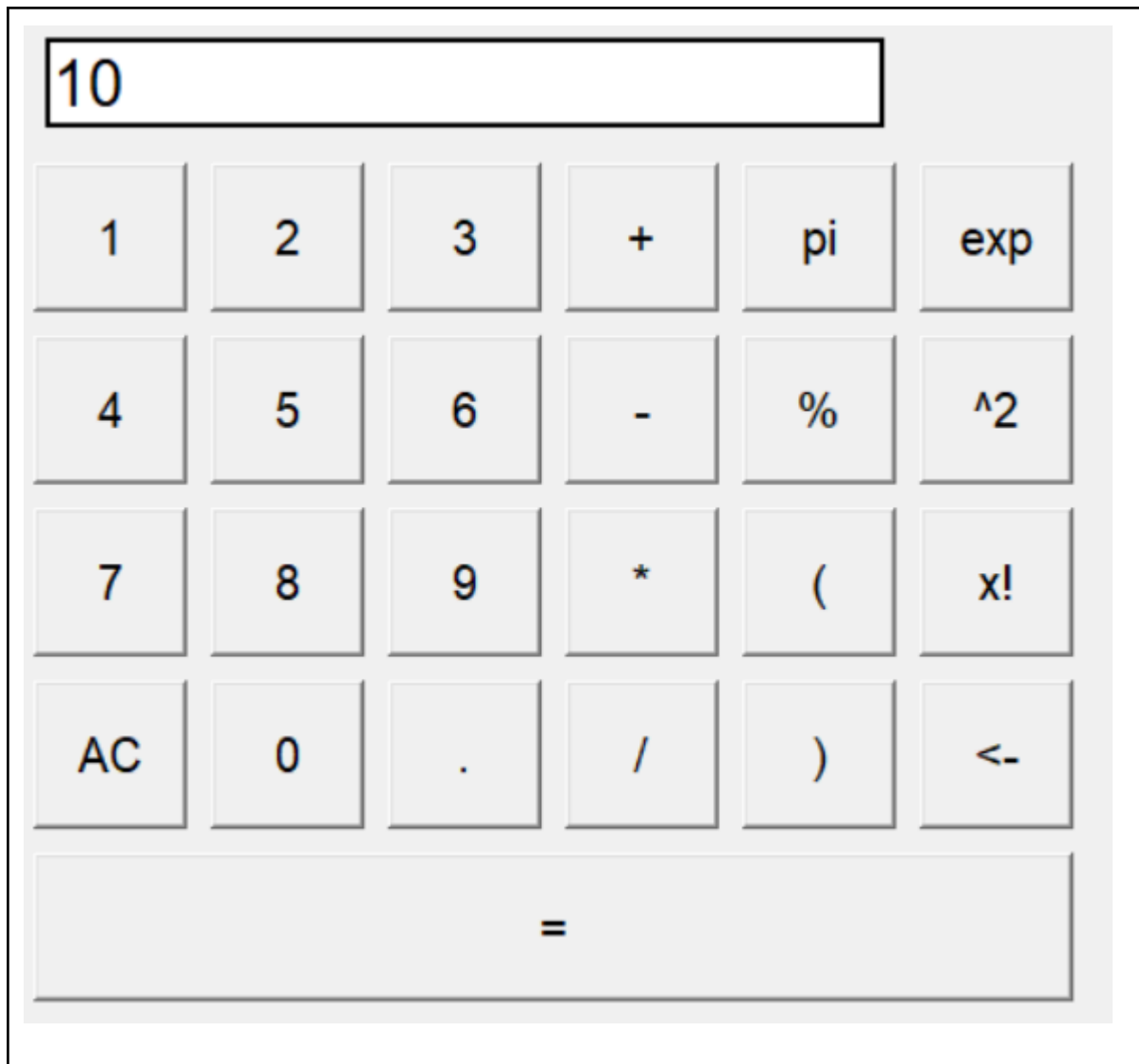


Fig 5.26.2 output

More details and objects can be found here :

<https://github.com/quantanics2018/Python/tree/main/Calculator>

5.27) Setting a Alarm Using Code

Today we usually think of an *alarm* as a loud noise that awakens us or warns us of danger. Its first use, however, was in Italy as a call to arms to soldiers. The Italian phrase *all'arme!* means literally “to arms” or “to your weapons.” The call was borrowed into other languages and came to be shortened to *alarme* in early French and Middle English. The word also came to be used as the name for the cry, then for any warning, and then to any device used to sound a warning, such as a bell or a gun. Since an alarm can cause fright or worry, such feelings also came to be known as *alarm*. By the 17th century, the word was used as a verb,

meaning “to warn of danger” and then “to frighten.”

```
from tkinter import *
import datetime
import time
import winsound

def alarm(set_alarm_timer):
    while True:
        time.sleep(1)
        current_time = datetime.datetime.now()
        now = current_time.strftime("%H:%M:%S")
        date = current_time.strftime("%d/%m/%Y")
        print("The Set Date is:", date)
        print(now)
        if now == set_alarm_timer:
            print("Time to Wake up")
            winsound.PlaySound("sound.wav", winsound.SND_ASYNC)
            break

def actual_time():
    set_alarm_timer = f"{hour.get()}:{min.get()}:{sec.get()}"
    alarm(set_alarm_timer)

clock = Tk()
clock.title("DataFlair Alarm Clock")
clock.geometry("1000x500")

addTime = Label(clock, text="Hour Min Sec", font=60)
addTime.place(x=110, y=100)

setYourAlarm = Label(clock, text="When to wake you up", fg="blue", relief="solid", font=("Helvetica", 15, "bold"))
setYourAlarm.place(x=0, y=2)

# The Variables we require to set the alarm(initialization):
hour = StringVar()
min = StringVar()
sec = StringVar()

# Time required to set the alarm clock:
hourTime = Entry(clock, textvariable=hour, bg="white", width=30)
hourTime.place(x=110, y=30)
minTime = Entry(clock, textvariable=min, bg="white", width=30)
minTime.place(x=150, y=30)
secTime = Entry(clock, textvariable=sec, bg="white", width=30)
secTime.place(x=200, y=30)

# To take the time input by user:
submit = Button(clock, text="Set Alarm", fg="red", width=10, command=actual_time)
```

fig 5.27.1 python code

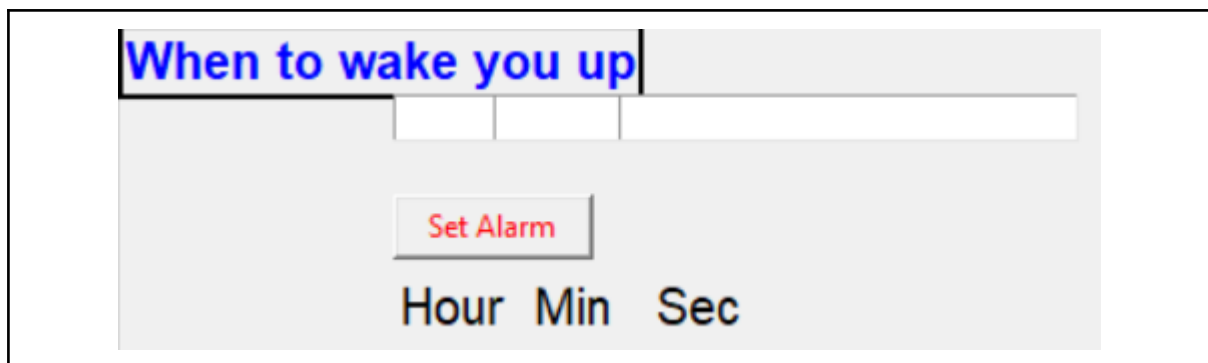


fig 5.27.2 output

More details and objects can be found [here](#)